

Contents and provenance

This listing includes application code, matching/workflow logic, database schema, tests and authored build/evaluation scripts. Vendored UI primitives, build scaffolding and package lockfiles are available in the repository. The MIT license applies to original code; dependencies retain their own licenses.

```
.github/workflows/ci.yml
app/api/workspace/route.ts
app/assets.d.ts
app/chatgpt-auth.ts
app/globals.css
app/layout.tsx
app/lotlight.tsx
app/page.tsx
db/index.ts
db/schema.ts
drizzle/0000_real_hellcat.sql
lib/lotlight/commands.ts
lib/lotlight/engine.ts
lib/lotlight/files.ts
lib/lotlight/sample.ts
lib/lotlight/semantic-worker.ts
lib/lotlight/storage.ts
lib/lotlight/types.ts
package.json
playwright.config.ts
scripts/build-ai.mjs
scripts/check-accessibility.mjs
scripts/check-api-isolation.mjs
scripts/create-submission-pdfs.py
scripts/evaluate-ai.mjs
scripts/execution-profile.mjs
scripts/install-ci.mjs
scripts/pnpm-install.mjs
scripts/record-demo.mjs
scripts/run-framework.mjs
scripts/sites-env.mjs
tests/e2e/accessibility.spec.ts
tests/e2e/workflow.spec.ts
tests/engine.test.ts
vite.config.ts
```

.github/workflows/ci.yml

```

1 name: Lotlight checks
2 on:
3   push:
4   pull_request:
5 jobs:
6   validate:
7     runs-on: ubuntu-latest
8     steps:
9       - uses: actions/checkout@v4
10      - uses: actions/setup-node@v4
11        with:
12          node-version: 22
13          cache: npm
14      - run: npm ci
15      - run: npm run typecheck
16      - run: npm test
17      - run: npm run build
18      - run: npx playwright install --with-deps chromium
19      - name: Initialize isolated test database
20        run: node --import ./scripts/sites-env.mjs ./node_modules/wrangler/bin/wrangler.js d1 execute DB
--local --config dist/server/wrangler.json --persist-to .wrangler/state --file drizzle/0000_real_hellcat.sql
21      - name: Browser workflows
22        run: |
23          npm run dev -- --port 5173 > /tmp/lotlight-dev.log 2>&1 &
24          for i in $(seq 1 60); do
25            if curl --fail http://localhost:5173/ > /dev/null 2>&1; then break; fi
26            sleep 1
27          done
28          LOTLIGHT_TEST_URL=http://localhost:5173 npm run test:e2e
29      - uses: actions/upload-artifact@v4
30        if: failure()
31        with:
32          name: browser-failure-details
33          path: test-results/

```

app/api/workspace/route.ts

```

1 import { getChatGPTUser } from "@app/chatgpt-auth";
2 import { readWorkspace, writeWorkspace } from "@lib/lotlight/storage";
3 import { reduceWorkspace } from "@lib/lotlight/commands";
4 import { z } from "zod";
5 export const dynamic = "force-dynamic";
6 const reply = (body: unknown, status = 200) =>
7   Response.json(body, {
8     status,
9     headers: {
10       "Cache-Control": "no-store",
11       "X-Content-Type-Options": "nosniff",
12     },
13   });
14 export async function GET() {
15   const user = await getChatGPTUser();
16   if (!user)
17     return reply({ error: "Sign in to access a saved workspace." }, 401);
18   try {
19     return reply(await readWorkspace(user.userId));
20   } catch (error) {
21     console.error(
22       "workspace_read_failed",
23       error instanceof Error ? error.name : "unknown",
24     );
25     return reply(
26       {
27         error:
28           "Your saved workspace is unavailable. Retry without changing your inputs.",
29       },
30       503,
31     );
32   }
33 }
34 export async function POST(request: Request) {
35   const user = await getChatGPTUser();
36   if (!user) return reply({ error: "Sign in to save your workspace." }, 401);
37   const origin = request.headers.get("origin");
38   if (!origin || origin !== new URL(request.url).origin)
39     return reply({ error: "Request origin rejected." }, 403);
40   if (!request.headers.get("content-type")?.startsWith("application/json"))
41     return reply({ error: "Use a JSON request." }, 415);
42   if (Number(request.headers.get("content-length") ?? 0) > 600000)
43     return reply({ error: "Request exceeds the size limit." }, 413);
44   let payload: { revision: number; command?: unknown };
45   try {
46     const body = await request.text();
47     if (body.length > 600000)
48       return reply({ error: "Request exceeds the size limit." }, 413);
49     payload = z
50       .object({ revision: z.number().int().min(0), command: z.unknown() })
51       .strict()
52       .parse(JSON.parse(body));
53   } catch {
54     return reply({ error: "Invalid request." }, 400);
55   }
56   try {
57     const state = await readWorkspace(user.userId);
58     if (state.revision !== payload.revision)
59       return reply(
60         {
61           error:
62             "This workspace changed in another tab. Reload the latest version before saving.",
63         },
64         409,
65       );
66     let next;
67     try {
68       next = reduceWorkspace(state, payload.command, user.displayName);
69     } catch (error) {
70       return reply(
71         {
72           error:
73             error instanceof z.ZodError
74               ? error.issues.map((i) => i.message).join(" ")
75               : error instanceof Error
76                 ? error.message
77                 : "Invalid update.",
78         },
79         400,
80       );

```

app/api/workspace/route.ts (continued)

```
81     }
82     if (JSON.stringify(next).length > 3000000)
83         return reply(
84             {
85                 error:
86                     "Workspace storage limit reached. Export your audit record before starting a new workspace.",
87             },
88             413,
89         );
90     if (!(await writeWorkspace(user.userId, payload.revision, next)))
91         return reply(
92             { error: "Another update arrived first. Reload before retrying." },
93             409,
94         );
95     return reply(next);
96 } catch (error) {
97     console.error(
98         "workspace_save_failed",
99         error instanceof Error ? error.name : "unknown",
100     );
101     return reply(
102         {
103             error:
104                 "Could not save. Your previous saved version is intact. Please retry.",
105         },
106         503,
107     );
108 }
109 }
```

app/assets.d.ts

```
1 declare module '*?url' { const url: string; export default url; }  
2  
3 declare module '*?worker' { const WorkerConstructor: {new():Worker}; export default WorkerConstructor; }
```

app/chatgpt-auth.ts

```

1 import { headers } from "next/headers";
2 import { redirect } from "next/navigation";
3
4 export type ChatGPTUser = {
5   userId: string;
6   displayName: string;
7   email: string;
8   fullName: string | null;
9 };
10
11 const USER_ID_HEADER = "oai-authenticated-user-id";
12 const USER_EMAIL_HEADER = "oai-authenticated-user-email";
13 const USER_FULL_NAME_HEADER = "oai-authenticated-user-full-name";
14 const USER_FULL_NAME_ENCODING_HEADER =
15   "oai-authenticated-user-full-name-encoding";
16 const PERCENT_ENCODED_UTF8 = "percent-encoded-utf-8";
17 const SIGN_IN_PATH = "/signin-with-chatgpt";
18 const SIGN_OUT_PATH = "/signout-with-chatgpt";
19 const CALLBACK_PATH = "/callback";
20
21 export async function getChatGPTUser(): Promise<ChatGPTUser | null> {
22   const requestHeaders = await headers();
23   const userId = requestHeaders.get(USER_ID_HEADER);
24   const email = requestHeaders.get(USER_EMAIL_HEADER);
25   if (!userId || !email) return null;
26
27   const encodedFullName = requestHeaders.get(USER_FULL_NAME_HEADER);
28   const fullName =
29     encodedFullName &&
30     requestHeaders.get(USER_FULL_NAME_ENCODING_HEADER) === PERCENT_ENCODED_UTF8
31       ? safeDecodeURIComponent(encodedFullName)
32       : null;
33
34   return {
35     userId,
36     displayName: fullName ?? email,
37     email,
38     fullName,
39   };
40 }
41
42 export async function requireChatGPTUser(
43   returnTo: string,
44 ): Promise<ChatGPTUser> {
45   const user = await getChatGPTUser();
46   if (user) return user;
47
48   redirect(chatGPTSignInPath(returnTo));
49 }
50
51 export function chatGPTSignInPath(returnTo: string): string {
52   const safeReturnTo = safeRelativeReturnPath(returnTo);
53   return `${SIGN_IN_PATH}?return_to=${encodeURIComponent(safeReturnTo)}`;
54 }
55
56 export function chatGPTSignOutPath(returnTo = "/"): string {
57   const safeReturnTo = safeRelativeReturnPath(returnTo);
58   return `${SIGN_OUT_PATH}?return_to=${encodeURIComponent(safeReturnTo)}`;
59 }
60
61 function safeRelativeReturnPath(value: string): string {
62   if (!value.startsWith("/") || value.startsWith("//")) return "/";
63
64   let url: URL;
65   try {
66     url = new URL(value, "https://app.local");
67   } catch {
68     return "/";
69   }
70   if (url.origin !== "https://app.local") return "/";
71   if (isReservedAuthPath(url.pathname)) return "/";
72
73   return `${url.pathname}${url.search}${url.hash}`;
74 }
75
76 function isReservedAuthPath(pathname: string): boolean {
77   return (
78     pathname === SIGN_IN_PATH ||
79     pathname === SIGN_OUT_PATH ||
80     pathname === CALLBACK_PATH

```

app/chatgpt-auth.ts (continued)

```
81   );  
82 }  
83  
84 function safeDecodeURIComponent(value: string): string | null {  
85   try {  
86     return decodeURIComponent(value);  
87   } catch {  
88     return null;  
89   }  
90 }
```

app/globals.css

```

1 @import "tailwindcss";
2 @import "tw-animate-css";
3 @import "../vendor/shadcn-tailwind-4.13.0.css";
4
5 @utility scrollbar-thin {
6   scrollbar-width: thin;
7 }
8
9 @utility scrollbar-none {
10   scrollbar-width: none;
11 }
12 &::-webkit-scrollbar {
13   display: none;
14 }
15 }
16
17 @utility scrollbar-gutter-stable {
18   scrollbar-gutter: stable;
19 }
20
21 :root {
22   --background: #f8f9f5;
23   --foreground: #263329;
24   --card: #ffffff;
25   --card-foreground: #171717;
26   --popover: #ffffff;
27   --popover-foreground: #171717;
28   --primary: #253c2d;
29   --primary-foreground: #fafafa;
30   --secondary: #f5f5f5;
31   --secondary-foreground: #171717;
32   --muted: #f5f5f5;
33   --muted-foreground: #697469;
34   --accent: #f5f5f5;
35   --accent-foreground: #171717;
36   --destructive: #e7000b;
37   --border: #e0e4da;
38   --input: #e5e5e5;
39   --ring: #a1a1a1;
40   --chart-1: #f54900;
41   --chart-2: #009689;
42   --chart-3: #104e64;
43   --chart-4: #ffb900;
44   --chart-5: #fe9a00;
45   --radius: 0.625rem;
46   --sidebar: #f1f3ed;
47   --sidebar-foreground: #263329;
48   --sidebar-primary: #171717;
49   --sidebar-primary-foreground: #fafafa;
50   --sidebar-accent: #f5f5f5;
51   --sidebar-accent-foreground: #171717;
52   --sidebar-border: #e5e5e5;
53   --sidebar-ring: #a1a1a1;
54 }
55
56 @theme inline {
57   --color-background: var(--background);
58   --color-foreground: var(--foreground);
59   --color-card: var(--card);
60   --color-card-foreground: var(--card-foreground);
61   --color-popover: var(--popover);
62   --color-popover-foreground: var(--popover-foreground);
63   --color-primary: var(--primary);
64   --color-primary-foreground: var(--primary-foreground);
65   --color-secondary: var(--secondary);
66   --color-secondary-foreground: var(--secondary-foreground);
67   --color-muted: var(--muted);
68   --color-muted-foreground: var(--muted-foreground);
69   --color-accent: var(--accent);
70   --color-accent-foreground: var(--accent-foreground);
71   --color-destructive: var(--destructive);
72   --color-border: var(--border);
73   --color-input: var(--input);
74   --color-ring: var(--ring);
75   --color-chart-1: var(--chart-1);
76   --color-chart-2: var(--chart-2);
77   --color-chart-3: var(--chart-3);
78   --color-chart-4: var(--chart-4);
79   --color-chart-5: var(--chart-5);
80   --color-sidebar: var(--sidebar);

```

app/globals.css (continued)

```

81  --color-sidebar-foreground: var(--sidebar-foreground);
82  --color-sidebar-primary: var(--sidebar-primary);
83  --color-sidebar-primary-foreground: var(--sidebar-primary-foreground);
84  --color-sidebar-accent: var(--sidebar-accent);
85  --color-sidebar-accent-foreground: var(--sidebar-accent-foreground);
86  --color-sidebar-border: var(--sidebar-border);
87  --color-sidebar-ring: var(--sidebar-ring);
88  --radius-sm: calc(var(--radius) - 4px);
89  --radius-md: calc(var(--radius) - 2px);
90  --radius-lg: var(--radius);
91  --radius-xl: calc(var(--radius) + 4px);
92  --font-sans: Arial, Helvetica, sans-serif;
93  --font-mono:
94      ui-monospace, SFMono-Regular, Menlo, Monaco, Consolas, "Liberation Mono",
95      "Courier New", monospace;
96 }
97
98 body {
99     margin: 0;
100    background: var(--background);
101    color: var(--foreground);
102    font-family: Arial, Helvetica, sans-serif;
103    -webkit-font-smoothing: antialiased;
104 }
105 button,
106 a,
107 input,
108 textarea {
109     -webkit-tap-highlight-color: transparent;
110 }
111 button,
112 a {
113     touch-action: manipulation;
114 }
115 button {
116     cursor: pointer;
117 }
118 button:disabled {
119     cursor: not-allowed;
120     opacity: 0.5;
121 }
122 a {
123     text-decoration: none;
124 }
125 button:focus-visible,
126 a:focus-visible,
127 input:focus-visible,
128 textarea:focus-visible {
129     outline: 3px solid #94aa45;
130     outline-offset: 3px;
131 }
132 .lot-sidebar {
133     border-right: 1px solid #e0e4da;
134 }
135 .lot-sidebar [data-slot="sidebar-inner"] {
136     padding: 24px 14px;
137     background: #f1f3ed;
138 }
139 .brand {
140     display: flex;
141     align-items: center;
142     gap: 10px;
143     font-size: 29px;
144     font-weight: 700;
145     letter-spacing: -1.4px;
146     padding: 0 6px;
147 }
148 .brand-mark {
149     display: grid;
150     place-items: center;
151     width: 36px;
152     height: 38px;
153     background: #ddec8b;
154     border-radius: 11px;
155 }
156 .brand-dot {
157     font-size: 10px;
158     align-self: flex-start;
159     margin-top: 7px;
160     letter-spacing: 0;

```

app/globals.css (continued)

```

161 }
162 .workspace-label {
163   margin: 35px 0 27px;
164   display: flex;
165   align-items: center;
166   gap: 9px;
167   font-size: 11px;
168   font-weight: 600;
169   letter-spacing: -0.15px;
170 }
171 .workspace-avatar {
172   background: white;
173   border: 1px solid #e0e4da;
174   border-radius: 8px;
175   padding: 9px;
176   color: #58694b;
177 }
178 .workspace-label small,
179 .creator small {
180   display: block;
181   color: #58694b;
182   font-size: 10px;
183   font-weight: 400;
184   margin-top: 5px;
185   letter-spacing: 0;
186 }
187 .nav-caption {
188   font-size: 10px;
189   font-weight: 600;
190   letter-spacing: 1.8px;
191   color: #58694b;
192   padding: 0 12px 15px;
193 }
194 .lot-sidebar [data-slot="sidebar-menu-button"] {
195   height: 44px;
196   gap: 12px;
197   padding: 0 12px;
198   font-size: 12px;
199   color: #58694b;
200 }
201 .lot-sidebar .nav-active {
202   background: #e1e7d7 !important;
203   color: #2d3d2a !important;
204   font-weight: 600;
205 }
206 .nav-count {
207   margin-left: auto;
208   background: #fafbf7;
209   padding: 1px 7px;
210   border-radius: 4px;
211   font-size: 10px;
212 }
213 .side-note {
214   margin: 42px 12px 15px;
215   padding-top: 22px;
216   border-top: 1px solid #dce1d4;
217 }
218 .side-note svg {
219   color: #58694b;
220 }
221 .side-note strong {
222   display: block;
223   font-size: 13px;
224   line-height: 1.6;
225   margin-top: 12px;
226   font-weight: 500;
227 }
228 .side-note p {
229   font-size: 11px;
230   line-height: 1.8;
231   color: #58694b;
232   margin-top: 9px;
233 }
234 .creator {
235   display: flex;
236   align-items: center;
237   gap: 10px;
238   font-size: 11px;
239   border-top: 1px solid #dce1d4;
240   padding-top: 20px;

```

app/globals.css (continued)

```
241 }
242 .creator > span {
243   border-radius: 50%;
244   background: #e2e8d6;
245   padding: 10px;
246   font-size: 10px;
247 }
248 .app-shell {
249   width: calc(100% - 224px);
250   min-width: 0;
251   flex: 1;
252 }
253 .topbar {
254   height: 75px;
255   border-bottom: 1px solid #e4e7df;
256   display: flex;
257   align-items: center;
258   justify-content: space-between;
259   padding: 0 44px;
260   font-size: 11px;
261   background: #fcfdf9;
262 }
263 .topbar > div {
264   display: flex;
265   align-items: center;
266   gap: 15px;
267   color: #58694b;
268 }
269 .topbar b {
270   font-weight: 500;
271   color: #495443;
272 }
273 .top-account {
274   display: flex;
275   align-items: center;
276   gap: 8px;
277   color: #4f5e47;
278 }
279 .mobile-menu {
280   display: none;
281 }
282 .main-content {
283   max-width: 1390px;
284   padding: 45px 44px 22px;
285   margin: auto;
286 }
287 .page-heading {
288   display: flex;
289   justify-content: space-between;
290   align-items: flex-end;
291   margin-bottom: 32px;
292 }
293 .eyebrow {
294   font-size: 10px;
295   letter-spacing: 1.8px;
296   font-weight: 700;
297   display: flex;
298   align-items: center;
299   gap: 8px;
300 }
301 .eyebrow > span {
302   width: 6px;
303   height: 6px;
304   border-radius: 50%;
305   background: #7c9646;
306 }
307 h1 {
308   font-size: clamp(36px, 3.8vw, 53px);
309   line-height: 1.09;
310   letter-spacing: -2.1px;
311   font-weight: 500;
312   margin: 19px 0 16px;
313 }
314 .intro {
315   font-size: 13px;
316   line-height: 1.9;
317   color: #58694b;
318 }
319 .heading-actions {
320   display: flex;
```

app/globals.css (continued)

```
321 flex-direction: column;
322 align-items: flex-end;
323 gap: 17px;
324 padding-bottom: 4px;
325 }
326 .demo-label {
327   font-size: 10px;
328   letter-spacing: 1.4px;
329   color: #58694b;
330   border: 1px solid #dce3d2;
331   border-radius: 5px;
332   padding: 6px 8px;
333   background: #f2f5eb;
334 }
335 .button {
336   display: inline-flex;
337   align-items: center;
338   justify-content: center;
339   gap: 8px;
340   border: 1px solid #d9e0d1;
341   padding: 11px 15px;
342   font-size: 11px;
343   font-weight: 500;
344   border-radius: 7px;
345   min-height: 39px;
346   white-space: nowrap;
347   background: white;
348   transition: background 0.15s;
349 }
350 .button:hover {
351   background: #edf1e7;
352 }
353 .button.primary {
354   background: #2b432f;
355   color: #fff;
356   border-color: #2b432f;
357 }
358 .button.primary:hover {
359   background: #38543b;
360 }
361 .overview-strip {
362   display: grid;
363   grid-template-columns: repeat(4, 1fr);
364   border-top: 1px solid #e0e4da;
365   border-bottom: 1px solid #e0e4da;
366   padding: 23px 0;
367   margin: 32px 0 39px;
368 }
369 .overview-strip > div {
370   padding: 0 24px;
371   border-right: 1px solid #e0e4da;
372 }
373 .overview-strip > div:first-child {
374   padding-left: 0;
375 }
376 .overview-strip > div:last-child {
377   border: 0;
378 }
379 .overview-strip span {
380   font-size: 10px;
381   color: #58694b;
382   display: block;
383   margin-bottom: 12px;
384 }
385 .overview-strip strong {
386   font-size: 32px;
387   font-weight: 400;
388   display: flex;
389   align-items: baseline;
390   gap: 13px;
391   letter-spacing: -1px;
392 }
393 .overview-strip small {
394   font-size: 10px;
395   letter-spacing: 0;
396   color: #58694b;
397   max-width: 90px;
398   line-height: 1.5;
399 }
400 .orange-text {
```

app/globals.css (continued)

```
401   color: #58694b;
402 }
403 .section-heading {
404   display: flex;
405   justify-content: space-between;
406   align-items: center;
407   margin-bottom: 17px;
408 }
409 .section-heading h2 {
410   font-size: 14px;
411   font-weight: 500;
412   display: flex;
413   align-items: center;
414   gap: 12px;
415 }
416 .section-heading h2 span {
417   color: #58694b;
418   font-size: 10px;
419 }
420 .section-heading > span {
421   font-size: 10px;
422   color: #58694b;
423 }
424 .review-grid {
425   display: grid;
426   grid-template-columns: minmax(0, 1.9fr) minmax(250px, 1fr);
427   gap: 22px;
428 }
429 .review-card {
430   background: white;
431   border: 1px solid #dce2d4;
432   border-radius: 11px;
433   padding: 25px 26px 0;
434   box-shadow: 0 3px 5px #2b432f03;
435   overflow: hidden;
436 }
437 .card-topline {
438   display: flex;
439   align-items: center;
440   justify-content: space-between;
441   gap: 10px;
442 }
443 .status-tag {
444   font-size: 10px;
445   font-weight: 600;
446   letter-spacing: 0.9px;
447   color: #58694b;
448   display: flex;
449   align-items: center;
450   gap: 6px;
451   background: #fff2e8;
452   border-radius: 5px;
453   padding: 6px 8px;
454 }
455 .status-tag > span {
456   width: 4px;
457   height: 4px;
458   border-radius: 50%;
459   background: #b57343;
460 }
461 .mono,
462 code {
463   font-family: ui-monospace, SFMono-Regular, monospace;
464   font-size: 10px;
465   color: #58694b;
466 }
467 .review-card h2 {
468   font-size: 23px;
469   letter-spacing: -0.8px;
470   font-weight: 500;
471   margin: 25px 0 10px;
472 }
473 .review-card > p {
474   font-size: 11px;
475   color: #58694b;
476 }
477 .scope-line {
478   display: flex;
479   gap: 8px;
480   align-items: center;
```

app/globals.css (continued)

```
481   margin: 21px 0 24px;
482 }
483 .scope-line > span {
484   font-size: 10px;
485   color: #58694b;
486   margin-right: 10px;
487 }
488 .scope-line code {
489   color: #58694b;
490   border: 1px solid #e4e8de;
491   background: #fafbf8;
492   padding: 4px 7px;
493   border-radius: 4px;
494 }
495 .match-visual {
496   background: #f6f8f1;
497   padding: 17px;
498   border-radius: 6px;
499 }
500 .match-bars {
501   display: flex;
502   gap: 5px;
503   margin-bottom: 11px;
504 }
505 .match-bars > span {
506   height: 8px;
507   flex: 1;
508   background: #dfe5d4;
509   border-radius: 2px;
510 }
511 .match-bars > span:nth-child(-n + 2) {
512   background: #c9865a;
513 }
514 .match-bars > span:nth-child(n + 3):nth-child(-n + 4) {
515   background: #d9ca80;
516 }
517 .match-visual > span {
518   font-size: 10px;
519   color: #58694b;
520 }
521 .match-visual b {
522   color: #556249;
523   font-weight: 600;
524 }
525 .match-visual > span > span {
526   margin: 0 8px;
527 }
528 .card-bottom {
529   display: flex;
530   justify-content: space-between;
531   align-items: center;
532   margin-top: 24px;
533   padding: 17px 0;
534   border-top: 1px solid #edf0e7;
535 }
536 .card-bottom > span {
537   display: flex;
538   align-items: center;
539   gap: 7px;
540   font-size: 10px;
541   color: #58694b;
542 }
543 .text-button {
544   display: flex;
545   align-items: center;
546   gap: 8px;
547   font-size: 11px;
548   color: #455b35;
549   font-weight: 600;
550 }
551 .principle-card {
552   background: #e6eec7;
553   border: 1px solid #dee8b8;
554   border-radius: 11px;
555   padding: 22px 25px;
556   display: flex;
557   flex-direction: column;
558 }
559 .mini-orbit {
560   height: 49px;
```

app/globals.css (continued)

```
561 position: relative;
562 margin-bottom: 21px;
563 display: flex;
564 align-items: center;
565 }
566 .mini-orbit svg {
567   color: #58694b;
568 }
569 .mini-orbit span {
570   position: absolute;
571   border: 1px solid #b7c98b;
572   border-radius: 50%;
573   width: 50px;
574   height: 50px;
575   left: -12px;
576 }
577 .mini-orbit span:last-child {
578   left: 8px;
579   border-color: #c9d5a9;
580 }
581 .principle-card .eyebrow {
582   font-size: 9px;
583   letter-spacing: 1.5px;
584   color: #58694b;
585 }
586 .principle-card h3 {
587   font-size: 25px;
588   letter-spacing: -0.8px;
589   line-height: 1.18;
590   font-weight: 400;
591   margin: 13px 0;
592 }
593 .principle-card > p:not(.eyebrow) {
594   font-size: 11px;
595   color: #58694b;
596   line-height: 1.8;
597   max-width: 225px;
598 }
599 .principle-footer {
600   display: flex;
601   justify-content: space-between;
602   align-items: center;
603   font-size: 10px;
604   letter-spacing: 1.2px;
605   color: #58694b;
606   margin-top: auto;
607   padding-top: 23px;
608 }
609 .bottom-explainer {
610   display: grid;
611   grid-template-columns: repeat(3, 1fr);
612   gap: 25px;
613   border-top: 1px solid #e3e7db;
614   margin-top: 36px;
615   padding: 23px 0 30px;
616 }
617 .step-number {
618   font-size: 10px;
619   color: #58694b;
620 }
621 .bottom-explainer h3 {
622   font-size: 12px;
623   margin: 8px 0;
624   color: #58694b;
625   font-weight: 500;
626 }
627 .bottom-explainer p {
628   font-size: 10px;
629   color: #58694b;
630   line-height: 1.7;
631 }
632 .page-footer {
633   display: flex;
634   justify-content: space-between;
635   gap: 16px;
636   border-top: 1px solid #e4e8de;
637   padding-top: 20px;
638   font-size: 10px;
639   color: #58694b;
640   line-height: 1.8;
```

app/globals.css (continued)

```
641 }
642 .page-footer > span:first-child {
643   display: flex;
644   align-items: center;
645   gap: 6px;
646 }
647 @media (min-width: 1500px) {
648   .main-content {
649     padding-top: 65px;
650   }
651   .review-card {
652     padding-top: 32px;
653   }
654   .principle-card {
655     padding-top: 30px;
656   }
657   .review-card h2 {
658     margin-top: 31px;
659   }
660 }
661 @media (max-width: 1100px) {
662   .main-content {
663     padding: 30px 25px;
664   }
665   .topbar {
666     padding: 0 25px;
667   }
668   .overview-strip strong {
669     flex-direction: column;
670     gap: 3px;
671   }
672   .overview-strip small {
673     max-width: none;
674   }
675   .review-grid {
676     grid-template-columns: minmax(0, 1fr);
677   }
678   .principle-card {
679     display: none;
680   }
681   .overview-strip > div {
682     padding: 0 13px;
683   }
684 }
685 @media (max-width: 767px) {
686   .mobile-menu {
687     display: flex;
688   }
689   .app-shell {
690     width: 100%;
691   }
692   .topbar {
693     padding: 0 18px;
694     height: 60px;
695   }
696   .main-content {
697     padding: 28px 19px;
698   }
699   .page-heading {
700     display: block;
701   }
702   .heading-actions {
703     align-items: flex-start;
704     flex-direction: row;
705     align-items: center;
706     margin-top: 22px;
707   }
708   .demo-label {
709     order: 2;
710   }
711   h1 {
712     font-size: 43px;
713   }
714   .overview-strip {
715     grid-template-columns: 1fr 1fr;
716     gap: 20px;
717     margin: 25px 0;
718   }
719   .overview-strip > div {
720     padding: 0 !important;
```

app/globals.css (continued)

```
721     border: 0;
722   }
723   .overview-strip strong {
724     flex-direction: row;
725     align-items: baseline;
726     font-size: 29px;
727     gap: 9px;
728   }
729   .overview-strip small {
730     max-width: 85px;
731   }
732   .section-heading {
733     align-items: flex-start;
734     gap: 15px;
735   }
736   .section-heading > span {
737     max-width: 115px;
738     font-size: 10px;
739     text-align: right;
740   }
741   .review-card {
742     padding: 20px 18px 0;
743   }
744   .review-card h2 {
745     font-size: 22px;
746   }
747   .bottom-explainer {
748     gap: 15px;
749   }
750   .bottom-explainer h3 {
751     line-height: 1.5;
752     font-size: 11px;
753   }
754   .page-footer {
755     flex-direction: column;
756   }
757   .card-bottom > span {
758     font-size: 10px;
759   }
760   .scope-line {
761     flex-wrap: wrap;
762   }
763   .topbar > div {
764     gap: 9px;
765   }
766 }
767 .notice-stack {
768   display: grid;
769   gap: 20px;
770 }
771 .workspace-tools {
772   display: flex;
773   gap: 25px;
774   margin-bottom: 25px;
775 }
776 .muted {
777   color: #58694b;
778   font-size: 10px;
779 }
780 .account-area {
781   gap: 20px !important;
782 }
783 .save-status {
784   font-size: 10px;
785   color: #58694b;
786 }
787 .compact {
788   margin-bottom: 28px;
789   align-items: center;
790 }
791 .compact h1 {
792   font-size: 36px;
793   letter-spacing: -1.4px;
794   margin: 12px 0;
795 }
796 .back-button {
797   display: flex;
798   align-items: center;
799   gap: 7px;
800   font-size: 10px;
```

app/globals.css (continued)

```

801   color: #58694b;
802   margin-bottom: 23px;
803 }
804 .review-disclaimer,
805 .info-banner,
806 .error-banner {
807   display: flex;
808   gap: 10px;
809   align-items: flex-start;
810   background: #f0f3e7;
811   border: 1px solid #e0e7d0;
812   border-radius: 7px;
813   padding: 13px 16px;
814   font-size: 11px;
815   line-height: 1.8;
816   color: #58694b;
817   margin-bottom: 25px;
818 }
819 .review-disclaimer svg,
820 .info-banner svg,
821 .error-banner svg {
822   flex-shrink: 0;
823   margin-top: 2px;
824 }
825 .error-banner {
826   background: #fff2ea;
827   color: #58694b;
828   border-color: #ebc9b7;
829 }
830 .error-banner .text-button {
831   margin-left: auto;
832   white-space: nowrap;
833 }
834 .review-tabs > [data-slot="tabs-list"] {
835   width: 100%;
836   justify-content: flex-start;
837   border-bottom: 1px solid #dfe5d5;
838   padding: 0;
839   height: 48px;
840   margin-bottom: 22px;
841   gap: 30px;
842 }
843 .review-tabs [data-slot="tabs-trigger"] {
844   flex: 0 1 auto;
845   font-size: 12px;
846   padding: 0 0 13px;
847   border-radius: 0;
848   gap: 10px;
849 }
850 .review-tabs [data-slot="tabs-trigger"]:after {
851   background: #65823c;
852   height: 2px;
853   bottom: 0;
854 }
855 .tab-count {
856   font-size: 10px;
857   padding: 2px 5px;
858   background: #e9eee0;
859   border-radius: 3px;
860   color: #58694b;
861 }
862 .ai-banner {
863   display: flex;
864   align-items: center;
865   gap: 15px;
866   padding: 20px;
867   border: 1px solid #dbe5c9;
868   background: #f0f5e7;
869   border-radius: 9px;
870   margin-bottom: 25px;
871 }
872 .ai-icon {
873   background: #e2ebcf;
874   border-radius: 8px;
875   padding: 11px;
876   color: #58694b;
877 }
878 .ai-banner > div:nth-child(2) {
879   flex: 1;
880 }

```

app/globals.css (continued)

```

881 .ai-banner strong {
882   font-size: 12px;
883   font-weight: 500;
884 }
885 .ai-banner p {
886   font-size: 10px;
887   line-height: 1.8;
888   color: #58694b;
889   margin-top: 5px;
890   max-width: 590px;
891 }
892 .table-toolbar {
893   display: flex;
894   align-items: center;
895   justify-content: space-between;
896   gap: 15px;
897   margin: 16px 0;
898 }
899 .legend {
900   display: flex;
901   gap: 15px;
902   align-items: center;
903   font-size: 10px;
904   color: #58694b;
905 }
906 .legend > span {
907   display: flex;
908   gap: 5px;
909   align-items: center;
910 }
911 .legend i {
912   width: 5px;
913   height: 5px;
914   border-radius: 50%;
915   display: block;
916 }
917 .dot-exact {
918   background: #bd805a;
919 }
920 .dot-missing {
921   background: #bdab5b;
922 }
923 .dot-unlisted {
924   background: #afb00;
925 }
926 .search-box {
927   display: flex;
928   align-items: center;
929   gap: 7px;
930   border: 1px solid #dce4d2;
931   border-radius: 6px;
932   padding: 8px 10px;
933   background: white;
934   color: #58694b;
935 }
936 .search-box input {
937   outline: none;
938   border: none;
939   width: 140px;
940   font-size: 10px;
941   color: #536447;
942   background: none;
943 }
944 .match-row {
945   background: #fff;
946   border: 1px solid #dfe5d6;
947   border-radius: 8px;
948   overflow: hidden;
949   margin-bottom: 10px;
950 }
951 .match-row-main {
952   display: grid;
953   grid-template-columns: minmax(200px, 1fr) 135px 135px 30px;
954   gap: 15px;
955   align-items: center;
956   padding: 19px 18px;
957 }
958 .match-product {
959   display: flex;
960   gap: 13px;

```

app/globals.css (continued)

```
961 align-items: center;
962 }
963 .match-symbol {
964   padding: 10px;
965   border-radius: 7px;
966   background: #f3f5ee;
967   color: #58694b;
968 }
969 .match-symbol.exact {
970   background: #fcf1e9;
971   color: #58694b;
972 }
973 .match-symbol.missing,
974 .match-symbol.possible {
975   background: #faf5e1;
976   color: #58694b;
977 }
978 .match-product h3 {
979   font-size: 12px;
980   font-weight: 500;
981   line-height: 1.5;
982 }
983 .match-product p {
984   font-size: 10px;
985   color: #58694b;
986   margin-top: 6px;
987   line-height: 1.7;
988 }
989 .match-product p > span {
990   margin: 0 4px;
991 }
992 .match-identifiers {
993   display: flex;
994   flex-direction: column;
995   gap: 5px;
996 }
997 .match-identifiers code {
998   font-size: 11px;
999   color: #58694b;
1000   letter-spacing: 0.2px;
1001 }
1002 .match-identifiers > span {
1003   font-size: 10px;
1004   color: #58694b;
1005 }
1006 .match-identifiers b {
1007   font-family: ui-monospace, monospace;
1008   font-weight: 400;
1009   color: #58694b;
1010   font-size: 10px;
1011   margin-left: 6px;
1012 }
1013 .match-state {
1014   display: flex;
1015   flex-direction: column;
1016   align-items: flex-start;
1017   gap: 5px;
1018 }
1019 .pill {
1020   font-size: 10px;
1021   padding: 5px 7px;
1022   border-radius: 4px;
1023   line-height: 1.4;
1024   font-weight: 500;
1025   display: inline-flex;
1026   width: fit-content;
1027   background: #f0f3ea;
1028   color: #58694b;
1029 }
1030 .pill.exact {
1031   background: #fcf0e7;
1032   color: #58694b;
1033 }
1034 .pill.missing,
1035 .pill.possible {
1036   background: #f9f3db;
1037   color: #58694b;
1038 }
1039 .pill.complete {
1040   background: #eaf3e4;
```

app/globals.css (continued)

```
1041   color: #58694b;
1042 }
1043 .match-state small {
1044   font-size: 10px;
1045   color: #58694b;
1046 }
1047 .icon-button {
1048   padding: 6px;
1049   color: #58694b;
1050   border-radius: 5px;
1051 }
1052 .icon-button:hover {
1053   background: #eef2e5;
1054 }
1055 .match-expanded {
1056   display: flex;
1057   align-items: flex-start;
1058   justify-content: space-between;
1059   gap: 20px;
1060   border-top: 1px solid #ecf0e5;
1061   padding: 20px 22px;
1062   background: #fcfdf9;
1063 }
1064 .match-expanded > div:first-child {
1065   flex: 1;
1066 }
1067 .match-expanded strong {
1068   font-size: 10px;
1069   font-weight: 600;
1070   color: #58694b;
1071 }
1072 .match-expanded p {
1073   font-size: 11px;
1074   line-height: 1.8;
1075   margin-top: 7px;
1076   color: #58694b;
1077 }
1078 .match-expanded blockquote {
1079   font-size: 10px;
1080   font-family: ui-monospace, monospace;
1081   border-left: 2px solid #cad7b7;
1082   padding: 7px 12px;
1083   margin: 11px 0 0;
1084   color: #58694b;
1085   white-space: pre-wrap;
1086 }
1087 .gate-hint {
1088   font-size: 10px;
1089   display: block;
1090   color: #58694b;
1091   max-width: 150px;
1092   margin-top: 7px;
1093   line-height: 1.6;
1094 }
1095 .fine-print {
1096   font-size: 10px;
1097   line-height: 1.8;
1098   color: #58694b;
1099   margin: 15px 0 22px;
1100 }
1101 .source-grid {
1102   display: grid;
1103   grid-template-columns: 1.3fr 1fr;
1104   gap: 24px;
1105 }
1106 .source-document {
1107   background: #fff;
1108   border: 1px solid #dfe5d6;
1109   border-radius: 8px;
1110   padding: 23px;
1111 }
1112 .source-document-heading {
1113   display: flex;
1114   align-items: center;
1115   gap: 8px;
1116   border-bottom: 1px solid #edf1e5;
1117   padding-bottom: 17px;
1118   color: #58694b;
1119   font-size: 11px;
1120 }
```

app/globals.css (continued)

```
1121 .source-document-heading a {
1122   display: flex;
1123   align-items: center;
1124   gap: 5px;
1125   margin-left: auto;
1126   font-size: 10px;
1127   color: #58694b;
1128 }
1129 .source-document pre {
1130   white-space: pre-wrap;
1131   overflow-wrap: anywhere;
1132   font:
1133     11px/2 ui-monospace,
1134     monospace;
1135   color: #58694b;
1136   margin: 20px 0;
1137 }
1138 .source-document > .text-button {
1139   font-size: 10px;
1140   margin-top: 25px;
1141 }
1142 .source-review-panel {
1143   padding: 20px 23px;
1144   background: #f1f4e9;
1145   border: 1px solid #e0e6d4;
1146   border-radius: 8px;
1147 }
1148 .source-review-panel h3 {
1149   font-size: 21px;
1150   letter-spacing: -0.5px;
1151   margin: 16px 0 10px;
1152 }
1153 .source-review-panel > p {
1154   font-size: 11px;
1155   line-height: 1.8;
1156   color: #58694b;
1157 }
1158 .scope-review-row {
1159   display: grid;
1160   grid-template-columns: 1fr 1fr;
1161   gap: 8px;
1162   border-bottom: 1px solid #dee5d1;
1163   padding: 18px 0;
1164 }
1165 .scope-review-row strong {
1166   font:
1167     12px ui-monospace,
1168     monospace;
1169   color: #58694b;
1170 }
1171 .scope-review-row > span {
1172   font:
1173     10px ui-monospace,
1174     monospace;
1175   color: #58694b;
1176   text-align: right;
1177 }
1178 .scope-review-row small {
1179   grid-column: 1/-1;
1180   font-size: 10px;
1181   line-height: 1.7;
1182   color: #58694b;
1183 }
1184 .source-action {
1185   padding: 20px 0;
1186 }
1187 .source-action strong {
1188   font-size: 10px;
1189 }
1190 .source-action p {
1191   font-size: 11px;
1192   line-height: 1.8;
1193   color: #58694b;
1194   margin: 8px 0;
1195 }
1196 .source-action small {
1197   font-size: 10px;
1198   color: #58694b;
1199 }
1200 .check-label {
```

app/globals.css (continued)

```
1201 display: flex;
1202 align-items: flex-start;
1203 gap: 9px;
1204 font-size: 10px;
1205 line-height: 1.8;
1206 color: #58694b;
1207 margin: 15px 0;
1208 }
1209 .check-label [data-slot="checkbox"] {
1210   margin-top: 2px;
1211   flex-shrink: 0;
1212 }
1213 .full-width {
1214   width: 100%;
1215 }
1216 .approved-box {
1217   display: flex;
1218   gap: 10px;
1219   padding: 15px;
1220   background: #e4edda;
1221   border: 1px solid #d5e1c5;
1222   border-radius: 7px;
1223   color: #58694b;
1224 }
1225 .approved-box strong {
1226   font-size: 11px;
1227   display: block;
1228 }
1229 .approved-box small {
1230   font-size: 10px;
1231   line-height: 1.8;
1232   display: block;
1233   margin-top: 6px;
1234 }
1235 .status-tag.approved {
1236   color: #58694b;
1237   background: #eaf2df;
1238 }
1239 .status-tag.approved > span {
1240   background: #8d9f6f;
1241 }
1242 .data-table {
1243   border: 1px solid #dfe5d6;
1244   border-radius: 8px;
1245   background: white;
1246   overflow: hidden;
1247 }
1248 .data-table th {
1249   background: #f2f5ed;
1250   font-size: 10px;
1251   color: #58694b;
1252   letter-spacing: 0.4px;
1253   height: 43px;
1254   padding: 0 15px;
1255 }
1256 .data-table td {
1257   padding: 17px 15px;
1258   font-size: 10px;
1259   color: #58694b;
1260   border-color: #e9efdf;
1261 }
1262 .data-table .product-cell {
1263   color: #58694b;
1264   font-weight: 500;
1265   font-size: 11px;
1266 }
1267 .data-table code {
1268   font-size: 10px;
1269   color: #58694b;
1270 }
1271 .missing-text {
1272   color: #58694b !important;
1273 }
1274 .download-template {
1275   margin: 20px 0 50px;
1276   font-size: 10px;
1277 }
1278 .empty-state {
1279   text-align: center;
1280   padding: 55px 25px;
```

app/globals.css (continued)

```
1281 background: #f4f6ee;
1282 border: 1px dashed #d4ddc7;
1283 border-radius: 9px;
1284 margin-bottom: 25px;
1285 color: #58694b;
1286 }
1287 .empty-state > svg {
1288   margin: 0 auto 18px;
1289 }
1290 .empty-state h3 {
1291   font-size: 19px;
1292   color: #58694b;
1293   font-weight: 400;
1294   margin-bottom: 10px;
1295 }
1296 .empty-state p {
1297   font-size: 11px;
1298   line-height: 1.8;
1299   max-width: 420px;
1300   margin: auto;
1301 }
1302 .empty-state .button {
1303   margin-top: 20px;
1304 }
1305 .action-cards {
1306   display: grid;
1307   gap: 13px;
1308 }
1309 .action-card {
1310   padding: 20px;
1311   border: 1px solid #dfe6d3;
1312   background: white;
1313   border-radius: 9px;
1314   display: flex;
1315   justify-content: space-between;
1316   gap: 20px;
1317 }
1318 .action-card h3 {
1319   font-size: 12px;
1320   line-height: 1.7;
1321   margin-top: 10px;
1322   color: #58694b;
1323 }
1324 .action-card p {
1325   font-size: 10px;
1326   line-height: 1.8;
1327   color: #58694b;
1328   margin-top: 5px;
1329 }
1330 .action-card > .mono {
1331   white-space: nowrap;
1332 }
1333 .activity-title {
1334   font-size: 17px;
1335   margin: 35px 0 25px;
1336   font-weight: 400;
1337 }
1338 .timeline {
1339   margin: 0 0 40px 6px;
1340 }
1341 .timeline-item {
1342   display: flex;
1343   gap: 18px;
1344   position: relative;
1345   padding-bottom: 26px;
1346   border-left: 1px solid #d7e1cb;
1347   padding-left: 25px;
1348 }
1349 .timeline-dot {
1350   position: absolute;
1351   width: 7px;
1352   height: 7px;
1353   background: #a9b994;
1354   left: -4px;
1355   top: 6px;
1356   border-radius: 50%;
1357   box-shadow: 0 0 4px #f8f9f5;
1358 }
1359 .timeline-item strong {
1360   font-size: 11px;
```

app/globals.css (continued)

```

1361 font-weight: 500;
1362 text-transform: capitalize;
1363 color: #58694b;
1364 }
1365 .timeline-item p {
1366 font-size: 11px;
1367 line-height: 1.8;
1368 color: #58694b;
1369 max-width: 850px;
1370 margin: 7px 0;
1371 }
1372 .timeline-item small {
1373 font-size: 10px;
1374 color: #58694b;
1375 }
1376 .about-page h1 {
1377 font-size: 43px;
1378 }
1379 .intro.large {
1380 max-width: 700px;
1381 font-size: 15px;
1382 }
1383 .about-story {
1384 margin: 35px 0;
1385 padding: 30px;
1386 border-radius: 9px;
1387 background: #eaf0d8;
1388 }
1389 .about-story h2 {
1390 font-size: 29px;
1391 letter-spacing: -1px;
1392 font-weight: 400;
1393 margin: 20px 0;
1394 }
1395 .about-story p {
1396 max-width: 700px;
1397 font-size: 12px;
1398 line-height: 1.9;
1399 color: #58694b;
1400 margin: 15px 0 20px;
1401 }
1402 .about-columns {
1403 display: grid;
1404 grid-template-columns: 1fr 1fr;
1405 gap: 35px;
1406 margin: 40px 0;
1407 }
1408 .about-columns h2 {
1409 font-size: 19px;
1410 font-weight: 400;
1411 margin-bottom: 16px;
1412 letter-spacing: -0.4px;
1413 }
1414 .about-columns p {
1415 font-size: 12px;
1416 line-height: 1.9;
1417 color: #58694b;
1418 margin: 12px 0;
1419 }
1420 .about-columns strong {
1421 font-weight: 500;
1422 color: #58694b;
1423 }
1424 .about-columns a {
1425 display: block;
1426 font-size: 10px;
1427 color: #58694b;
1428 text-decoration: underline;
1429 text-underline-offset: 3px;
1430 margin-top: 12px;
1431 }
1432 .creator-credit {
1433 display: flex;
1434 align-items: center;
1435 gap: 16px;
1436 border-top: 1px solid #e0e6d6;
1437 padding: 25px 0 40px;
1438 }
1439 .creator-credit strong {
1440 font-size: 12px;

```

app/globals.css (continued)

```
1441 font-weight: 500;
1442 }
1443 .creator-credit p {
1444   font-size: 10px;
1445   line-height: 1.7;
1446   color: #58694b;
1447   margin-top: 5px;
1448 }
1449 .creator-credit a {
1450   font-size: 10px;
1451   white-space: nowrap;
1452   margin-left: auto;
1453   color: #58694b;
1454 }
1455 .loading-state {
1456   padding: 100px 30px;
1457   text-align: center;
1458 }
1459 .loading-state > svg {
1460   margin: auto;
1461   color: #58694b;
1462 }
1463 .loading-state h1 {
1464   font-size: 30px;
1465 }
1466 .loading-state p {
1467   font-size: 12px;
1468   color: #58694b;
1469 }
1470 .spin {
1471   animation: spin 1s linear infinite;
1472 }
1473 @keyframes spin {
1474   to {
1475     transform: rotate(360deg);
1476   }
1477 }
1478 .wide-dialog {
1479   max-width: 680px !important;
1480 }
1481 .notice-dialog {
1482   max-width: 830px !important;
1483   max-height: 90vh;
1484   overflow: auto;
1485 }
1486 .wide-dialog {
1487   max-height: 90vh;
1488   overflow: auto;
1489 }
1490 [data-slot="dialog-content"],
1491 [data-slot="alert-dialog-content"] {
1492   background: #fcfdf9;
1493   border-color: #d7e0ca;
1494   color: #405331;
1495   border-radius: 12px;
1496   padding: 28px;
1497 }
1498 [data-slot="dialog-title"],
1499 [data-slot="alert-dialog-title"] {
1500   font-size: 25px !important;
1501   font-weight: 400;
1502   letter-spacing: -0.7px;
1503   line-height: 1.25 !important;
1504 }
1505 [data-slot="dialog-description"],
1506 [data-slot="alert-dialog-description"] {
1507   font-size: 11px !important;
1508   line-height: 1.8;
1509   color: #58694b;
1510 }
1511 [data-slot="dialog-footer"] {
1512   margin-top: 20px;
1513 }
1514 .field-label {
1515   display: flex;
1516   flex-direction: column;
1517   gap: 7px;
1518   font-size: 10px;
1519   color: #58694b;
1520   margin-bottom: 17px;
```

app/globals.css (continued)

```
1521 }
1522 .field-label input,
1523 .field-label textarea {
1524   background: white;
1525   border: 1px solid #d9e3cb;
1526   border-radius: 6px;
1527   padding: 10px 12px;
1528   font:
1529     12px/1.7 Arial,
1530     sans-serif;
1531   color: #4e663b;
1532   min-width: 0;
1533   width: 100%;
1534 }
1535 .field-label input::placeholder,
1536 .field-label textarea::placeholder {
1537   color: #58694b;
1538 }
1539 .field-label textarea {
1540   resize: vertical;
1541   min-height: 65px;
1542 }
1543 .form-two {
1544   display: grid;
1545   grid-template-columns: 1fr 1fr;
1546   gap: 16px;
1547 }
1548 .file-input-label {
1549   position: relative;
1550   overflow: hidden;
1551   display: flex;
1552   align-items: center;
1553   justify-content: center;
1554   gap: 9px;
1555   cursor: pointer;
1556   background: #eff4e5;
1557   border: 1px dashed #cbdab7;
1558   border-radius: 6px;
1559   padding: 14px;
1560   font-size: 11px;
1561   color: #58694b;
1562   margin: 5px 0 18px;
1563 }
1564 .file-input-label input {
1565   position: absolute;
1566   inset: 0;
1567   opacity: 0;
1568   cursor: pointer;
1569   width: 100%;
1570   height: 100%;
1571 }
1572 .form-meta {
1573   display: flex;
1574   justify-content: space-between;
1575   align-items: center;
1576   gap: 15px;
1577 }
1578 .form-meta .text-button {
1579   font-size: 10px;
1580 }
1581 .form-meta > span {
1582   font-size: 10px;
1583   color: #58694b;
1584 }
1585 .form-error {
1586   font-size: 11px;
1587   line-height: 1.7;
1588   padding: 12px;
1589   background: #fcf0e9;
1590   border: 1px solid #efd8c9;
1591   border-radius: 6px;
1592   color: #58694b;
1593   margin: 15px 0;
1594 }
1595 .source-upload-row {
1596   display: flex;
1597   gap: 25px;
1598   align-items: center;
1599 }
1600 .source-upload-row .file-input-label {
```

app/globals.css (continued)

```

1601   min-width: 185px;
1602 }
1603 .source-upload-row .text-button {
1604   font-size: 10px;
1605   margin-bottom: 14px;
1606 }
1607 .scope-editor-heading {
1608   margin: 25px 0 8px;
1609 }
1610 .scope-editor-heading h2 {
1611   font-size: 13px;
1612 }
1613 .scope-editor-heading .text-button {
1614   font-size: 10px;
1615 }
1616 .scope-editor {
1617   background: #f2f6eb;
1618   border: 1px solid #dce6ce;
1619   border-radius: 7px;
1620   padding: 17px;
1621   margin: 13px 0;
1622 }
1623 .scope-editor-fields {
1624   display: grid;
1625   grid-template-columns: 1fr 1.5fr 30px;
1626   gap: 15px;
1627   align-items: center;
1628 }
1629 .scope-editor-fields .field-label {
1630   margin: 0;
1631 }
1632 .scope-editor > .field-label {
1633   margin-bottom: 0;
1634 }
1635 .scope-editor .check-label {
1636   font-size: 10px;
1637 }
1638 .add-scope {
1639   margin: 17px 0 25px;
1640   font-size: 10px;
1641 }
1642 .quantity-note {
1643   align-self: center;
1644   font-size: 12px;
1645   color: #58694b;
1646   padding-bottom: 12px;
1647 }
1648 .quantity-note small {
1649   display: block;
1650   margin-top: 5px;
1651   font-size: 10px;
1652   color: #58694b;
1653 }
1654 .export-options {
1655   display: grid;
1656   gap: 12px;
1657   margin-top: 12px;
1658 }
1659 .export-options button {
1660   display: flex;
1661   align-items: center;
1662   gap: 15px;
1663   text-align: left;
1664   border: 1px solid #dce5d0;
1665   padding: 18px;
1666   border-radius: 8px;
1667 }
1668 .export-options button:hover {
1669   background: #f2f6e9;
1670 }
1671 .export-options button > svg {
1672   color: #58694b;
1673   width: 21px;
1674 }
1675 .export-options button > svg:last-child {
1676   margin-left: auto;
1677 }
1678 .export-options strong {
1679   font-size: 12px;
1680   font-weight: 500;

```

app/globals.css (continued)

```
1681 display: block;
1682 }
1683 .export-options small {
1684   display: block;
1685   font-size: 10px;
1686   line-height: 1.7;
1687   color: #58694b;
1688   margin-top: 5px;
1689 }
1690 .report-toolbar {
1691   display: flex;
1692   justify-content: space-between;
1693   margin-bottom: 25px;
1694 }
1695 .audit-report {
1696   background: white;
1697   padding: 40px;
1698   border: 1px solid #dfe6d5;
1699   margin-bottom: 35px;
1700 }
1701 .report-head {
1702   display: flex;
1703   justify-content: space-between;
1704   align-items: center;
1705 }
1706 .report-head > span {
1707   font-size: 10px;
1708   letter-spacing: 1px;
1709   color: #58694b;
1710 }
1711 .audit-report h1 {
1712   font-size: 30px;
1713   letter-spacing: -1px;
1714   margin: 30px 0 15px;
1715 }
1716 .audit-report p {
1717   font-size: 11px;
1718   line-height: 1.8;
1719   color: #58694b;
1720   margin: 10px 0;
1721 }
1722 .audit-report h2 {
1723   font-size: 18px;
1724   font-weight: 500;
1725   margin: 28px 0 15px;
1726 }
1727 .audit-report h3 {
1728   font-size: 12px;
1729   margin: 20px 0 10px;
1730 }
1731 .audit-report a {
1732   font-size: 10px;
1733   overflow-wrap: anywhere;
1734   color: #58694b;
1735 }
1736 .audit-report th {
1737   font-size: 10px;
1738   white-space: normal;
1739 }
1740 .audit-report td {
1741   font-size: 10px;
1742   white-space: normal;
1743   vertical-align: top;
1744   color: #58694b;
1745   line-height: 1.7;
1746 }
1747 .audit-report td small {
1748   font-size: 10px;
1749 }
1750 .report-notice {
1751   padding: 15px;
1752   background: #f7f2df;
1753 }
1754 .report-event {
1755   padding: 12px 0;
1756   border-bottom: 1px solid #edf0e6;
1757   break-inside: avoid;
1758 }
1759 .report-event strong {
1760   font-size: 10px;
```

app/globals.css (continued)

```
1761 }
1762 .report-event small {
1763   font-size: 10px;
1764   color: #58694b;
1765   float: right;
1766 }
1767 .report-event p {
1768   font-size: 10px;
1769 }
1770 .response-heading {
1771   margin: 10px 0 25px;
1772 }
1773 .response-heading .fine-print {
1774   margin: 8px 0;
1775 }
1776 .page-footer {
1777   margin-top: 30px;
1778 }
1779 @media (max-width: 1100px) {
1780   .source-grid {
1781     grid-template-columns: 1fr;
1782   }
1783   .match-row-main {
1784     grid-template-columns: minmax(160px, 1fr) 105px 120px 24px;
1785     gap: 9px;
1786     padding: 15px 12px;
1787   }
1788   .match-symbol {
1789     display: none;
1790   }
1791   .legend {
1792     gap: 8px;
1793   }
1794   .ai-banner {
1795     padding: 15px;
1796   }
1797   .table-toolbar {
1798     flex-wrap: wrap;
1799   }
1800   .search-box {
1801     margin-left: auto;
1802   }
1803 }
1804 @media (max-width: 767px) {
1805   .save-status {
1806     display: none;
1807   }
1808   .compact {
1809     display: block;
1810   }
1811   .compact > .button {
1812     margin-top: 12px;
1813   }
1814   .compact h1 {
1815     font-size: 32px;
1816   }
1817   .review-tabs > [data-slot="tabs-list"] {
1818     gap: 16px;
1819   }
1820   .review-tabs [data-slot="tabs-trigger"] {
1821     font-size: 11px;
1822     gap: 5px;
1823   }
1824   .ai-banner {
1825     flex-wrap: wrap;
1826     gap: 11px;
1827   }
1828   .ai-icon {
1829     padding: 8px;
1830   }
1831   .ai-banner > div:nth-child(2) {
1832     flex-basis: calc(100% - 55px);
1833   }
1834   .ai-banner > .button {
1835     margin-left: 43px;
1836   }
1837   .match-row-main {
1838     grid-template-columns: 1fr 26px;
1839     position: relative;
1840     gap: 13px;
```

app/globals.css (continued)

```

1841 }
1842 .match-product {
1843   grid-column: 1;
1844 }
1845 .match-row-main > .icon-button {
1846   grid-column: 2;
1847   grid-row: 1;
1848 }
1849 .match-identifiers {
1850   grid-column: 1;
1851   flex-direction: row;
1852   gap: 12px;
1853 }
1854 .match-state {
1855   grid-column: 1/-1;
1856   flex-direction: row;
1857   align-items: center;
1858 }
1859 .match-expanded {
1860   flex-direction: column;
1861   padding: 16px;
1862 }
1863 .match-expanded > div:last-child {
1864   width: 100%;
1865 }
1866 .legend {
1867   font-size: 10px;
1868   flex-wrap: wrap;
1869 }
1870 .search-box {
1871   width: 100%;
1872 }
1873 .search-box input {
1874   width: 100%;
1875 }
1876 .source-review-panel,
1877 .source-document {
1878   padding: 19px;
1879 }
1880 .form-two {
1881   grid-template-columns: 1fr;
1882   gap: 0;
1883 }
1884 .source-upload-row {
1885   flex-direction: column;
1886   align-items: stretch;
1887   gap: 0;
1888 }
1889 .scope-editor-fields {
1890   grid-template-columns: 1fr 1fr 22px;
1891   gap: 8px;
1892 }
1893 .source-upload-row .text-button {
1894   font-size: 10px;
1895 }
1896 .scope-editor-heading {
1897   gap: 15px;
1898   align-items: center;
1899 }
1900 .scope-editor-heading .text-button {
1901   text-align: right;
1902 }
1903 .about-columns {
1904   grid-template-columns: 1fr;
1905   gap: 25px;
1906 }
1907 .about-page h1 {
1908   font-size: 35px;
1909 }
1910 .creator-credit {
1911   flex-wrap: wrap;
1912 }
1913 .creator-credit > div {
1914   flex: 1;
1915 }
1916 .creator-credit a {
1917   width: 100%;
1918   margin-left: 51px;
1919 }
1920 .audit-report {

```

app/globals.css (continued)

```

1921     padding: 20px;
1922   }
1923   .report-head > span {
1924     max-width: 100px;
1925     line-height: 1.7;
1926     text-align: right;
1927   }
1928   .report-event small {
1929     display: block;
1930     float: none;
1931     margin-top: 5px;
1932   }
1933   .action-card {
1934     flex-direction: column;
1935     gap: 10px;
1936   }
1937   .workspace-tools {
1938     gap: 20px;
1939     flex-wrap: wrap;
1940   }
1941   .form-meta {
1942     align-items: flex-start;
1943   }
1944   .form-meta > span {
1945     text-align: right;
1946   }
1947   .gate-hint {
1948     max-width: none;
1949   }
1950   .export-options button {
1951     padding: 12px;
1952     gap: 10px;
1953   }
1954   .export-options small {
1955     font-size: 10px;
1956   }
1957 }
1958 @media (prefers-reduced-motion: reduce) {
1959   *,
1960   *:before,
1961   *:after {
1962     animation: none !important;
1963     transition: none !important;
1964   }
1965 }
1966 @media print {
1967   [data-slot="sidebar"],
1968   [data-slot="sidebar-wrapper"] > [data-slot="sidebar"],
1969   .topbar,
1970   .page-footer,
1971   .report-toolbar,
1972   [data-sonner-toaster] {
1973     display: none !important;
1974   }
1975   [data-slot="sidebar-container"],
1976   [data-slot="sidebar-gap"] {
1977     display: none !important;
1978   }
1979   .app-shell {
1980     width: 100% !important;
1981   }
1982   .main-content {
1983     padding: 0 !important;
1984     max-width: none;
1985   }
1986   .audit-report {
1987     border: 0;
1988     padding: 0;
1989     margin: 0;
1990   }
1991   .audit-report th,
1992   .audit-report td {
1993     font-size: 10px;
1994     color: #333;
1995   }
1996   .audit-report p {
1997     color: #333;
1998   }
1999   .audit-report tr {
2000     break-inside: avoid;

```

app/globals.css (continued)

```

2001  }
2002  .audit-report section {
2003    break-before: auto;
2004  }
2005  .audit-report h2 {
2006    break-after: avoid;
2007  }
2008  .report-notice {
2009    border: 1px solid #999;
2010    background: none;
2011  }
2012  @page {
2013    size: A4;
2014    margin: 14mm;
2015  }
2016 }
2017
2018 .orange-text{color:#ad5a32}.status-
    tag{color:#994a2d}.pill.exact{color:#944f2c}.pill.missing,.pill.possible{color:#806b24}
2019 .review-tabs [data-slot='tabs-trigger']{color:#58694b}

```

app/layout.tsx

```
1 import type { Metadata } from "next";
2 import "../globals.css";
3
4 export const metadata: Metadata = {
5   title: "Lotlight \u2014 Recall to resolution",
6   description:
7     "Evidence-backed medical recall review. Connect the notice to your inventory and document the response.
8     A project by Shivam Gupta.",
9   icons: {
10     icon: "/favicon.svg",
11     shortcut: "/favicon.svg",
12   },
13 };
14 export default function RootLayout({
15   children,
16 }: Readonly<{
17   children: React.ReactNode;
18 }>) {
19   return (
20     <html lang="en">
21       <body className="antialiased">{children}</body>
22     </html>
23   );
24 }
```

app/lotlight.tsx

```
1 "use client";
2 import { useCallback, useEffect, useMemo, useRef, useState } from "react";
3 import {
4   ArrowUpRight,
5   ArrowRight,
6   ScanLine,
7   Layers,
8   ClipboardCheck,
9   BookOpen,
10  Plus,
11  ShieldCheck,
12  Package,
13  FileText,
14  ChevronRight,
15  LogIn,
16  Download,
17  Check,
18  AlertTriangle,
19  Search,
20  RotateCcw,
21  Loader2,
22  Sparkles,
23  X,
24  ExternalLink,
25  ArrowLeft,
26  CheckCircle2,
27  Clock,
28  MoreHorizontal,
29  Upload,
30  Trash2,
31 } from "lucide-react";
32 import {
33   Sidebar,
34   SidebarProvider,
35   SidebarContent,
36   SidebarHeader,
37   SidebarFooter,
38   SidebarMenu,
39   SidebarMenuItem,
40   SidebarMenuButton,
41   SidebarTrigger,
42 } from "@/components/ui/sidebar";
43 import {
44   Dialog,
45   DialogContent,
46   DialogTitle,
47   DialogDescription,
48   DialogHeader,
49   DialogFooter,
50 } from "@/components/ui/dialog";
51 import { Tabs, TabsList, TabsTrigger, TabsContent } from "@/components/ui/tabs";
52 import {
53   Table,
54   TableBody,
55   TableCell,
56   TableHead,
57   TableHeader,
58   TableRow,
59 } from "@/components/ui/table";
60 import {
61   Select,
62   SelectContent,
63   SelectItem,
64   SelectTrigger,
65   SelectValue,
66 } from "@/components/ui/select";
67 import { Checkbox } from "@/components/ui/checkbox";
68 import {
69   AlertDialog,
70   AlertDialogContent,
71   AlertDialogTitle,
72   AlertDialogDescription,
73   AlertDialogFooter,
74   AlertDialogCancel,
75   AlertDialogAction,
76 } from "@/components/ui/alert-dialog";
77 import { Toaster, toast } from "sonner";
78 import {
79   initialWorkspace,
80   SAMPLE_CSV,
```

app/lotlight.tsx (continued)

```

81  SAMPLE_NOTICE_TEXT,
82  BAXTER_URL,
83 } from "@lib/lotlight/sample";
84 import {
85   activeAction,
86   reduceWorkspace,
87   type Command,
88 } from "@lib/lotlight/commands";
89 import {
90   reconcile,
91   extractScope,
92   parseCsv,
93   validateScope,
94 } from "@lib/lotlight/engine";
95 import {
96   download,
97   exportAudit,
98   exportMatches,
99   readNoticeFile,
100 } from "@lib/lotlight/files";
101 import type {
102   Workspace,
103   Notice,
104   Match,
105   Scope,
106   UserInfo,
107   ResponseAction,
108   InventoryItem,
109 } from "@lib/lotlight/types";
110
111 type View =
112   | "overview"
113   | "inventory"
114   | "actions"
115   | "about"
116   | "review"
117   | "report";
118 const NAV = [
119   { icon: Layers, label: "Recall workspace", view: "overview" },
120   { icon: Package, label: "Inventory", view: "inventory" },
121   { icon: ClipboardCheck, label: "Action log", view: "actions" },
122   { icon: BookOpen, label: "Evidence & approach", view: "about" },
123 ] as const;
124 const LABELS = {
125   exact: "Identifier match",
126   missing: "Needs information",
127   possible: "AI candidate",
128   unlisted: "No listed match",
129 };
130 const date = (s: string) =>
131   new Date(s).toLocaleDateString("en-GB", {
132     day: "2-digit",
133     month: "short",
134     year: "numeric",
135     timeZone: "UTC",
136   });
137 export default function Lotlight({ user }: { user: UserInfo }) {
138   const [hydrated, setHydrated] = useState(false);
139   useEffect(() => {
140     setHydrated(true);
141   }, []);
142   const [noticeNonce, setNoticeNonce] = useState(0);
143   const [correcting, setCorrecting] = useState<InventoryItem | null>(null);
144   const [state, setState] = useState<Workspace>(initialWorkspace);
145   const [view, setView] = useState<View>("overview");
146   const [loading, setLoading] = useState(!user);
147   const [busy, setBusy] = useState(false);
148   const busyRef = useRef(false);
149   const [saveError, setSaveError] = useState("");
150   const [noticeId, setNoticeId] = useState("baxter-demo");
151   const [noticeDialog, setNoticeDialog] = useState(false);
152   const [editing, setEditing] = useState<Notice | undefined>();
153   const [inventoryDialog, setInventoryDialog] = useState(false);
154   const [actionMatch, setActionMatch] = useState<Match | null>(null);
155   const [exportOpen, setExportOpen] = useState(false);
156   const [reset, setReset] = useState<"reset_demo" | "clear_workspace" | null>(
157     null,
158   );
159   const [scores, setScores] = useState<Record<string, number>>({});
160   const [aiStatus, setAiStatus] = useState("idle");

```

app/lotlight.tsx (continued)

```

161 const [aiMessage, setAiMessage] = useState("");
162 const worker = useRef<Worker | null>(null);
163 const [query, setQuery] = useState("");
164 const notice =
165   state.notices.find((n) => n.id === noticeId) ?? state.notices[0];
166 const matches = useMemo(
167   () => (notice ? reconcile(state.inventory, notice, scores) : []),
168   [state.inventory, notice, scores],
169 );
170 const attention = matches.filter((m) => m.kind !== "unlisted");
171 const exact = matches.filter((m) => m.kind === "exact");
172 const uncertain = matches.filter(
173   (m) => m.kind === "missing" || m.kind === "possible",
174 );
175 const completed = notice
176   ? attention.filter(
177     (m) =>
178       activeAction(state, notice.id, m.item.id)?.status === "completed",
179   ).length
180   : 0;
181 const load = useCallback(async () => {
182   if (!user) return;
183   setLoading(true);
184   try {
185     const res = await fetch("/api/workspace", { cache: "no-store" });
186     const data = (await res.json()) as Workspace & { error: string };
187     if (!res.ok) throw new Error(data.error);
188     setState(data);
189     setSaveError("");
190   } catch (error) {
191     setSaveError(
192       error instanceof Error ? error.message : "Could not load workspace.",
193     );
194   } finally {
195     setLoading(false);
196   }
197 }, [user]);
198 useEffect(() => {
199   void load();
200 }, [load]);
201 useEffect(() => {
202   setScores({});
203   setAiStatus("idle");
204   worker.current?.terminate();
205   worker.current = null;
206   return () => {
207     worker.current?.terminate();
208   };
209 }, [notice?.id, notice?.version, state.inventoryVersion]);
210 const send = async (command: Command): Promise<boolean> => {
211   if (busyRef.current || loading) return false;
212   busyRef.current = true;
213   setBusy(true);
214   try {
215     if (user) {
216       if (saveError)
217         throw new Error("Reload your saved workspace before making changes.");
218       const res = await fetch("/api/workspace", {
219         method: "POST",
220         headers: { "Content-Type": "application/json" },
221         body: JSON.stringify({ revision: state.revision, command }),
222       });
223       const data = (await res.json()) as Workspace & { error: string };
224       if (!res.ok) {
225         if (res.status === 409) setSaveError(data.error);
226         throw new Error(data.error);
227       }
228       setState(data);
229     } else
230       setState(reduceWorkspace(state, command, "Demo operator (unsaved)"));
231     toast.success(
232       user ? "Saved to your workspace" : "Updated this unsaved demo",
233     );
234     return true;
235   } catch (error) {
236     toast.error(
237       error instanceof Error
238         ? error.message
239         : "Could not save. Your inputs are preserved.",
240     );
241   }

```

app/lotlight.tsx (continued)

```

241     return false;
242   } finally {
243     busyRef.current = false;
244     setBusy(false);
245   }
246 };
247 function openReview(n: Notice) {
248   setNoticeId(n.id);
249   setView("review");
250   setQuery("");
251 }
252 function runAI() {
253   if (!notice || !state.inventory.length || aiStatus === "loading") return;
254   setAiStatus("loading");
255   setAiMessage(
256     "Loading the local model. First use downloads about 23 MB of model weights.",
257   );
258   try {
259     worker.current?.terminate();
260     const current = new Worker("/ai/semantic-worker.js", { type: "module" });
261     worker.current = current;
262     current.onmessage = (e) => {
263       if (e.data.type === "progress") setAiMessage(e.data.message);
264       if (e.data.type === "result") {
265         setScores(e.data.scores);
266         setAiStatus("ready");
267         setAiMessage(
268           "Semantic comparison complete. Similarity ranks candidates; it cannot establish a recall
match.",
269         );
270       }
271       if (e.data.type === "error") {
272         setAiStatus("error");
273         setAiMessage(
274           "Local AI could not load. Identifier checks still work. Check your connection and retry.",
275         );
276         current.terminate();
277       }
278     };
279     current.onerror = (event) => {
280       console.error(
281         "AI worker error",
282         event.message,
283         event.filename,
284         event.lineno,
285       );
286       setAiStatus("error");
287       setAiMessage(
288         "Local AI is unavailable in this browser. Identifier checks still work.",
289       );
290       current.terminate();
291     };
292     current.postMessage({
293       query: `${notice.manufacturer} ${notice.text.match(/^Product:\s*(.+)\$/m)?.[1] ?? notice.title}`,
294       items: state.inventory.map((i) => ({ id: i.id, product: i.product })),
295     });
296   } catch (error) {
297     console.error("AI worker initialization", error);
298     setAiStatus("error");
299     setAiMessage(
300       "This browser could not start the local AI worker. Identifier checks remain available.",
301     );
302   }
303 }
304 useEffect(() => {
305   const context = (
306     document as Document & {
307       modelContext?: {
308         registerTool: (tool: unknown, options: unknown) => void;
309       };
310     }
311   ).modelContext;
312   if (!context) return;
313   const lifecycle = new AbortController();
314   try {
315     context.registerTool(
316       {
317         name: "lotlight_read_review",
318         title: "Read recall review",
319         description:

```

app/lotlight.tsx (continued)

```

320         "Read the current inventory review, source approval, and unresolved lines. Does not change
records.",
321         inputSchema: {
322             type: "object",
323             properties: {},
324             additionalProperties: false,
325         },
326         annotations: { readOnlyHint: true, untrustedContentHint: true },
327         execute(input: unknown) {
328             if (
329                 !input ||
330                 typeof input !== "object" ||
331                 Object.keys(input).length
332             )
333                 throw new Error("Expected an empty object.");
334             return {
335                 notice: notice?.title ?? null,
336                 sourceApproved: !!notice?.approvedAt,
337                 inventoryLines: state.inventory.length,
338                 matches: matches.map((m) => ({
339                     product: m.item.product,
340                     catalog: m.item.catalog,
341                     lot: m.item.lot,
342                     kind: m.kind,
343                     reason: m.reason,
344                 })),
345                 unsavedDemo: !user,
346             };
347         },
348     },
349     { signal: lifecycle.signal },
350 );
351 } catch {
352     /* Optional browser capability. */
353 }
354 return () => lifecycle.abort();
355 }, [notice, state.inventory, matches, user]);
356 const isTraining = state.notices.some((n) => n.training);
357 const navTitle = {
358     overview: "Recall review",
359     inventory: "Inventory",
360     actions: "Action log",
361     about: "Evidence & approach",
362     review: "Recall review",
363     report: "Audit report",
364 }[view];
365 return (
366     <SidebarProvider
367         style={{ "--sidebar-width": "224px" } as React.CSSProperties}
368     >
369         <Toaster richColors position="bottom-right" />
370         <Sidebar className="lot-sidebar">
371             <SidebarHeader>
372                 <a className="brand" href="/">
373                     <span className="brand-mark">
374                         <ScanLine size={23} />
375                     </span>
376                     lotlight<span className="brand-dot">\u2733</span>
377                 </a>
378                 <div className="workspace-label">
379                     <span className="workspace-avatar">{user ? "P" : "W"}</span>
380                     <div>
381                         {user ? "Personal workspace" : "Willow Community Clinic"}
382                         <small>
383                             {user
384                                 ? "Private account records"
385                                 : "Fictional demonstration clinic"}
386                         </small>
387                     </div>
388                 </div>
389             </SidebarHeader>
390             <SidebarContent>
391                 <span className="nav-caption">WORKSPACE</span>
392                 <SidebarMenu>
393                     {NAV.map(({ icon: Icon, label, view: target }) => (
394                         <SidebarMenuItem key={target}>
395                             <SidebarMenuButton
396                                 onClick={() => {
397                                     setView(target);
398                                     setQuery("");

```

app/lotlight.tsx (continued)

```

399         }}
400         isActive={
401             view === target ||
402             (view === "review" && target === "overview")
403         }
404         className={
405             view === target ||
406             (view === "review" && target === "overview")
407             ? "nav-active"
408             : ""
409         }
410     >
411         <Icon size={18} />
412         <span>{label}</span>
413         {target === "overview" && (
414             <span className="nav-count">{state.notices.length}</span>
415         )}
416     </SidebarMenuButton>
417 </SidebarMenuItem>
418 )))
419 </SidebarMenu>
420 <div className="side-note">
421     <ShieldCheck size={20} />
422     <strong>
423         Human judgment.
424         <br />A better paper trail.
425     </strong>
426     <p>Every match shows its evidence. Every action has an owner.</p>
427 </div>
428 </SidebarContent>
429 <SidebarFooter>
430     <div className="creator">
431         <span>SG</span>
432         <div>
433             Shivam Gupta<small>Creator \xb7 DSH Hacks V2</small>
434         </div>
435     </div>
436 </SidebarFooter>
437 </Sidebar>
438 <div className="app-shell">
439     <header className="topbar">
440         <div>
441             <SidebarTrigger className="mobile-menu" />
442             <span>Workspace</span>
443             <ChevronRight size={14} />
444             <b>{navTitle}</b>
445         </div>
446         <div className="account-area">
447             <span className="save-status">
448                 {loading
449                     ? "Loading\u2026"
450                     : busy
451                     ? "Saving\u2026"
452                     : saveError
453                     ? "Save unavailable"
454                     : user
455                     ? "Account storage"
456                     : "Demo \xb7 not saved"}
457             </span>
458             <a
459                 className="top-account"
460                 href={
461                     user
462                     ? "/signout-with-chatgpt?return_to=%2F"
463                     : "/signin-with-chatgpt?return_to=%2F"
464                 }
465                 title={
466                     user ? user.email : "Demo changes will not transfer on sign-in"
467                 }
468             >
469                 <LogIn size={15} />
470                 {user ? "Sign out" : "Sign in to save"}
471             </a>
472         </div>
473     </header>
474     <main className="main-content" data-ready={hydrated} inert={!hydrated}>
475         {saveError && (
476             <div className="error-banner" role="alert">
477                 <AlertTriangle size={18} />
478                 <span>{saveError}</span>

```

app/lotlight.tsx (continued)

```

479         <button className="text-button" onClick={() => void load()}>
480             Reload saved version
481         </button>
482     </div>
483 }}
484 {loading ? (
485     <div className="loading-state">
486         <Loader2 className="spin" />
487         <h1>Opening your workspace</h1>
488         <p>Loading your saved inventory and source reviews.</p>
489     </div>
490 ) : (
491     <>
492         {view === "overview" && (
493             <>
494                 <div className="page-heading">
495                     <div>
496                         <p className="eyebrow">
497                             <span />
498                             THE LAST MILE OF RECALL RESPONSE
499                         </p>
500                         <h1>
501                             A notice is only
502                             <br />
503                             the beginning.
504                         </h1>
505                         <p className="intro">
506                             Find what\u2019s on your shelves. See why it matches.
507                             <br />
508                             Give every response a clear next step.
509                         </p>
510                     </div>
511                     <div className="heading-actions">
512                         <span className="demo-label">
513                             {isTraining
514                                 ? "HISTORICAL NOTICE \xb7 SYNTHETIC STOCK"
515                                 : "PRIVATE REVIEW WORKSPACE"}
516                         </span>
517                         <button
518                             className="button primary"
519                             onClick={() => {
520                                 setEditing(undefined);
521                                 setNoticeDialog(true);
522                             }}
523                         >
524                             <Plus size={17} /> New recall review
525                         </button>
526                     </div>
527                 </div>
528                 <div className="overview-strip">
529                     <div>
530                         <span>Inventory lines</span>
531                         <strong>
532                             {state.inventory.length}
533                             <small>
534                                 across{" "}
535                                 {new Set(state.inventory.map((i) => i.location)).size}{" "}
536                                 locations
537                             </small>
538                         </strong>
539                     </div>
540                     <div>
541                         <span>Identifier matches</span>
542                         <strong className="orange-text">
543                             {exact.length}
544                             <small>in the selected notice</small>
545                         </strong>
546                     </div>
547                     <div>
548                         <span>Needs more evidence</span>
549                         <strong>
550                             {uncertain.length}
551                             <small>never silently cleared</small>
552                         </strong>
553                     </div>
554                     <div>
555                         <span>Responses completed</span>
556                         <strong>
557                             {completed}
558                             <small>with staff-recorded evidence</small>

```

app/lotlight.tsx (continued)

```

559         </strong>
560     </div>
561 </div>
562 <div className="section-heading">
563     <h2>
564         Recall workspace{" "}
565         <span>
566             {String(state.notices.length).padStart(2, "0")}
567         </span>
568     </h2>
569     <span>
570         {notice
571             ? "Summary for " + notice.title
572             : "Ready for your first notice"}
573     </span>
574 </div>
575 <div className="review-grid">
576     <div className="notice-stack">
577         {state.notices.length === 0 ? (
578             <Empty
579                 title="Start with the notice"
580                 text="Add the original text and paired identifiers, then import a stock
spreadsheet."
581                 action={
582                     <button
583                         className="button primary"
584                         onClick={() => {
585                             setEditing(undefined);
586                             setNoticeDialog(true);
587                         }}
588                     >
589                         Create a review <Plus size={15} />
590                     </button>
591                 }
592             >
593         ) : (
594             state.notices.map((n) => {
595                 const m = reconcile(state.inventory, n);
596                 const count = m.filter(
597                     (x) => x.kind !== "unlisted",
598                 ).length;
599                 return (
600                     <section className="review-card" key={n.id}>
601                         <div className="card-topline">
602                             <span
603                                 className={
604                                     "status-tag " +
605                                     (n.approvedAt ? "approved" : "")
606                                 }
607                             >
608                                 <span />
609                                 {n.approvedAt
610                                     ? "SOURCE REVIEWED"
611                                     : "SOURCE REVIEW REQUIRED"}
612                             </span>
613                             <span className="mono">
614                                 {n.training
615                                     ? "FA-2025-039 \xb7 TRAINING"
616                                     : `VERSION ${n.version}`}
617                             </span>
618                         </div>
619                         <h2>{n.title}</h2>
620                         <p>
621                             {n.manufacturer} \xb7 Notice dated {date(n.date)}
622                             {n.training
623                                 ? " \xb7 Two-pair training extract"
624                                 : ""}
625                         </p>
626                         <div className="scope-line">
627                             <span>Reviewed scope</span>
628                             {n.scope.slice(0, 3).map((s, i) => (
629                                 <code key={i}>
630                                     {s.catalog} /{" "}
631                                     {s.allLots ? "all lots" : s.lots.join(", ")}
632                                 </code>
633                             ))}
634                         </div>
635                         <div className="match-visual">
636                             <div className="match-bars">
637                                 {m.slice(0, 24).map((x) => (

```

app/lotlight.tsx (continued)

```

638         <span
639             key={x.item.id}
640             style={{
641                 background:
642                     x.kind === "exact"
643                         ? "#c9865a"
644                         : x.kind === "unlisted"
645                             ? "#dfe5d4"
646                             : "#d9ca80",
647             }}
648         />
649     )}}
650 </div>
651     <span>
652         <b>{m.length}</b> inventory lines checked{" "}
653         <span>\xb7</span> <b>{count}</b> need attention
654     </span>
655 </div>
656 <div className="card-bottom">
657     <span>
658         <FileText size={16} />{" "}
659         {n.approvedAt
660             ? "Staff-attested source review"
661             : "Source-linked decisions"}
662     </span>
663     <button
664         className="text-button"
665         onClick={() => openReview(n)}
666     >
667         Open review <ArrowRight size={16} />
668     </button>
669 </div>
670 </section>
671 );
672 })
673 })
674 </div>
675 <aside className="principle-card">
676     <div className="mini-orbit">
677         <ScanLine size={25} />
678         <span />
679         <span />
680     </div>
681     <p className="eyebrow">BUILT FOR THE IN-BETWEEN</p>
682     <h3>
683         Published doesn\u2019t
684         <br />
685         mean resolved.
686     </h3>
687     <p>
688         Connect the notice to the shelf, and the shelf to a
689         documented response.
690     </p>
691     <button
692         className="principle-footer"
693         onClick={() => setView("about")}
694     >
695         EVIDENCE BEFORE ACTION <ArrowUpRight size={17} />
696     </button>
697 </aside>
698 </div>
699 <div className="bottom-explainer">
700     <div>
701         <span className="step-number">01</span>
702         <h3>Bring the notice</h3>
703         <p>Preserve the original wording and scope.</p>
704     </div>
705     <div>
706         <span className="step-number">02</span>
707         <h3>Reconcile the shelf</h3>
708         <p>Exact identifiers first. AI helps find aliases.</p>
709     </div>
710     <div>
711         <span className="step-number">03</span>
712         <h3>Document the response</h3>
713         <p>Assign, verify, and record completion.</p>
714     </div>
715 </div>
716 <div className="workspace-tools">
717     <button

```

app/lotlight.tsx (continued)

```

718         className="text-button muted"
719         onClick={() => setReset("reset_demo")}
720     >
721     <RotateCcw size={13} /> Restart demo
722 </button>
723 <button
724     className="text-button muted"
725     onClick={() => setReset("clear_workspace")}
726 >
727     <Plus size={13} /> Start empty workspace
728 </button>
729 </div>
730 </>
731 }}
732 {view === "review" && notice && (
733     <>
734     <button
735         className="back-button"
736         onClick={() => setView("overview")}
737     >
738     <ArrowLeft size={14} /> Recall workspace
739 </button>
740 <div className="page-heading compact">
741     <div>
742         <p className="eyebrow">
743             {notice.training
744               ? "HISTORICAL TRAINING EXTRACT"
745               : "SOURCE REVIEW"}{" "}
746             \xb7 VERSION {notice.version}
747         </p>
748         <h1>{notice.title}</h1>
749         <p className="intro">
750             {notice.manufacturer} \xb7 Issued {date(notice.date)} \xb7{" "}
751             {notice.scope.length} catalog/lot groups
752         </p>
753     </div>
754     <button
755         className="button"
756         onClick={() => setExportOpen(true)}
757     >
758     <Download size={15} /> Export record
759 </button>
760 </div>
761 <div className="review-disclaimer">
762     <ShieldCheck size={16} />
763     <span>
764         {notice.training
765           ? "This historical example contains only two catalog/lot pairs. Verify the full,
766             current notice before any real-world action."
767           : "This review covers only the source and identifiers entered here. Verify the
768             complete current manufacturer instructions before acting."}{" "}
769         Inventory: {state.inventoryProvenance ?? "synthetic"}.
770     </span>
771 </div>
772 <Tabs defaultValue="inventory" className="review-tabs">
773     <TabsList variant="line">
774         <TabsTrigger value="inventory">
775             Inventory review{" "}
776             <span className="tab-count">{matches.length}</span>
777         </TabsTrigger>
778         <TabsTrigger value="source">Source & scope</TabsTrigger>
779         <TabsTrigger value="response">
780             Response plan{" "}
781             <span className="tab-count">{attention.length}</span>
782         </TabsTrigger>
783     </TabsList>
784     <TabsContent value="inventory">
785         <div className="ai-banner">
786             <div className="ai-icon">
787                 <Sparkles size={20} />
788             </div>
789             <div>
790                 <strong>Find the product behind the wording.</strong>
791                 <p>
792                     {aiStatus === "idle"
793                       ? "Local AI compares descriptions like \u201cIV line extender\u201d and
794                         \u201cextension set.\u201d Identifiers always decide the match."

```

app/lotlight.tsx (continued)

```

795         <button
796           className="button"
797           disabled={
798             aiStatus === "loading" || !state.inventory.length
799           }
800           onClick={runAI}
801         >
802           {aiStatus === "loading" ? (
803             <Loader2 className="spin" size={15} />
804           ) : (
805             <Sparkles size={15} />
806           )}{ " "}
807           {aiStatus === "ready"
808             ? "Run again"
809             : aiStatus === "error"
810               ? "Retry AI"
811               : "Run local AI"}
812         </button>
813       {aiStatus === "loading" && (
814         <button
815           aria-label="Cancel AI"
816           className="icon-button"
817           onClick={() => {
818             worker.current?.terminate();
819             setAiStatus("idle");
820             setAiMessage("");
821           }}
822         >
823           <X size={17} />
824         </button>
825       )}
826     </div>
827     <div className="table-toolbar">
828       <div className="legend">
829         <span>
830           <i className="dot-exact" />
831           {exact.length} identifier matches
832         </span>
833         <span>
834           <i className="dot-missing" />
835           {uncertain.length} need review
836         </span>
837         <span>
838           <i className="dot-unlisted" />
839           {matches.length - attention.length} not listed
840         </span>
841       </div>
842       <label className="search-box">
843         <Search size={15} />
844         <input
845           placeholder="Search inventory\u2026"
846           aria-label="Search inventory"
847           value={query}
848           onChange={(e) => setQuery(e.target.value)}
849         />
850       </label>
851     </div>
852     <div className="inventory-review-list">
853       {matches
854         .filter((m) =>
855           [
856             m.item.product,
857             m.item.catalog,
858             m.item.lot,
859             m.item.location,
860           ]
861             .join(" ")
862             .toLowerCase()
863             .includes(query.toLowerCase()),
864         )
865         .map((m) => (
866           <MatchRow
867             key={m.item.id}
868             match={m}
869             action={activeAction(state, notice.id, m.item.id)}
870             onAction={() => setActionMatch(m)}
871             sourceApproved={!!notice.approvedAt}
872           />
873         ))}
874       {!matches.length && (

```

app/lotlight.tsx (continued)

```

875         <Empty
876           title="Your shelves are next"
877           text="Import a CSV to reconcile stock against this source."
878           action={
879             <button
880               className="button"
881               onClick={() => setInventoryDialog(true)}
882             >
883               Import inventory
884             </button>
885           }
886         />
887       )}
888     </div>
889     <p className="fine-print">
890       AI similarity is not a probability of recall. A missing
891       or unlisted identifier never establishes safety. Model:
892       MiniLM-L6-v2, quantized, running on this device.
893     </p>
894   </TabsContent>
895   <TabsContent value="source">
896     <SourceReview
897       notice={notice}
898       busy={busy}
899       onApprove={() =>
900         send({
901           type: "approve_notice",
902           noticeId: notice.id,
903           sourceConfirmed: true,
904         })
905       }
906       onEdit={() => {
907         setEditing(notice);
908         setNoticeDialog(true);
909       }}
910     />
911   </TabsContent>
912   <TabsContent value="response">
913     <div className="section-heading response-heading">
914       <div>
915         <h2>Every open line gets a next step</h2>
916         <p className="fine-print">
917           {completed} of {attention.length} responses
918           complete. Missing identifiers stay open until
919           corrected in inventory.
920         </p>
921       </div>
922       {!notice.approvedAt && (
923         <span className="status-tag">
924           Review the source first
925         </span>
926       )}
927     </div>
928     {attention.map((m) => (
929       <MatchRow
930         key={m.item.id}
931         match={m}
932         action={activeAction(state, notice.id, m.item.id)}
933         onAction={() => setActionMatch(m)}
934         sourceApproved={!notice.approvedAt}
935       />
936     ))}
937     {!attention.length && (
938       <Empty
939         title="No listed identifier matches"
940         text="This result covers only this reviewed scope. It is not a safety
certification."
941       />
942     )}
943   </TabsContent>
944 </Tabs>
945 </>
946 )}
947 {view === "inventory" && (
948   <
949     <div className="page-heading compact">
950       <div>
951         <p className="eyebrow">
952           THE STOCKROOM, WITHOUT THE GUESSWORK
953         </p>

```

app/lotlight.tsx (continued)

```

954         <h1>Your inventory.</h1>
955         <p className="intro">
956             {state.inventoryProvenance ?? "synthetic"} \xb7{" "}
957             {state.inventory.length} lines \xb7{" "}
958             {state.inventory.reduce((n, i) => n + i.quantity, 0)}{" "}
959             units \xb7 Inventory revision {state.inventoryVersion}
960         </p>
961     </div>
962     <button
963         className="button primary"
964         onClick={() => setInventoryDialog(true)}
965     >
966         <Upload size={16} /> Import CSV
967     </button>
968 </div>
969 <div className="info-banner">
970     <Package size={17} />
971     <span>
972         Missing catalog or lot numbers are preserved and flagged.
973         Replacing inventory opens a new review version and
974         archives previous response actions.
975     </span>
976 </div>
977 <div className="data-table">
978     <Table>
979         <TableHeader>
980             <TableRow>
981                 {[
982                     "Product",
983                     "Manufacturer",
984                     "Catalog",
985                     "Lot",
986                     "Quantity",
987                     "Location",
988                     "Review",
989                 ].map((h) => (
990                     <TableHead key={h}>{h}</TableHead>
991                 ))}
992             </TableRow>
993         </TableHeader>
994         <TableBody>
995             {state.inventory.map((i) => (
996                 <TableRow key={i.id}>
997                     <TableCell className="product-cell">
998                         {i.product}
999                     </TableCell>
1000                     <TableCell>
1001                         {i.manufacturer || (
1002                             <span className="missing-text">Missing</span>
1003                         )}
1004                     </TableCell>
1005                     <TableCell>
1006                         <code>{i.catalog || "Missing"}</code>
1007                     </TableCell>
1008                     <TableCell>
1009                         <code className={`!i.lot ? "missing-text" : ""}>
1010                             {i.lot || "Missing"}
1011                         </code>
1012                     </TableCell>
1013                     <TableCell>{i.quantity}</TableCell>
1014                     <TableCell>{i.location}</TableCell>
1015                     <TableCell>
1016                         <button
1017                             className="text-button"
1018                             onClick={() => setCorrecting(i)}
1019                             aria-label={`Correct ${i.product} in ${i.location}`}
1020                         >
1021                             Correct <ArrowRight size={12} />
1022                         </button>
1023                     </TableCell>
1024                 </TableRow>
1025             ))}
1026         </TableBody>
1027     </Table>
1028 </div>
1029 {state.inventory.length && (
1030     <Empty
1031         title="Nothing on the shelf yet"
1032         text="Download the template, add your inventory, and import it."
1033     />

```

app/lotlight.tsx (continued)

```

1034      })
1035      <button
1036        className="text-button download-template"
1037        onClick={() =>
1038          download(
1039            "lotlight-inventory-template.csv",
1040            SAMPLE_CSV,
1041            "text/csv",
1042          )
1043        }
1044      >
1045        <Download size={14} /> Download synthetic example CSV
1046      </button>
1047    </>
1048  )}
1049  {view === "actions" && (
1050    <>
1051      <div className="page-heading compact">
1052        <div>
1053          <p className="eyebrow">ACCOUNTABILITY YOU CAN FOLLOW</p>
1054          <h1>The response record.</h1>
1055          <p className="intro">
1056            Staff-recorded activity, with source and inventory
1057            versions.
1058          </p>
1059        </div>
1060        <button
1061          className="button"
1062          onClick={() => setExportOpen(true)}
1063        >
1064          <Download size={15} /> Export record
1065        </button>
1066      </div>
1067      <div className="action-cards">
1068        {state.actions.map((a) => {
1069          const n = state.notices.find((n) => n.id === a.noticeId);
1070          const historic =
1071            activeAction(state, a.noticeId, a.itemId)?.id !== a.id;
1072          return (
1073            <div key={a.id} className="action-card">
1074              <div>
1075                <span
1076                  className={
1077                    "pill " +
1078                    (historic
1079                      ? "unlisted"
1080                      : a.status === "completed"
1081                        ? "complete"
1082                        : "missing")
1083                >
1084              >
1085                {historic ? "Historical version" : a.status}
1086              </span>
1087              <h3>{a.itemLabel}</h3>
1088              <p>
1089                {a.owner} \xb7 Due {date(a.dueDate)} \xb7{" "}
1090                {a.handledQuantity} units handled
1091              </p>
1092              <p>{a.evidence || "Response evidence pending"}</p>
1093            </div>
1094            <span className="mono">{date(a.updatedAt)}</span>
1095          </div>
1096        );
1097      )}
1098    </div>
1099    {!state.actions.length && (
1100      <Empty
1101        title="The paper trail starts here"
1102        text="Review a notice\u2019s source, then assign a response to an inventory line."
1103        action={
1104          <button
1105            className="button"
1106            onClick={() => notice && openReview(notice)}
1107          >
1108            Open recall review <ArrowRight size={15} />
1109          </button>
1110        }
1111      </>
1112    )}
1113    <h2 className="activity-title">Activity timeline</h2>

```

app/lotlight.tsx (continued)

```

1114         <div className="timeline">
1115             {[...state.audit].reverse().map((e) => (
1116                 <div key={e.id} className="timeline-item">
1117                     <span className="timeline-dot" />
1118                     <div>
1119                         <strong>{e.type.replace(/[_\.]/g, " ")}</strong>
1120                         <p>{e.detail}</p>
1121                         <small>
1122                             {e.actor} \xb7 {new Date(e.at).toLocaleString()}
1123                         </small>
1124                     </div>
1125                 </div>
1126             )]}
1127         </div>
1128     </>
1129 )}
1130 {view === "about" && <About />}
1131 {view === "report" && (
1132     <>
1133         <div className="report-toolbar">
1134             <button
1135                 className="text-button"
1136                 onClick={() => setView("overview")}
1137             >
1138                 <ArrowLeft size={14} /> Workspace
1139             </button>
1140             <button
1141                 className="button primary"
1142                 onClick={() => window.print()}
1143             >
1144                 <Download size={15} /> Print / save PDF
1145             </button>
1146         </div>
1147         <AuditReport state={state} />
1148     </>
1149 )}
1150 <footer className="page-footer">
1151     <span>
1152         <ShieldCheck size={14} /> Decision support. Verify current
1153         manufacturer instructions before acting.
1154     </span>
1155     <span>Built with care, by Shivam Gupta</span>
1156 </footer>
1157 </>
1158 )}
1159 </main>
1160 </div>
1161 {correcting && (
1162     <CorrectionDialog
1163         item={correcting}
1164         busy={busy}
1165         onClose={() => setCorrecting(null)}
1166         onSave={async (command) => {
1167             if (await send(command)) setCorrecting(null);
1168         }}
1169     />
1170 )}
1171 <NoticeDialog
1172     key={editing?.id ?? `new-${noticeNonce}`}
1173     open={noticeDialog}
1174     onOpenChange={setNoticeDialog}
1175     existing={editing}
1176     busy={busy}
1177     onSave={async (n) => {
1178         if (await send({ type: "save_notice", notice: n })) {
1179             setNoticeDialog(false);
1180             setNoticeNonce((x) => x + 1);
1181             setView("overview");
1182         }
1183     }}
1184 />
1185 <InventoryDialog
1186     open={inventoryDialog}
1187     onOpenChange={setInventoryDialog}
1188     busy={busy}
1189     onSave={async (csv, synthetic) => {
1190         if (await send({ type: "import_inventory", csv, synthetic })) {
1191             setInventoryDialog(false);
1192         }
1193     }}

```

app/lotlight.tsx (continued)

```

1194     />
1195     {actionMatch && notice && {
1196       <AlertDialog
1197         match={actionMatch}
1198         notice={notice}
1199         action={activeAction(state, notice.id, actionMatch.item.id)}
1200         busy={busy}
1201         onClose={() => setActionMatch(null)}
1202         onSave={async (command) => {
1203           if (await send(command)) setActionMatch(null);
1204         }}
1205       />
1206     }}
1207   <Dialog open={exportOpen} onOpenChange={setExportOpen}>
1208     <DialogContent>
1209       <DialogHeader>
1210         <DialogTitle>Take the evidence with you.</DialogTitle>
1211         <DialogDescription>
1212           Exports include source scope, unresolved lines, and staff-recorded
1213           actions. They do not certify device safety.
1214         </DialogDescription>
1215       </DialogHeader>
1216       <div className="export-options">
1217         <button
1218           onClick={() => {
1219             void exportAudit(state, !!user);
1220             setExportOpen(false);
1221           }}
1222         >
1223           <FileText />
1224           <span>
1225             <strong>Complete audit record</strong>
1226             <small>
1227               JSON with source text, versions, activity, and SHA-256 digest
1228             </small>
1229           </span>
1230           <Download size={17} />
1231         </button>
1232         <button
1233           onClick={() => {
1234             exportMatches(state, !!user);
1235             setExportOpen(false);
1236           }}
1237         >
1238           <Layers />
1239           <span>
1240             <strong>Inventory review spreadsheet</strong>
1241             <small>
1242               CSV with identifiers, match rationale, and response status
1243             </small>
1244           </span>
1245           <Download size={17} />
1246         </button>
1247         <button
1248           onClick={() => {
1249             setView("report");
1250             setExportOpen(false);
1251           }}
1252         >
1253           <ClipboardCheck />
1254           <span>
1255             <strong>Readable audit report</strong>
1256             <small>Print or save as PDF from your browser</small>
1257           </span>
1258           <ArrowRight size={17} />
1259         </button>
1260       </div>
1261     </DialogContent>
1262   </Dialog>
1263   <AlertDialog
1264     open={!!reset}
1265     onOpenChange={(open) => {
1266       if (!open) setReset(null);
1267     }}
1268   >
1269     <AlertDialogContent>
1270       <AlertDialogTitle>
1271         {reset === "clear_workspace"
1272           ? "Start an empty workspace?"
1273           : "Restart the demonstration?"}

```

app/lotlight.tsx (continued)

```

1274     </AlertDialogTitle>
1275     <AlertDialogDescription>
1276       This replaces the current inventory, notices, and response records.
1277       Export your audit record first if you need to keep it.{ " "}
1278       {user
1279         ? "The change will update your saved account."
1280         : "Your demo is temporary and unsaved."}
1281     </AlertDialogDescription>
1282     <AlertDialogFooter>
1283       <AlertDialogCancel>Keep my workspace</AlertDialogCancel>
1284       <AlertDialogAction
1285         onClick={() => {
1286           if (reset)
1287             void send({ type: reset }).then((ok) => {
1288               if (ok) {
1289                 setView("overview");
1290                 setNoticeId("baxter-demo");
1291               }
1292             });
1293           setReset(null);
1294         }}
1295       >
1296         Replace workspace
1297       </AlertDialogAction>
1298     </AlertDialogFooter>
1299   </AlertDialogContent>
1300 </AlertDialog>
1301 </SidebarProvider>
1302 );
1303 }
1304 function Empty({
1305   title,
1306   text,
1307   action,
1308 }: {
1309   title: string;
1310   text: string;
1311   action?: React.ReactNode;
1312 }) {
1313   return (
1314     <div className="empty-state">
1315       <ScanLine size={26} />
1316       <h3>{title}</h3>
1317       <p>{text}</p>
1318       {action}
1319     </div>
1320   );
1321 }
1322 function MatchRow({
1323   match: m,
1324   action,
1325   onAction,
1326   sourceApproved,
1327 }: {
1328   match: Match;
1329   action?: ResponseAction;
1330   onAction: () => void;
1331   sourceApproved: boolean;
1332 }) {
1333   const [expanded, setExpanded] = useState(false);
1334   return (
1335     <article className={"match-row " + m.kind}>
1336       <div className="match-row-main">
1337         <div className="match-product">
1338           <span className={"match-symbol " + m.kind}>
1339             {m.kind === "exact" ? (
1340               <ScanLine size={18} />
1341             ) : m.kind === "unlisted" ? (
1342               <Package size={18} />
1343             ) : (
1344               <AlertTriangle size={18} />
1345             )}
1346           </span>
1347           <div>
1348             <h3>{m.item.product}</h3>
1349             <p>
1350               {m.item.location} <span>\xb7</span> {m.item.quantity} units{" "}
1351               <span>\xb7</span> {m.item.manufacturer || "Manufacturer missing"}
1352             </p>
1353           </div>

```

app/lotlight.tsx (continued)

```

1354     </div>
1355     <div className="match-identifiers">
1356       <code>{m.item.catalog || "No catalog"}</code>
1357       <span>
1358         LOT <b>{m.item.lot || "Missing"}</b>
1359       </span>
1360     </div>
1361     <div className="match-state">
1362       <span
1363         className={
1364           "pill " + (action?.status === "completed" ? "complete" : m.kind)
1365         }
1366       >
1367         {action?.status === "completed"
1368           ? "Response complete"
1369           : LABELS[m.kind]}
1370       </span>
1371       {m.similarity !== undefined && (
1372         <small>AI similarity {m.similarity.toFixed(2)}</small>
1373       )}
1374     </div>
1375     <button
1376       className="icon-button"
1377       aria-label={`Show evidence for ${m.item.product} in ${m.item.location}`}
1378       aria-expanded={expanded}
1379       onClick={() => setExpanded(!expanded)}
1380     >
1381       <ChevronRight size={17} className={expanded ? "rotate-90" : ""} />
1382     </button>
1383   </div>
1384   {expanded && (
1385     <div className="match-expanded">
1386       <div>
1387         <strong>Why this result</strong>
1388         <p>{m.reason}</p>
1389         <blockquote>{m.evidence}</blockquote>
1390         {action && (
1391           <p className="action-summary">
1392             Owner: {action.owner} \xb7 {action.status} \xb7{" "}
1393             {action.handledQuantity}/{m.item.quantity} units handled
1394           </p>
1395         )}
1396       </div>
1397       {m.kind !== "unlisted" && (
1398         <div>
1399           <button
1400             className="button"
1401             disabled={!sourceApproved}
1402             onClick={onAction}
1403           >
1404             {action ? "Update response" : "Assign response"}{" "}
1405             <ArrowRight size={14} />
1406           </button>
1407           {!sourceApproved && (
1408             <small className="gate-hint">
1409               Confirm the Source & scope tab first.
1410             </small>
1411           )}
1412         </div>
1413       )}
1414     </div>
1415   )}
1416 </article>
1417 );
1418 }
1419 function SourceReview({
1420   notice,
1421   busy,
1422   onApprove,
1423   onEdit,
1424 }): {
1425   notice: Notice;
1426   busy: boolean;
1427   onApprove: () => Promise<boolean>;
1428   onEdit: () => void;
1429 }) {
1430   const [checked, setChecked] = useState(false);
1431   return (
1432     <div className="source-grid">
1433       <section className="source-document">

```

app/lotlight.tsx (continued)

```

1434     <div className="source-document-heading">
1435       <FileText size={18} />
1436       <strong>Preserved source text</strong>
1437       <a href={notice.sourceUrl} target="_blank" rel="noreferrer">
1438         Open original <ExternalLink size={13} />
1439       </a>
1440     </div>
1441     <pre>{notice.text}</pre>
1442     <button className="text-button" onClick={onEdit}>
1443       Edit source or paired scope <ArrowRight size={14} />
1444     </button>
1445   </section>
1446   <aside className="source-review-panel">
1447     <span className="eyebrow">HUMAN REVIEW GATE</span>
1448     <h3>The source sets the scope.</h3>
1449     <p>
1450       Confirm catalog-to-lot pairs against the full current notice. Ranges,
1451       exclusions, kit components, and packaging exceptions need specialist
1452       review. Supported scope uses one labeled catalog/lot line at a time.
1453     </p>
1454     {notice.scope.map((s, i) => (
1455       <div className="scope-review-row" key={i}>
1456         <strong>{s.catalog}</strong>
1457         <span>{s.allLots ? "All lots" : s.lots.join(", ")}</span>
1458         <small>\u201c{s.evidence}\u201d</small>
1459       </div>
1460     ))}
1461     <div className="source-action">
1462       <strong>Response summary</strong>
1463       <p>{notice.action}</p>
1464       <small>
1465         Staff-entered summary. The original instructions control.
1466       </small>
1467     </div>
1468     {notice.approvedAt ? (
1469       <div className="approved-box">
1470         <CheckCircle2 size={20} />
1471         <div>
1472           <strong>Source review recorded</strong>
1473           <small>
1474             {notice.approvedBy}
1475             <br />
1476             {new Date(notice.approvedAt).toLocaleString()}
1477           </small>
1478         </div>
1479       </div>
1480     ) : (
1481       <>
1482         <label className="check-label">
1483           <Checkbox
1484             checked={checked}
1485             onChange={(value) => setChecked(value === true)}
1486             id="source-check"
1487           />
1488           <span>
1489             {notice.training
1490               ? "For this demonstration, I have reviewed the source link and confirmed these two
1491               training pairs."
1492               : "I checked the current original notice, including affected pairs, exceptions, and
1493               response instructions."}
1494           </span>
1495         </label>
1496         <button
1497           className="button primary full-width"
1498           disabled={!checked || busy}
1499           onClick={() => void onApprove()}
1500         >
1501           {busy ? (
1502             <Loader2 size={15} className="spin" />
1503           ) : (
1504             <Check size={15} />
1505           )}{ " "}
1506           Confirm source review
1507         </button>
1508       </>
1509     )}
1510   </aside>
1511 </div>
);
}

```

app/lotlight.tsx (continued)

```

1512 function InventoryDialog({
1513   open,
1514   onOpenChange,
1515   busy,
1516   onSave,
1517 }): {
1518   open: boolean;
1519   onOpenChange: (b: boolean) => void;
1520   busy: boolean;
1521   onSave: (s: string, synthetic: boolean) => Promise<void>;
1522 }) {
1523   const [csv, setCsv] = useState("");
1524   const [synthetic, setSynthetic] = useState(false);
1525   const [error, setError] = useState("");
1526   const preview = useMemo(() => {
1527     try {
1528       return csv ? parseCsv(csv) : null;
1529     } catch {
1530       return null;
1531     }
1532   }, [csv]);
1533   return (
1534     <Dialog open={open} onOpenChange={onOpenChange}>
1535       <DialogContent className="wide-dialog">
1536         <DialogHeader>
1537           <DialogTitle>Bring your inventory.</DialogTitle>
1538           <DialogDescription>
1539             CSV columns: product, manufacturer, catalog, lot, quantity,
1540             location. Up to 500 lines. Use equipment and supply records only,
1541             without patient information.
1542           </DialogDescription>
1543         </DialogHeader>
1544         <label className="file-input-label">
1545           <Upload size={18} /> Choose a CSV
1546           <input
1547             type="file"
1548             accept=".csv,text/csv"
1549             aria-label="Upload inventory CSV"
1550             onChange={async (e) => {
1551               const file = e.target.files?.[0];
1552               if (file) {
1553                 if (file.size > 500000) {
1554                   setError("Use a CSV smaller than 500 KB.");
1555                   return;
1556                 }
1557                 setCsv(await file.text());
1558                 setError("");
1559               }
1560             }}
1561           />
1562         </label>
1563         <label className="field-label">
1564           Or paste CSV
1565           <textarea
1566             aria-label="Inventory CSV"
1567             rows={9}
1568             value={csv}
1569             onChange={(e) => {
1570               setCsv(e.target.value);
1571               setError("");
1572             }}
1573             placeholder="product,manufacturer,catalog,lot,quantity,location"
1574           />
1575         </label>
1576         <div className="form-meta">
1577           <button
1578             className="text-button"
1579             onClick={() => {
1580               setCsv(SAMPLE_CSV);
1581               setSynthetic(true);
1582             }}
1583           >
1584             Use synthetic example
1585           </button>
1586           <span>
1587             {preview
1588               ? `${preview.length} valid lines \xb7 ${preview.filter((i) => !i.lot || !i.catalog).length}
with missing identifiers`
1589               : "Paste or upload to validate"}
1590           </span>

```

app/lotlight.tsx (continued)

```

1591     </div>
1592     {error && (
1593       <p className="form-error" role="alert">
1594         {error}
1595       </p>
1596     )}
1597     <label className="check-label">
1598       <Checkbox
1599         checked={synthetic}
1600         onCheckedChange={(v) => setSynthetic(v === true)}
1601       />
1602       <span>This inventory is synthetic demonstration data</span>
1603     </label>
1604     <p className="fine-print">
1605       Import replaces the inventory. Previous response actions become
1606       historical; they cannot close new inventory obligations.
1607     </p>
1608     <DialogFooter>
1609       <button className="button" onClick={() => onOpenChange(false)}>
1610         Cancel
1611       </button>
1612       <button
1613         className="button primary"
1614         disabled={busy || !csv}
1615         onClick={() => {
1616           try {
1617             parseCsv(csv);
1618             void onSave(csv, synthetic);
1619           } catch (e) {
1620             setError(e instanceof Error ? e.message : "Invalid CSV");
1621           }
1622         }}
1623       >
1624         Validate & import <ArrowRight size={14} />
1625       </button>
1626     </DialogFooter>
1627   </DialogContent>
1628 </Dialog>
1629 );
1630 }
1631 function NoticeDialog({
1632   open,
1633   onOpenChange,
1634   existing,
1635   busy,
1636   onSave,
1637 }: {
1638   open: boolean;
1639   onOpenChange: (b: boolean) => void;
1640   existing?: Notice;
1641   busy: boolean;
1642   onSave: (
1643     n: Extract<Command, { type: "save_notice" }>["notice"],
1644   ) => Promise<void>;
1645 }) {
1646   const [title, setTitle] = useState(existing?.title ?? "");
1647   const [manufacturer, setManufacturer] = useState(
1648     existing?.manufacturer ?? "",
1649   );
1650   const [noticeDate, setDate] = useState(
1651     existing?.date ?? new Date().toISOString().slice(0, 10),
1652   );
1653   const [url, setUrl] = useState(existing?.sourceUrl ?? "");
1654   const [text, setText] = useState(existing?.text ?? "");
1655   const [action, setAction] = useState(existing?.action ?? "");
1656   const [scope, setScope] = useState<Scope[]>(existing?.scope ?? []);
1657   const [training, setTraining] = useState(existing?.training ?? false);
1658   const [error, setError] = useState("");
1659   const [reading, setReading] = useState(false);
1660   const editScope = (i: number, patch: Partial<Scope>) =>
1661     setScope((rows) => rows.map((r, j) => (j === i ? { ...r, ...patch } : r)));
1662   return (
1663     <Dialog open={open} onOpenChange={onOpenChange}>
1664       <DialogContent className="notice-dialog">
1665         <DialogHeader>
1666           <DialogTitle>
1667             {existing
1668               ? "Revise the source. Reopen the review."
1669               : "Start with the source."}
1670           </DialogTitle>

```

app/lotlight.tsx (continued)

```

1671     <DialogDescription>
1672       Import a text-based PDF or paste the notice. Preserve paired scope
1673       and supporting excerpts. OCR and automatic range interpretation are
1674       not included.
1675     </DialogDescription>
1676   </DialogHeader>
1677   <Form
1678     onSubmit={(e) => {
1679       e.preventDefault();
1680       const n = {
1681         id: existing?.id,
1682         title,
1683         manufacturer,
1684         date: noticeDate,
1685         sourceUrl: url,
1686         text,
1687         action,
1688         scope,
1689         training,
1690       };
1691       const problem = validateScope(n);
1692       if (problem) {
1693         setError(problem);
1694         return;
1695       }
1696       setError("");
1697       void onSave(n);
1698     }}
1699   >
1700     <div className="form-two">
1701       <label className="field-label">
1702         Product / review title
1703         <input
1704           required
1705           value={title}
1706           onChange={(e) => setTitle(e.target.value)}
1707           placeholder="e.g. CLEARLINK IV sets"
1708         />
1709       </label>
1710       <label className="field-label">
1711         Manufacturer as listed in inventory
1712         <input
1713           required
1714           value={manufacturer}
1715           onChange={(e) => setManufacturer(e.target.value)}
1716           placeholder="e.g. Baxter"
1717         />
1718       </label>
1719       <label className="field-label">
1720         Notice date
1721         <input
1722           required
1723           type="date"
1724           value={noticeDate}
1725           onChange={(e) => setDate(e.target.value)}
1726         />
1727       </label>
1728       <label className="field-label">
1729         Original source URL
1730         <input
1731           required
1732           type="url"
1733           value={url}
1734           onChange={(e) => setUrl(e.target.value)}
1735           placeholder="https://manufacturer.example/notice"
1736         />
1737       </label>
1738     </div>
1739     <div className="source-upload-row">
1740       <label className="file-input-label">
1741         <Upload size={16} />
1742         {reading ? "Reading PDF\u2026" : "Import PDF or text"}
1743         <input
1744           type="file"
1745           accept=".pdf,.txt"
1746           aria-label="Upload recall notice"
1747           disabled={reading}
1748           onChange={async (e) => {
1749             const f = e.target.files?.[0];
1750             if (!f) return;

```

app/lotlight.tsx (continued)

```

1751         setReading(true);
1752         try {
1753             const t = await readNoticeFile(f);
1754             if (t.length > 50000)
1755                 throw new Error("Notice text exceeds 50,000 characters.");
1756             setText(t);
1757             setScope(extractScope(t));
1758             setError("");
1759         } catch (error) {
1760             setError(
1761                 error instanceof Error
1762                     ? error.message
1763                     : "Could not read file.",
1764             );
1765         } finally {
1766             setReading(false);
1767         }
1768     }}
1769     />
1770 </label>
1771 <button
1772     type="button"
1773     className="text-button"
1774     onClick={() => {
1775         setTitle("CLEARLINK IV sets");
1776         setManufacturer("Baxter");
1777         setDate("2025-08-29");
1778         setUrl(BAXTER_URL);
1779         setText(SAMPLE_NOTICE_TEXT);
1780         setScope(extractScope(SAMPLE_NOTICE_TEXT));
1781         setTraining(true);
1782         setAction(
1783             "Identify affected stock and follow the manufacturer\u2019s supplier-specific isolation,
return or replacement, and acknowledgement instructions. Verify the current full notice first.",
1784         );
1785     }}
1786 >
1787     Use historical training extract
1788 </button>
1789 </div>
1790 <label className="field-label">
1791     Original notice text / clearly labeled training extract
1792     <textarea
1793         required
1794         rows={6}
1795         value={text}
1796         onChange={(e) => setText(e.target.value)}
1797         maxLength={50000}
1798     />
1799 </label>
1800 <div className="section-heading scope-editor-heading">
1801     <h2>Paired catalog / lot scope</h2>
1802     <button
1803         type="button"
1804         className="text-button"
1805         onClick={() => {
1806             const found = extractScope(text);
1807             setScope(found);
1808             if (!found.length)
1809                 setError(
1810                     "No explicitly labeled pairs found. Add rows manually and copy supporting excerpts from
the source.",
1811                 );
1812             }}
1813     >
1814         Extract labeled pairs
1815     </button>
1816 </div>
1817 <p className="fine-print">
1818     Auto-extraction accepts \u201cCatalog: ABC; Lots: 123, 456\u201d. For tables,
1819     append a staff-checked labeled transcription to the preserved
1820     source. Each scope excerpt must contain one complete pair.
1821     Conditional ranges are unsupported.
1822 </p>
1823 {scope.map((s, i) => (
1824     <div className="scope-editor" key={i}>
1825         <div className="scope-editor-fields">
1826             <label className="field-label">
1827                 Catalog
1828                 <input

```

app/lotlight.tsx (continued)

```

1829         required
1830         value={s.catalog}
1831         onChange={(e) => editScope(i, { catalog: e.target.value })}
1832       />
1833     </label>
1834     <label className="field-label">
1835       Exact lots, comma-separated
1836       <input
1837         value={s.lots.join(", ")}
1838         disabled={s.allLots}
1839         onChange={(e) =>
1840           editScope(i, {
1841             lots: e.target.value.split(",").map((l) => l.trim()),
1842           })
1843         />
1844     </label>
1845     <button
1846       type="button"
1847       className="icon-button"
1848       aria-label={`Remove scope row ${i + 1}`}
1849       onClick={() =>
1850         setScope((rows) => rows.filter((_, j) => i !== j))
1851       />
1852     </button>
1853   </div>
1854   <label className="check-label">
1855     <Checkbox
1856       checked={s.allLots}
1857       onChange={(v) =>
1858         editScope(i, {
1859           allLots: v === true,
1860           lots: v === true ? [] : s.lots,
1861         })
1862       />
1863     <span>
1864       Original source explicitly says \u201call lots\u201d without exceptions
1865     </span>
1866   </label>
1867   <label className="field-label">
1868     Verbatim excerpt supporting this specific pair
1869     <textarea
1870       required
1871       rows={2}
1872       value={s.evidence}
1873       onChange={(e) => editScope(i, { evidence: e.target.value })}
1874     />
1875   </label>
1876 </div>
1877 )))
1878 <button
1879   type="button"
1880   className="text-button add-scope"
1881   onClick={() =>
1882     setScope([
1883       ...scope,
1884       { catalog: "", lots: [], allLots: false, evidence: "" },
1885     ])
1886   />
1887 </button>
1888 <label className="field-label">
1889   Staff-entered response summary
1890   <textarea
1891     required
1892     minLength={15}
1893     rows={3}
1894     value={action}
1895     onChange={(e) => setAction(e.target.value)}
1896     placeholder="Summarize the actual manufacturer instructions. Do not infer stop-use from the
1897 word recall."
1898   />
1899 </label>
1900 <label className="check-label">
1901   <Checkbox
1902     checked={training}

```

app/lotlight.tsx (continued)

```

1908         onChangeChange={(v) => setTraining(v === true)}
1909     />
1910     <span>This is a historical or synthetic training example</span>
1911 </label>
1912 {error && (
1913     <p className="form-error" role="alert">
1914         {error}
1915     </p>
1916 )}
1917 <DialogFooter>
1918     <button
1919         type="button"
1920         className="button"
1921         onClick={() => onOpenChange(false)}
1922     >
1923         Cancel
1924     </button>
1925     <button
1926         type="submit"
1927         className="button primary"
1928         disabled={busy || reading}
1929     >
1930         Save for source review <ArrowRight size={14} />
1931     </button>
1932 </DialogFooter>
1933 </form>
1934 </DialogContent>
1935 </Dialog>
1936 );
1937 }
1938 function ActionDialog({
1939     match,
1940     notice,
1941     action,
1942     busy,
1943     onClose,
1944     onSave,
1945 }: {
1946     match: Match;
1947     notice: Notice;
1948     action?: ResponseAction;
1949     busy: boolean;
1950     onClose: () => void;
1951     onSave: (c: Command) => Promise<void>;
1952 }) {
1953     const [owner, setOwner] = useState(action?.owner ?? "");
1954     const [due, setDue] = useState(
1955         action?.dueDate ??
1956         new Date(Date.now() + 86400000).toISOString().slice(0, 10),
1957     );
1958     const [status, setStatus] = useState<ResponseAction["status"]>(
1959         action?.status ?? "assigned",
1960     );
1961     const [verification, setVerification] = useState(action?.verification ?? "");
1962     const [evidence, setEvidence] = useState(action?.evidence ?? "");
1963     const [handled, setHandled] = useState(action?.handledQuantity ?? 0);
1964     return (
1965         <Dialog
1966             open
1967             onChange={{(open) => {
1968                 if (!open) onClose();
1969             }}}
1970         >
1971             <DialogContent className="wide-dialog">
1972                 <DialogHeader>
1973                     <DialogTitle>A clear next step.</DialogTitle>
1974                     <DialogDescription>
1975                         {match.item.product} \xb7 {match.item.catalog || "Catalog missing"} \xb7{" "}
1976                         {match.item.lot || "Lot missing"} \xb7 {match.item.quantity} units
1977                     </DialogDescription>
1978                 </DialogHeader>
1979                 <form
1980                     onSubmit={(e) => {
1981                         e.preventDefault();
1982                         void onSave({
1983                             type: "save_action",
1984                             noticeId: notice.id,
1985                             itemId: match.item.id,
1986                             owner,
1987                             dueDate: due,

```

app/lotlight.tsx (continued)

```

1988         status,
1989         verification,
1990         evidence,
1991         handledQuantity: handled,
1992     });
1993 }
1994 >
1995 <div className="form-two">
1996   <label className="field-label">
1997     Responsible person
1998     <input
1999       required
2000       minLength={2}
2001       value={owner}
2002       onChange={(e) => setOwner(e.target.value)}
2003       placeholder="e.g. Maya Rao, stock coordinator"
2004     />
2005   </label>
2006   <label className="field-label">
2007     Response due date
2008     <input
2009       required
2010       type="date"
2011       value={due}
2012       onChange={(e) => setDue(e.target.value)}
2013     />
2014   </label>
2015 </div>
2016 <label className="field-label">
2017   Response stage
2018   <Select
2019     value={status}
2020     onChange={(v) => setStatus(v as ResponseAction["status"])}
2021   >
2022     <SelectTrigger className="full-width" aria-label="Response stage">
2023       <SelectValue />
2024     </SelectTrigger>
2025     <SelectContent>
2026       <SelectItem value="assigned">
2027         Assigned \xb7 review pending
2028       </SelectItem>
2029       <SelectItem value="verified" disabled={match.kind !== "exact"}>
2030         Verified \xb7 physical label checked
2031       </SelectItem>
2032       <SelectItem value="completed" disabled={match.kind !== "exact"}>
2033         Completed \xb7 response evidence recorded
2034       </SelectItem>
2035     </SelectContent>
2036   </Select>
2037 </label>
2038 {match.kind !== "exact" && (
2039   <div className="info-banner">
2040     <AlertTriangle size={16} />
2041     <span>
2042       Collect missing identifiers, then replace the inventory with
2043       corrected records. This uncertain line cannot be marked
2044       complete.
2045     </span>
2046   </div>
2047 )}
2048 <label className="field-label">
2049   Physical-label and current-source verification
2050   <textarea
2051     rows={2}
2052     value={verification}
2053     onChange={(e) => setVerification(e.target.value)}
2054     placeholder="Record what you checked, by whom, and against which current instructions."
2055   />
2056 </label>
2057 <div className="form-two">
2058   <label className="field-label">
2059     Units handled
2060     <input
2061       type="number"
2062       min={0}
2063       max={match.item.quantity}
2064       required
2065       value={handled}
2066       onChange={(e) => setHandled(Number(e.target.value))}
2067   />

```

app/lotlight.tsx (continued)

```

2068         </label>
2069         <div className="quantity-note">
2070             {handled} of {match.item.quantity} units
2071             <br />
2072             <small>Partial handling keeps the response open.</small>
2073         </div>
2074     </div>
2075     <label className="field-label">
2076         Response evidence / reference
2077         <textarea
2078             rows={3}
2079             value={evidence}
2080             onChange={(e) => setEvidence(e.target.value)}
2081             placeholder="e.g. Staff-recorded isolation log Q-104, supplier return reference RMA-210. No
patient data."
2082         />
2083     </label>
2084     <p className="fine-print">
2085         Completion records the staff response for this stock and source
2086         version. It does not independently verify the evidence or terminate
2087         the recall.
2088     </p>
2089     <DialogFooter>
2090         <button type="button" className="button" onClick={onClose}>
2091             Cancel
2092         </button>
2093         <button
2094             type="submit"
2095             className="button primary"
2096             disabled={busy || !notice.approvedAt}
2097         >
2098             Save response <Check size={14} />
2099         </button>
2100     </DialogFooter>
2101 </form>
2102 </DialogContent>
2103 </Dialog>
2104 );
2105 }
2106 function About() {
2107     return (
2108         <div className="about-page">
2109             <p className="eyebrow">THE REASON BEHIND THE REVIEW</p>
2110             <h1>
2111                 A recall is a handoff.
2112             <br />
2113                 Someone has to catch it.
2114             </h1>
2115             <p className="intro large">
2116                 Lotlight helps independent clinics turn a manufacturer notice and a
2117                 spreadsheet into a review someone can explain, own, and finish.
2118             </p>
2119             <div className="about-story">
2120                 <span className="eyebrow">THE DEMONSTRATION</span>
2121                 <h2>
2122                     One letter. Four stock locations.
2123                 <br />
2124                     One missing lot number.
2125                 </h2>
2126                 <p>
2127                     Willow is a fictional clinic. Its eight inventory lines include two
2128                     exact pairs from a real Baxter notice, an incomplete label, and an \u201cIV
2129                     line extender\u201d that AI can surface despite different wording. Every
2130                     count is demonstration data.
2131                 </p>
2132                 <a
2133                     className="text-button"
2134                     href={BAXTER_URL}
2135                     target="_blank"
2136                     rel="noreferrer"
2137                 >
2138                     Read the original manufacturer notice <ExternalLink size={14} />
2139                 </a>
2140             </div>
2141             <div className="about-columns">
2142                 <section>
2143                     <h2>What the AI actually does</h2>
2144                     <p>
2145                         A quantized MiniLM sentence transformer compares product
2146                         descriptions locally in a browser worker. It can rank an alias for

```

app/lotlight.tsx (continued)

```

2147         review. It never supplies a recalled lot, clears an unknown
2148         identifier, writes medical instructions, or approves a response.
2149     </p>
2150 <p>
2151     Exact matching uses paired catalog and lot scope plus manufacturer
2152     identity. Source extraction supports explicitly labeled lines; other
2153     formats require human pairing.
2154 </p>
2155 <a
2156     href="https://huggingface.co/Xenova/all-MiniLM-L6-v2"
2157     target="_blank"
2158     rel="noreferrer"
2159 >
2160     Model card and Apache 2.0 license \u2197
2161 </a>
2162 </section>
2163 <section>
2164     <h2>A credible first customer</h2>
2165     <p>
2166         Our proposed customer is an independent clinic using spreadsheet
2167         stock records. Enterprise products such as ECRI already manage
2168         recalls. Lotlight explores a smaller, transparent
2169         inventory-reconciliation workflow.
2170     </p>
2171     <p>
2172         <strong>Pricing hypothesis: $149 per site per month.</strong> At an
2173         assumed loaded staff cost of $30/hour, five hours saved monthly
2174         would cover that price. This is a testable business hypothesis, not
2175         customer traction.
2176     </p>
2177     <a
2178         href="https://home.ecri.org/solutions/healthcare-product-alerts"
2179         target="_blank"
2180         rel="noreferrer"
2181     >
2182         Explore the established category \u2197
2183     </a>
2184 </section>
2185 <section>
2186     <h2>Clear boundaries</h2>
2187     <p>
2188         This is a tested hackathon MVP, not a clinically validated or
2189         certified recall-management system. It has private per-account
2190         storage, not shared clinic roles. It does not monitor all recalls,
2191         contact suppliers, provide OCR, or decide patient treatment.
2192     </p>
2193     <p>
2194         No patient information is needed. Public model files download on
2195         first AI use; inference happens on the device. Signed-in records are
2196         stored in the hosted database. Anonymous demo changes remain in
2197         memory and are lost on reload or sign-in.
2198     </p>
2199 </section>
2200 <section>
2201     <h2>Evidence, without overclaiming</h2>
2202     <p>
2203         The FDA explains that a recall can involve correction or removal.
2204         Follow the actual current instructions. openFDA is a weekly research
2205         source and explicitly does not support public alerts or recall
2206         lifecycle tracking. Lotlight therefore starts with an uploaded
2207         notice.
2208     </p>
2209     <a
2210         href="https://www.fda.gov/medical-devices/medical-device-recalls-and-early-alerts/what-medical-
device-recall"
2211         target="_blank"
2212         rel="noreferrer"
2213     >
2214         FDA: what a medical device recall means \u2197
2215     </a>
2216     <a
2217         href="https://open.fda.gov/apis/device/recall/"
2218         target="_blank"
2219         rel="noreferrer"
2220     >
2221         openFDA usage limitations \u2197
2222     </a>
2223 </section>
2224 </div>
2225 <div className="creator-credit">

```

app/lotlight.tsx (continued)

```

2226         <span className="brand-mark">
2227             <ScanLine size={24} />
2228         </span>
2229         <div>
2230             <strong>A project by Shivam Gupta</strong>
2231             <p>
2232                 Built for DSH Hacks V2 with AI-assisted research, implementation,
2233                 testing, and presentation preparation.
2234             </p>
2235         </div>
2236         <a
2237             href="https://github.com/shi1720/DSH-Hacks-V2"
2238             target="_blank"
2239             rel="noreferrer"
2240         >
2241             View the code \u2197
2242         </a>
2243     </div>
2244 </div>
2245 );
2246 }
2247 function AuditReport({ state }: { state: Workspace }) {
2248     return (
2249         <article className="audit-report">
2250             <div className="report-head">
2251                 <div className="brand">
2252                     <span className="brand-mark">
2253                         <ScanLine size={23} />
2254                     </span>
2255                     lotlight
2256                 </div>
2257                 <span>AUDIT RECORD \xb7 {date(new Date().toISOString())}</span>
2258             </div>
2259             <h1>Recall review record</h1>
2260             <p>
2261                 Workspace revision {state.revision}. Inventory revision{" "}
2262                 {state.inventoryVersion}. {state.inventory.length} stock lines.
2263                 Staff-recorded evidence, not independent verification.
2264             </p>
2265             <p>
2266                 Inventory provenance: {state.inventoryProvenance ?? "synthetic"}. Last
2267                 record update: {state.savedAt ?? "No recorded update"}.
2268             </p>
2269             <p className="report-notice">
2270                 {state.notices.some((n) => n.training)
2271                 ? "HISTORICAL TRAINING SOURCE: this does not show current recall status. Inventory provenance is
2272                 listed separately above."
2273                 : "USER-SUPPLIED RECORDS: verify complete current manufacturer instructions before acting."}
2274             </p>
2275             {state.notices.map((n) => (
2276                 <section key={n.id}>
2277                     <h2>
2278                         {n.title} \xb7 version {n.version}
2279                     </h2>
2280                     <p>
2281                         {n.manufacturer} \xb7 Notice {n.date} \xb7{" "}
2282                         {n.approvedAt
2283                         ? `Source review recorded by ${n.approvedBy} on ${date(n.approvedAt)}`
2284                         : "SOURCE NOT APPROVED"}
2285                     </p>
2286                     <a href={n.sourceUrl}>{n.sourceUrl}</a>
2287                     <h3>Source scope</h3>
2288                     {n.scope.map((s, i) => (
2289                         <p key={i}>
2290                             {s.catalog}: {s.allLots ? "All lots" : s.lots.join(", ")}
2291                             <br />
2292                             <em>{s.evidence}</em>
2293                         </p>
2294                     ))}
2295                     <h3>Response summary</h3>
2296                     <p>{n.action}</p>
2297                     <Table>
2298                         <TableHeader>
2299                             <TableRow>
2300                                 <TableHead>Stock / location</TableHead>
2301                                 <TableHead>Identifiers</TableHead>
2302                                 <TableHead>Result</TableHead>
2303                                 <TableHead>Response</TableHead>
2304                             </TableRow>

```

app/lotlight.tsx (continued)

```

2305         <TableBody>
2306             {reconcile(state.inventory, n).map((m) => {
2307                 const a = activeAction(state, n.id, m.item.id);
2308                 return (
2309                     <TableRow key={m.item.id}>
2310                         <TableCell>
2311                             {m.item.product}
2312                             <br />
2313                             {m.item.location} \xb7 {m.item.quantity} units
2314                         </TableCell>
2315                         <TableCell>
2316                             {m.item.catalog || "Missing catalog"}
2317                             <br />
2318                             {m.item.lot || "Missing lot"}
2319                         </TableCell>
2320                         <TableCell>
2321                             {LABELS[m.kind]}
2322                             <br />
2323                             <small>{m.reason}</small>
2324                         </TableCell>
2325                         <TableCell>
2326                             {a
2327                                 ? `${a.status} \xb7 ${a.owner} \xb7 ${a.handledQuantity} units handled`
2328                                 : "No response recorded"}
2329                             <br />
2330                             {a?.evidence}
2331                         </TableCell>
2332                     </TableRow>
2333                 );
2334             })}
2335         </TableBody>
2336     </Table>
2337 </section>
2338 )}
2339 <h2>Activity history</h2>
2340 {state.audit.map((e) => (
2341     <div className="report-event" key={e.id}>
2342         <strong>
2343             {e.type} \xb7 {e.actor}
2344         </strong>
2345         <small>{e.at}</small>
2346         <p>{e.detail}</p>
2347     </div>
2348 )}
2349 <p className="fine-print">
2350     No listed match is not a safety clearance. The JSON export includes full
2351     source text and historical response records. No inference about patient
2352     safety or FDA recall termination is made.
2353 </p>
2354 </article>
2355 );
2356 }
2357 function CorrectionDialog({
2358     item,
2359     busy,
2360     onClose,
2361     onSave,
2362 }: {
2363     item: InventoryItem;
2364     busy: boolean;
2365     onClose: () => void;
2366     onSave: (c: Command) => Promise<void>;
2367 }) {
2368     const [form, setForm] = useState(item);
2369     return (
2370         <Dialog
2371             open
2372             onOpenChange={(v) => {
2373                 if (!v) onClose();
2374             }}
2375         >
2376         <DialogContent className="wide-dialog">
2377             <DialogHeader>
2378                 <DialogTitle>Correct the physical record.</DialogTitle>
2379                 <DialogDescription>
2380                     Check the label before editing. This line gets a new revision and
2381                     fresh response review. Other unchanged lines keep their current
2382                     response state.
2383                 </DialogDescription>
2384             </DialogHeader>

```

app/lotlight.tsx (continued)

```

2385     <form
2386       onSubmit={(e) => {
2387         e.preventDefault();
2388         void onSave({
2389           type: "correct_item",
2390           id: item.id,
2391           product: form.product,
2392           manufacturer: form.manufacturer,
2393           catalog: form.catalog,
2394           lot: form.lot,
2395           quantity: form.quantity,
2396           location: form.location,
2397         });
2398       }}
2399     >
2400     <div className="form-two">
2401       {(
2402         ["product", "manufacturer", "catalog", "lot", "location"] as const
2403       ).map((key) => (
2404         <label className="field-label" key={key}>
2405           {key[0].toUpperCase() + key.slice(1)}
2406           <input
2407             value={form[key]}
2408             required={key === "product" || key === "location"}
2409             onChange={(e) => setForm({ ...form, [key]: e.target.value })}
2410           />
2411         </label>
2412       ))}
2413     <label className="field-label">
2414       Quantity
2415       <input
2416         type="number"
2417         min={1}
2418         max={1000000}
2419         required
2420         value={form.quantity}
2421         onChange={(e) =>
2422           setForm({ ...form, quantity: Number(e.target.value) })
2423         }
2424       />
2425     </label>
2426   </div>
2427   <DialogFooter>
2428     <button type="button" className="button" onClick={onClose}>
2429       Cancel
2430     </button>
2431     <button type="submit" className="button primary" disabled={busy}>
2432       Save correction <Check size={14} />
2433     </button>
2434   </DialogFooter>
2435 </form>
2436 </DialogContent>
2437 </Dialog>
2438 );
2439 }

```

app/page.tsx

```

1 import Lotlight from "./lotlight";
2 import { getChatGPTUser } from "./chatgpt-auth";
3 export const dynamic = "force-dynamic";
4 export default async function Home() {
5   const user = await getChatGPTUser();
6   return (
7     <Lotlight
8       user={user ? { name: user.displayName, email: user.email } : null}
9     />
10   );
11 }

```

db/index.ts

```
1 import { env } from "cloudflare:workers";
2 import { drizzle } from "drizzle-orm/d1";
3 import * as schema from "./schema";
4
5 export function getDb() {
6   if (!env.DB) {
7     throw new Error(
8       "Cloudflare D1 binding `DB` is unavailable. Set the `d1` field in .openai/hosting.json to `DB` or let
        your control plane inject the real binding values before using the database."
9     );
10  }
11
12  return drizzle(env.DB, { schema });
13 }
```

db/schema.ts

```
1 import { sqliteTable, text, integer } from "drizzle-orm/sqlite-core";
2 export const workspaces = sqliteTable("workspaces", {
3   ownerId: text("owner_id").primaryKey(),
4   state: text("state").notNull(),
5   revision: integer("revision").notNull().default(0),
6   updatedAt: text("updated_at").notNull(),
7 });
```

drizzle/0000_real_hellcat.sql

```
1 CREATE TABLE `workspaces` (  
2   `owner_id` text PRIMARY KEY NOT NULL,  
3   `state` text NOT NULL,  
4   `revision` integer DEFAULT 0 NOT NULL,  
5   `updated_at` text NOT NULL  
6 );
```

lib/lotlight/commands.ts

```

1 import { z } from "zod";
2 import { parseCsv, validateScope, matchItem } from "../engine";
3 import { initialWorkspace } from "../sample";
4 import type { Workspace, ResponseAction } from "../types";
5 const short = z.string().trim().min(1).max(200);
6 const isoDate = z
7   .string()
8   .regex(/^d{4}-d{2}-d{2}$/)
9   .refine(
10     (s) =>
11       !Number.isNaN(Date.parse(s)) &&
12       new Date(s).toISOString().slice(0, 10) === s,
13     "Use a valid calendar date.",
14   );
15 const scopeSchema = z
16   .object({
17     catalog: short,
18     lots: z.array(short).max(100),
19     allLots: z.boolean(),
20     evidence: z.string().trim().min(1).max(5000),
21   })
22   .strict();
23 const noticeInput = z
24   .object({
25     id: z.string().max(100).optional(),
26     title: short,
27     manufacturer: short,
28     date: isoDate,
29     sourceUrl: z
30       .string()
31       .url()
32       .max(1500)
33       .refine(
34         (s) => new URL(s).protocol === "https:",
35         "Use an HTTPS source URL.",
36       ),
37     text: z.string().trim().min(20).max(50000),
38     action: z.string().trim().min(15).max(2000),
39     scope: z.array(scopeSchema).min(1).max(40),
40     training: z.boolean(),
41   })
42   .strict();
43 export const commandSchema = z.discriminatedUnion("type", [
44   z
45     .object({
46       type: z.literal("import_inventory"),
47       csv: z.string().max(500000),
48       synthetic: z.boolean().optional(),
49     })
50     .strict(),
51   z.object({ type: z.literal("save_notice"), notice: noticeInput }).strict(),
52   z
53     .object({
54       type: z.literal("correct_item"),
55       id: short,
56       product: short,
57       manufacturer: z.string().trim().max(200),
58       catalog: z.string().trim().max(200),
59       lot: z.string().trim().max(200),
60       quantity: z.number().int().min(1).max(1000000),
61       location: short,
62     })
63     .strict(),
64   z
65     .object({
66       type: z.literal("approve_notice"),
67       noticeId: short,
68       sourceConfirmed: z.literal(true),
69     })
70     .strict(),
71   z
72     .object({
73       type: z.literal("save_action"),
74       noticeId: short,
75       itemId: short,
76       owner: z.string().trim().min(2).max(100),
77       dueDate: isoDate,
78       status: z.enum(["assigned", "verified", "completed"]),
79       verification: z.string().trim().max(2000),
80       evidence: z.string().trim().max(2000),

```

lib/lotlight/commands.ts (continued)

```

81     handledQuantity: z.number().int().min(0).max(1000000),
82   })
83   .strict(),
84   z.object({ type: z.literal("reset_demo") }).strict(),
85   z.object({ type: z.literal("clear_workspace") }).strict(),
86 });
87 export type Command = z.infer<typeof commandSchema>;
88 export function activeAction(
89   state: Workspace,
90   noticeId: string,
91   itemId: string,
92 ): ResponseAction | undefined {
93   const notice = state.notices.find((n) => n.id === noticeId);
94   const item = state.inventory.find((i) => i.id === itemId);
95   if (!notice || !item) return undefined;
96   return state.actions.find(
97     (a) =>
98       a.noticeId === noticeId &&
99       a.itemId === itemId &&
100       a.noticeVersion === notice?.version &&
101       (a.itemVersion ?? 1) === (item?.version ?? 1),
102   );
103 }
104 export function reduceWorkspace(
105   current: Workspace,
106   untrusted: unknown,
107   actor: string,
108   now = new Date().toISOString(),
109 ): Workspace {
110   const cmd = commandSchema.parse(untrusted);
111   let next = structuredClone(current);
112   next.noticeHistory ??= [];
113   next.inventoryHistory ??= [];
114   next.inventoryProvenance ??= "synthetic";
115   let detail = "";
116   if (current.audit.length >= 2000 && cmd.type !== "clear_workspace")
117     throw new Error(
118       "Workspace audit limit reached. Export your record before starting a fresh workspace.",
119     );
120   if (cmd.type === "reset_demo") {
121     next = initialWorkspace();
122     next.audit = current.audit;
123     next.inventoryVersion = current.inventoryVersion + 1;
124     detail =
125       "Started a new synthetic demonstration. Prior activity remains in the audit log; export detailed case
126     records before resetting.";
127   }
128   if (cmd.type === "clear_workspace") {
129     next = {
130       inventory: [],
131       inventoryProvenance: "user-supplied",
132       noticeHistory: [],
133       inventoryHistory: [],
134       inventoryVersion: current.inventoryVersion + 1,
135       notices: [],
136       actions: [],
137       audit: [],
138       revision: current.revision,
139       savedAt: null,
140     };
141     detail =
142       "Started an empty workspace. Earlier records were removed at the account holder\u2019s request.";
143   }
144   if (cmd.type === "import_inventory") {
145     next.inventoryHistory.push({
146       version: next.inventoryVersion,
147       items: structuredClone(next.inventory),
148       provenance: next.inventoryProvenance,
149     });
150     next.inventory = parseCsv(cmd.csv);
151     next.inventoryProvenance = cmd.synthetic ? "synthetic" : "user-supplied";
152     next.inventoryVersion++;
153     detail = `Imported ${next.inventory.length} inventory lines as revision ${next.inventoryVersion}. Prior
154     response actions are historical and do not close new inventory obligations.`;
155   }
156   if (cmd.type === "correct_item") {
157     const old = next.inventory.find((i) => i.id === cmd.id);
158     if (!old) throw new Error("Inventory line not found.");
159     next.inventoryHistory.push({
160       version: next.inventoryVersion,

```

lib/lotlight/commands.ts (continued)

```

159     items: structuredClone(next.inventory),
160     provenance: next.inventoryProvenance,
161   });
162   next.inventoryVersion++;
163   const { type, ...fields } = cmd;
164   next.inventory = next.inventory.map((i) =>
165     i.id === cmd.id ? { ...fields, version: (old.version ?? 1) + 1 } : i,
166   );
167   detail = `Corrected ${old.product} at ${old.location}. Item revision ${old.version ?? 1} + 1,
inventory revision ${next.inventoryVersion}. This line\u2019s responses require fresh review; other
unchanged lines retain their response state.`;
168 }
169 if (cmd.type === "save_notice") {
170   const error = validateScope(cmd.notice);
171   if (error) throw new Error(error);
172   const existing = cmd.notice.id
173     ? next.notices.find((n) => n.id === cmd.notice.id)
174     : undefined;
175   if (cmd.notice.id && !existing) throw new Error("Notice not found.");
176   if (!existing && next.notices.length >= 20)
177     throw new Error("This workspace supports up to 20 notices.");
178   if (existing) next.noticeHistory.push(structuredClone(existing));
179   const notice = {
180     ...cmd.notice,
181     id: existing?.id ?? crypto.randomUUID(),
182     version: (existing?.version ?? 0) + 1,
183     approvedBy: null,
184     approvedAt: null,
185   };
186   next.notices = existing
187     ? next.notices.map((n) => (n.id === existing.id ? notice : n))
188     : [notice, ...next.notices];
189   detail = `${existing ? "Updated" : "Added"} notice \u201c${notice.title}\u201d, version
${notice.version}. Source approval is required. Earlier version actions remain historical.`;
190 }
191 if (cmd.type === "approve_notice") {
192   const notice = next.notices.find((n) => n.id === cmd.noticeId);
193   if (!notice) throw new Error("Notice not found.");
194   const error = validateScope(notice);
195   if (error) throw new Error(error);
196   notice.approvedBy = actor;
197   notice.approvedAt = now;
198   detail = `Staff confirmed source and paired scope for \u201c${notice.title}\u201d version
${notice.version}${notice.training ? " in a training scenario" : ""}. This is staff attestation, not
independent verification.`;
199 }
200 if (cmd.type === "save_action") {
201   const notice = next.notices.find((n) => n.id === cmd.noticeId);
202   const item = next.inventory.find((i) => i.id === cmd.itemId);
203   if (!notice || !item)
204     throw new Error("Notice or inventory line not found.");
205   if (!notice.approvedAt)
206     throw new Error(
207       "Review and confirm the notice source before assigning a response.",
208     );
209   const match = matchItem(item, notice);
210   if (match.kind === "unlisted")
211     throw new Error(
212       "This line has no listed identifier match in this notice.",
213     );
214   if (cmd.handledQuantity > item.quantity)
215     throw new Error("Handled quantity cannot exceed recorded stock.");
216   if (cmd.status !== "assigned") {
217     if (match.kind !== "exact")
218       throw new Error(
219         "Resolve missing identifiers in the inventory before recording verification or completion.",
220       );
221     if (cmd.verification.length < 12)
222       throw new Error(
223         "Record how you checked the physical label and current manufacturer instructions (at least 12
characters).",
224       );
225   }
226   if (cmd.status === "assigned" && cmd.handledQuantity !== 0)
227     throw new Error(
228       "Verify the physical label before recording handled units.",
229     );
230   if (cmd.handledQuantity > 0 && cmd.evidence.length < 12)
231     throw new Error(
232       "Record response evidence and a supporting reference for handled units (at least 12 characters).",

```

lib/lotlight/commands.ts (continued)

```

233     );
234     if (
235       cmd.status === "completed" &&
236       (cmd.handledQuantity !== item.quantity || cmd.evidence.length < 12)
237     )
238       throw new Error(
239         "Completion requires all recorded units handled and a response evidence reference.",
240       );
241     const existing = activeAction(next, notice.id, item.id);
242     const action: ResponseAction = {
243       id: existing?.id ?? crypto.randomUUID(),
244       noticeId: notice.id,
245       noticeVersion: notice.version,
246       itemId: item.id,
247       inventoryVersion: next.inventoryVersion,
248       itemVersion: item.version ?? 1,
249       itemLabel: `${item.product} \xb7 ${item.catalog} || "no catalog" \xb7 ${item.lot} || "no lot" \xb7
    ${item.location}`,
250       owner: cmd.owner,
251       dueDate: cmd.dueDate,
252       status: cmd.status,
253       verification: cmd.verification,
254       evidence: cmd.evidence,
255       handledQuantity: cmd.handledQuantity,
256       updatedAt: now,
257       updatedBy: actor,
258     };
259     next.actions = existing
260       ? next.actions.map((a) => (a.id === existing.id ? action : a))
261       : [action, ...next.actions];
262     detail = `${cmd.status === "completed" ? "Response recorded as complete" : cmd.status === "verified" ?
    "Physical verification recorded" : "Response assigned"}: ${action.itemLabel}. Owner: ${action.owner}.
    Handled ${cmd.handledQuantity}/${item.quantity}. Verification: ${cmd.verification} || "pending". Evidence:
    ${cmd.evidence} || "pending".`;
263   }
264   next.revision = current.revision + 1;
265   next.savedAt = now;
266   next.audit.push({
267     id: crypto.randomUUID(),
268     at: now,
269     actor,
270     type: cmd.type,
271     detail,
272   });
273   return next;
274 }

```

lib/lotlight/engine.ts

```

1 import type { InventoryItem, Match, Notice, Scope } from "./types";
2 /** Conservative normalization preserves separators and leading zeroes. */
3 export const identifier = (value: string) =>
4   value.normalize("NFKC").trim().toUpperCase();
5 export const company = (value: string) =>
6   value.normalize("NFKC").trim().toLowerCase().replace(/\s+/g, " ");
7 export function matchItem(
8   item: InventoryItem,
9   notice: Notice,
10  similarity?: number,
11 ): Match {
12   const manufacturerMatches =
13     company(item.manufacturer) === company(notice.manufacturer);
14   const productScope = notice.scope.filter(
15     (s) => identifier(s.catalog) === identifier(item.catalog),
16   );
17   const semanticallyRelated = similarity !== undefined && similarity >= 0.3;
18   const words = notice.title
19     .toLowerCase()
20     .split(/\W+/)
21     .filter((w) => w.length > 3);
22   const nameRelated = words.some((w) => item.product.toLowerCase().includes(w));
23   const base = { item, similarity };
24   if (item.manufacturer && !manufacturerMatches && productScope.length)
25     return {
26       ...base,
27       kind: "missing",
28       reason:
29         "Catalog agrees, but manufacturer identity differs. Verify whether this is an alias or a different
30 maker.",
31       evidence: `Inventory: ${item.manufacturer}. Source: ${notice.manufacturer}. Catalog ${item.catalog}.`,
32     };
33   if (item.manufacturer && !manufacturerMatches)
34     return {
35       ...base,
36       kind: "unlisted",
37       reason:
38         "Manufacturer differs. No listed match in this notice; this is not a safety clearance.",
39       evidence: `Inventory manufacturer: ${item.manufacturer}. Notice manufacturer:
40 ${notice.manufacturer}.`,
41     };
42   if (!item.catalog) {
43     if (manufacturerMatches || semanticallyRelated || nameRelated)
44       return {
45         ...base,
46         kind: semanticallyRelated ? "possible" : "missing",
47         reason: semanticallyRelated
48           ? "AI found a similar description, but catalog and lot evidence must be checked."
49           : "Catalog number is missing. Check the physical label before deciding.",
50         evidence:
51           "No catalog identifier available. Product-name similarity cannot prove recall scope.",
52       };
53     return {
54       ...base,
55       kind: "missing",
56       reason:
57         "Product identity is incomplete. Obtain manufacturer and catalog information.",
58       evidence: "Missing catalog and unverified manufacturer.",
59     };
60   }
61   if (!productScope.length)
62     return {
63       ...base,
64       kind: "unlisted",
65       reason:
66         "Catalog is not listed in this reviewed scope. This does not establish safety or cover other
67 notices.",
68       evidence: `Inventory catalog ${item.catalog}; listed: ${notice.scope.map((s) => s.catalog).join(",
69 ")}.`,
70     };
71   if (!item.manufacturer)
72     return {
73       ...base,
74       kind: "missing",
75       reason:
76         "Catalog matches but manufacturer is missing. Verify the physical label.",
77       evidence: productScope[0].evidence,
78     };
79   if (!item.lot && !productScope.some((s) => s.allLots))
80     return {

```

lib/lotlight/engine.ts (continued)

```

77     ...base,
78     kind: "missing",
79     reason:
80       "Catalog matches, but the lot is missing. The item cannot be cleared.",
81     evidence: productScope.map((s) => s.evidence).join("\n"),
82   };
83   const scope = productScope.find(
84     (s) =>
85       s.allLots || s.lots.some((l) => identifier(l) === identifier(item.lot)),
86   );
87   if (scope)
88     return {
89       ...base,
90       kind: "exact",
91       reason:
92         "Manufacturer, catalog, and listed lot scope agree. Verify the physical label and current notice
93       before acting.",
94       evidence: scope.evidence,
95     };
96   return {
97     ...base,
98     kind: "unlisted",
99     reason:
100       "Catalog matches, but this lot is not listed in the reviewed scope. No safety clearance is implied.",
101     evidence: productScope.map((s) => s.evidence).join("\n"),
102   };
103 }
104 export function reconcile(
105   inventory: InventoryItem[],
106   notice: Notice,
107   scores: Record<string, number> = {},
108 ) {
109   const order = { exact: 0, missing: 1, possible: 2, unlisted: 3 };
110   return inventory
111     .map((item) => matchItem(item, notice, scores[item.id]))
112     .sort(
113       (a, b) =>
114         order[a.kind] - order[b.kind] ||
115         (b.similarity ?? 0) - (a.similarity ?? 0),
116     );
117 }
118 /** Extract only explicitly paired labeled lines. Never cross-join catalogs and lots. */
119 export function extractScope(text: string): Scope[] {
120   const result: Scope[] = [];
121   for (const line of text.split(/\r?\n/)) {
122     const m = line.match(
123       /\^s*(?:Catalog|REF|Model)\s*:\s*([A-Za-z0-9._/-]+\s*;\s*Lots?\s*:\s*(.+)\s*$/i,
124     );
125     if (!m) continue;
126     const raw = m[2].trim();
127     if (
128       /(?:\bto\b|\bthrough\b|\bexcept\b|\bexcluding\b|\bbetween\b|\bonly\b|\bif\b|\\*\b|\\.)/i.test(
129         raw,
130       )
131     ) continue;
132     const allLots = /^all lots$/i.test(raw);
133     const lots = allLots
134       ? []
135       : raw
136         .split(/[,;]/)
137         .map((l) => l.trim())
138         .filter(Boolean);
139     if (!allLots && lots.some((l) => !/^[A-Za-z0-9._/-]+$/.test(l))) continue;
140     result.push({ catalog: m[1], lots, allLots, evidence: line.trim() });
141   }
142   return result;
143 }
144 export function validateScope(
145   notice: Pick<Notice, "text" | "scope" | "manufacturer">,
146 ): string | null {
147   if (!notice.scope.length)
148     return "Add at least one explicit catalog and lot pair. Ranges, exclusions, and serial-number scope need
149     specialist review.";
150   if (!notice.text.toLowerCase().includes(notice.manufacturer.toLowerCase()))
151     return "Manufacturer must appear in the source text.";
152   for (const s of notice.scope) {
153     if (!s.evidence.trim() || !notice.text.includes(s.evidence))
154       return "Every scope row needs a verbatim supporting excerpt from the source text.";
155     const lines = s.evidence.split(/\r?\n/).filter((l) => l.trim());

```

lib/lotlight/engine.ts (continued)

```

155     const parsed = extractScope(s.evidence);
156     if (lines.length !== 1 || parsed.length !== 1)
157         return "Use one unambiguous labeled source line per scope row: Catalog: ABC; Lots: 123, 456. For
tables, append a staff-checked labeled transcription to the preserved source first.";
158     const pair = parsed[0];
159     if (
160         identifier(pair.catalog) !== identifier(s.catalog) ||
161         pair.allLots !== s.allLots
162     )
163         return "Declared scope must exactly match its single labeled source pair.";
164     const expected = pair.lots.map(identifier).sort();
165     const actual = s.lots.map(identifier).sort();
166     if (JSON.stringify(expected) !== JSON.stringify(actual))
167         return "Declared lots must match the complete lot list in the supporting source pair exactly.";
168 }
169 return null;
170 }
171 export function parseCsv(text: string): InventoryItem[] {
172     if (text.length > 500_000) throw new Error("CSV exceeds the 500 KB limit.");
173     const rows: string[][] = [];
174     let row: string[] = [],
175         field = "",
176         quoted = false;
177     const source = text.replace(/\uFEFF/, "");
178     for (let i = 0; i < source.length; i++) {
179         const c = source[i];
180         if (c === '"') {
181             if (quoted && source[i + 1] === '"') {
182                 field += '"';
183                 i++;
184             } else if (quoted || field.length === 0) quoted = !quoted;
185             else throw new Error("Invalid quote in CSV.");
186         } else if (c === "," && !quoted) {
187             row.push(field);
188             field = "";
189         } else if ((c === "\n" || c === "\r") && !quoted) {
190             if (c === "\r" && source[i + 1] === "\n") i++;
191             row.push(field);
192             if (row.some((v) => v.trim())) rows.push(row);
193             row = [];
194             field = "";
195         } else field += c;
196     }
197     if (quoted) throw new Error("CSV has an unclosed quoted field.");
198     row.push(field);
199     if (row.some((v) => v.trim())) rows.push(row);
200     if (rows.length < 2)
201         throw new Error("CSV needs a header and at least one inventory line.");
202     if (rows.length > 501)
203         throw new Error("Import up to 500 inventory lines per workspace.");
204     const headers = rows.shift().map((h) => h.trim().toLowerCase());
205     const required = [
206         "product",
207         "manufacturer",
208         "catalog",
209         "lot",
210         "quantity",
211         "location",
212     ];
213     if (new Set(headers).size !== headers.length)
214         throw new Error("CSV contains duplicate column names.");
215     for (const name of required)
216         if (!headers.includes(name))
217             throw new Error(
218                 `Missing column: ${name}. Use the downloadable template.`
219             );
220     const seen = new Set<string>();
221     return rows.map((values, i) => {
222         if (values.length !== headers.length)
223             throw new Error(`Row ${i + 2}: column count does not match the header.`);
224         const get = (key: string) => (values[headers.indexOf(key)] ?? "").trim();
225         const quantity = Number(get("quantity"));
226         if (
227             !get("quantity") ||
228             !Number.isInteger(quantity) ||
229             quantity < 1 ||
230             quantity > 1_000_000
231         )
232             throw new Error(
233                 `Row ${i + 2}: quantity must be a whole number from 1 to 1,000,000.`
234             );
235     });

```

lib/lotlight/engine.ts (continued)

```

234     );
235     if (!get("product") || !get("location"))
236       throw new Error(`Row ${i + 2}: product and location are required.`);
237     if (values.some((v) => v.length > 500))
238       throw new Error(`Row ${i + 2}: a field exceeds 500 characters.`);
239     const key = [
240       get("product"),
241       get("manufacturer"),
242       get("catalog"),
243       get("lot"),
244       get("location"),
245     ]
246       .map(identifier)
247       .join("|");
248     if (seen.has(key))
249       throw new Error(
250         `Row ${i + 2}: duplicate inventory line. Combine quantities before importing.`
251       );
252     seen.add(key);
253     return {
254       id: crypto.randomUUID(),
255       product: get("product"),
256       manufacturer: get("manufacturer"),
257       catalog: get("catalog"),
258       lot: get("lot"),
259       quantity,
260       location: get("location"),
261     };
262   });
263 }
264 export function csvCell(value: unknown): string {
265   let s = String(value ?? "");
266   if (/^[\\s]*[=+@-]/.test(s)) s = "'" + s;
267   return "'" + s.replace(/"/g, '""') + "'";
268 }
269 export function exportCsv(rows: unknown[][]): string {
270   return "\uFEFF" + rows.map((r) => r.map(csvCell).join(",")).join("\r\n");
271 }

```

lib/lotlight/files.ts

```

1 import { exportCsv, reconcile } from "./engine";
2 import { activeAction } from "./commands";
3 import type { Workspace } from "./types";
4 export function download(name: string, content: string, type = "text/plain") {
5   const url = URL.createObjectURL(new Blob([content], { type }));
6   const a = document.createElement("a");
7   a.href = url;
8   a.download = name;
9   document.body.appendChild(a);
10  a.click();
11  a.remove();
12  setTimeout(() => URL.revokeObjectURL(url), 1000);
13 }
14 export async function exportAudit(state: Workspace, saved = false) {
15   const payload = {
16     product: "Lotlight",
17     formatVersion: 1,
18     storageMode: saved ? "account workspace" : "unsaved in-memory demo",
19     exportedAt: new Date().toISOString(),
20     scope:
21       "Staff-recorded decision-support record. Not independent verification, current recall status, or
safety certification.",
22     inventoryProvenance: state.inventoryProvenance,
23     sourceProvenance: state.notices.some((n) => n.training)
      ? "Contains historical training extracts."
      : "User-supplied sources; not independently verified.",
24     workspace: state,
25   };
26   const text = JSON.stringify(payload);
27   const hash = Array.from(
28     new Uint8Array(
29       await crypto.subtle.digest("SHA-256", new TextEncoder().encode(text)),
30     ),
31   )
32     .map((b) => b.toString(16).padStart(2, "0"))
33     .join("");
34   download(
35     "lotlight-audit-record.json",
36     JSON.stringify(
37       {
38         payload,
39         integrity: {
40           algorithm: "SHA-256",
41           digest: hash,
42           note: "Digest of JSON.stringify(payload). Detects accidental changes; not a digital signature or
tamper-proof guarantee.",
43         },
44         null,
45         2,
46       },
47       "application/json",
48     );
49   }
50 }
51 export function exportMatches(state: Workspace, saved = false) {
52   const rows: unknown[][] = [
53     [
54       "storage_mode",
55       "inventory_provenance",
56       "source_provenance",
57       "notice_date",
58       "exported_at",
59       "notice",
60       "notice_version",
61       "source_url",
62       "source_approved_by",
63       "inventory_version",
64       "product",
65       "manufacturer",
66       "catalog",
67       "lot",
68       "quantity",
69       "location",
70       "match",
71       "rationale",
72       "evidence",
73       "owner",
74       "status",
75       "handled_quantity",
76       "response_evidence",

```

lib/lotlight/files.ts (continued)

```

79     ],
80   ];
81   for (const notice of state.notices)
82     for (const match of reconcile(state.inventory, notice)) {
83       const a = activeAction(state, notice.id, match.item.id);
84       rows.push([
85         saved ? "account workspace" : "unsaved in-memory demo",
86         state.inventoryProvenance,
87         notice.training
88           ? "historical training extract"
89           : "user-supplied source",
90         notice.date,
91         new Date().toISOString(),
92         notice.title,
93         notice.version,
94         notice.sourceUrl,
95         notice.approvedBy ?? "unreviewed",
96         state.inventoryVersion,
97         match.item.product,
98         match.item.manufacturer,
99         match.item.catalog,
100        match.item.lot,
101        match.item.quantity,
102        match.item.location,
103        match.kind,
104        match.reason,
105        match.evidence,
106        a?.owner,
107        a?.status ?? "open",
108        a?.handledQuantity ?? 0,
109        a?.evidence,
110      ]);
111    }
112    download(
113      "lotlight-inventory-review.csv",
114      exportCsv(rows),
115      "text/csv;charset=utf-8",
116    );
117  }
118  export async function readNoticeFile(file: File): Promise<string> {
119    if (file.size > 5_000_000) throw new Error("Use a notice smaller than 5 MB.");
120    if (file.name.toLowerCase().endsWith(".txt")) return file.text();
121    if (!file.name.toLowerCase().endsWith(".pdf"))
122      throw new Error("Upload a text-based PDF or a .txt notice.");
123    const pdfjs = await import("pdfjs-dist");
124    const worker = await import("pdfjs-dist/build/pdf.worker.min.mjs?url");
125    pdfjs.GlobalWorkerOptions.workerSrc = worker.default;
126    const loading = pdfjs.getDocument({
127      data: await file.arrayBuffer(),
128      isEvalSupported: false,
129    });
130    const pdf = await loading.promise;
131    try {
132      if (pdf.numPages > 20)
133        throw new Error("Use a notice with 20 pages or fewer.");
134      let text = "";
135      for (let i = 1; i <= pdf.numPages; i++) {
136        const page = await pdf.getPage(i);
137        const content = await page.getTextContent();
138        text +=
139          content.items
140            .map((item) =>
141              "str" in item
142                ? item.str + ("hasEOL" in item && item.hasEOL ? "\n" : " ")
143                : "",
144            )
145            .join("") + "\n";
146        if (text.length > 50000)
147          throw new Error(
148            "Extracted text exceeds 50,000 characters. Upload the relevant notice pages.",
149          );
150      }
151      if (text.trim().length < 20)
152        throw new Error(
153          "This PDF appears to be scanned. OCR is not included; paste a verified transcription instead.",
154        );
155      return text;
156    } finally {
157      await pdf.destroy();
158    }

```

lib/lotlight/files.ts (continued)

159 }

lib/lotlight/sample.ts

```

1 import type { Workspace, Notice } from "../types";
2 export const BAXTER_URL =
3   "https://www.medline.com/media/assets/pdf/vendor-list/FA-2025-039-Customer-Letter-Final-Combined.pdf";
4 export const SAMPLE_NOTICE_TEXT = `TRAINING EXTRACT \u2014 paraphrased from Baxter manufacturer notice
   FA-2025-039, dated 2025-08-29. This is a historical demonstration, not a current safety notice. Consult the
   full original notice for every affected product and instruction. Only two catalog/lot pairs are included
   here.
5 Manufacturer: Baxter
6 Product: CLEARLINK IV extension and solution sets
7 Catalog: 2C8632; Lots: R25C31031
8 Catalog: 2R8858; Lots: R25A13024
9 Reason (paraphrase): A potential leak can compromise delivery and introduce risks during use.
10 Manufacturer wording: \u201cImmediately locate, isolate, and cease all use of the affected product.\u201d
11 Response (paraphrase): Identify stock, follow the supplier-specific return or replacement route, and
   acknowledge the notice. Direct-purchase and distributor-purchase response routes differ.
12 Source: ${BAXTER_URL}`;
13 export const SAMPLE_CSV = `product,manufacturer,catalog,lot,quantity,location
14 CLEARLINK extension set,Baxter,2C8632,R25C31031,24,Infusion room
15 CLEARLINK solution set,Baxter,2R8858,R25A13024,12,Main stockroom
16 CLEARLINK extension set,Baxter,2C8632,,8,Procedure room
17 IV line extender,Baxter,,,6,Mobile cart
18 CLEARLINK extension set,Baxter,2C8632,R24Z99999,18,Main stockroom
19 CLEARLINK solution set,Baxter,2R8858,R24Z88888,10,Infusion room
20 Sterile gauze,Demo Supply Co,GAUZE-4,G202601,40,Procedure room
21 Examination gloves,Demo Supply Co,GLV-M,G202602,100,Main stockroom`;
22 export const sampleNotice: Notice = {
23   id: "baxter-demo",
24   title: "CLEARLINK IV sets",
25   manufacturer: "Baxter",
26   date: "2025-08-29",
27   sourceUrl: BAXTER_URL,
28   text: SAMPLE_NOTICE_TEXT,
29   action:
30     "Identify affected stock and follow the manufacturer\u2019s supplier-specific isolation, return or
   replacement, and acknowledgement instructions. Verify the current full notice first.",
31   scope: [
32     {
33       catalog: "2C8632",
34       lots: ["R25C31031"],
35       allLots: false,
36       evidence: "Catalog: 2C8632; Lots: R25C31031",
37     },
38     {
39       catalog: "2R8858",
40       lots: ["R25A13024"],
41       allLots: false,
42       evidence: "Catalog: 2R8858; Lots: R25A13024",
43     },
44   ],
45   version: 1,
46   training: true,
47   approvedBy: null,
48   approvedAt: null,
49 };
50 export function initialWorkspace(): Workspace {
51   const lines = SAMPLE_CSV.split("\n").slice(1);
52   return {
53     inventoryProvenance: "synthetic",
54     noticeHistory: [],
55     inventoryHistory: [],
56     inventory: lines.map((line, i) => {
57       const [product, manufacturer, catalog, lot, quantity, location] =
58         line.split(",");
59       return {
60         id: `sample-${i + 1}`,
61         product,
62         manufacturer,
63         catalog,
64         lot,
65         quantity: Number(quantity),
66         location,
67       };
68     }),
69     inventoryVersion: 1,
70     notices: [structuredClone(sampleNotice)],
71     actions: [],
72     audit: [
73       {
74         id: "initial",
75         at: "2026-09-16T00:00:00.000Z",

```

lib/lotlight/sample.ts (continued)

```
76         actor: "Lotlight demo",
77         type: "workspace.created",
78         detail:
79           "Historical Baxter training extract loaded with synthetic inventory. No current recall status is
            implied.",
80       },
81     ],
82     revision: 0,
83     savedAt: null,
84   };
85 }
```

lib/lotlight/semantic-worker.ts

```

1  /// <reference lib="webworker" />
2  import {
3    env,
4    pipeline,
5    type FeatureExtractionPipeline,
6  } from "@huggingface/transformers";
7  env.allowLocalModels = false;
8  if (env.backends.onnx.wasm) {
9    env.backends.onnx.wasm.numThreads = 1;
10   env.backends.onnx.wasm.wasmPaths = new URL("/ai/", self.location.origin).href;
11 }
12 type Progress = { status: string; progress?: number };
13 const createExtractor = pipeline as unknown as (
14   task: string,
15   model: string,
16   options: Record<string, unknown>,
17 ) => Promise<FeatureExtractionPipeline>;
18 let extractor: Promise<FeatureExtractionPipeline> | undefined;
19 self.onmessage = async (
20   event: MessageEvent<{
21     query: string;
22     items: { id: string; product: string }[];
23   }>,
24 ) => {
25   try {
26     extractor ??= createExtractor(
27       "feature-extraction",
28       "Xenova/all-MiniLM-L6-v2",
29       {
30         revision: "751bff37182d3f1213fa05d7196b954e230abad9",
31         dtype: "q8",
32         device: "wasm",
33         progress_callback: (p: Progress) => {
34           if ("status" in p)
35             self.postMessage({
36               type: "progress",
37               message:
38                 p.status === "progress" && "progress" in p
39                 ? `Loading local AI \xb7 ${Math.round(p.progress as number)}%`
40                 : "Preparing local semantic model\u2026",
41             });
42         },
43       },
44     );
45     const model = await extractor;
46     const { query, items } = event.data;
47     self.postMessage({
48       type: "progress",
49       message: "Comparing product descriptions on your device\u2026",
50     });
51     const q = await model(query.slice(0, 500), {
52       pooling: "mean",
53       normalize: true,
54     });
55     const vector = Array.from(q.data as Float32Array);
56     const scores: Record<string, number> = {};
57     for (let i = 0; i < items.length; i += 16) {
58       const batch = items.slice(i, i + 16);
59       const embeddings = await model(
60         batch.map((item) => item.product.slice(0, 500)),
61         { pooling: "mean", normalize: true },
62       );
63       const vectors = embeddings.tolist() as number[][];
64       batch.forEach((item, j) => {
65         scores[item.id] = vectors[j].reduce(
66           (sum, value, k) => sum + value * vector[k],
67           0,
68         );
69       });
70       self.postMessage({
71         type: "progress",
72         message: `Compared ${Math.min(i + 16, items.length)} of ${items.length} inventory descriptions`,
73       });
74     }
75     self.postMessage({ type: "result", scores });
76   } catch (error) {
77     extractor = undefined;
78     self.postMessage({
79       type: "error",
80       message:

```

lib/lotlight/semantic-worker.ts (continued)

```
81         error instanceof Error ? error.message : "Semantic model unavailable",
82     });
83 }
84 };
```

lib/lotlight/storage.ts

```
1 import { env } from "cloudflare:workers";
2 import { initialWorkspace } from "./sample";
3 import type { Workspace } from "./types";
4 export function database() {
5   if (!env.DB)
6     throw new Error(
7       "Persistent storage is temporarily unavailable. Please retry.",
8     );
9   return env.DB;
10 }
11 export async function readWorkspace(ownerId: string): Promise<Workspace> {
12   const db = database();
13   const row = await db
14     .prepare("SELECT state FROM workspaces WHERE owner_id = ?")
15     .bind(ownerId)
16     .first<{ state: string }>();
17   if (row) return JSON.parse(row.state) as Workspace;
18   const state = initialWorkspace();
19   await db
20     .prepare(
21       "INSERT OR IGNORE INTO workspaces (owner_id, state, revision, updated_at) VALUES (?, ?, ?, ?)",
22     )
23     .bind(ownerId, JSON.stringify(state), 0, new Date().toISOString())
24     .run();
25   const saved = await db
26     .prepare("SELECT state FROM workspaces WHERE owner_id = ?")
27     .bind(ownerId)
28     .first<{ state: string }>();
29   if (!saved)
30     throw new Error("Could not initialize your workspace. Please retry.");
31   return JSON.parse(saved.state);
32 }
33 export async function writeWorkspace(
34   ownerId: string,
35   expectedRevision: number,
36   state: Workspace,
37 ) {
38   const result = await database()
39     .prepare(
40       "UPDATE workspaces SET state = ?, revision = ?, updated_at = ? WHERE owner_id = ? AND revision = ?",
41     )
42     .bind(
43       JSON.stringify(state),
44       state.revision,
45       state.savedAt,
46       ownerId,
47       expectedRevision,
48     )
49     .run();
50   return result.meta.changes === 1;
51 }
```

lib/lotlight/types.ts

```
1 export type InventoryItem = {
2   id: string;
3   version?: number;
4   product: string;
5   manufacturer: string;
6   catalog: string;
7   lot: string;
8   quantity: number;
9   location: string;
10 };
11 export type Scope = {
12   catalog: string;
13   lots: string[];
14   allLots: boolean;
15   evidence: string;
16 };
17 export type Notice = {
18   id: string;
19   title: string;
20   manufacturer: string;
21   date: string;
22   sourceUrl: string;
23   text: string;
24   action: string;
25   scope: Scope[];
26   version: number;
27   training: boolean;
28   approvedBy: string | null;
29   approvedAt: string | null;
30 };
31 export type MatchKind = "exact" | "missing" | "possible" | "unlisted";
32 export type Match = {
33   item: InventoryItem;
34   kind: MatchKind;
35   reason: string;
36   evidence: string;
37   similarity?: number;
38 };
39 export type ResponseAction = {
40   id: string;
41   noticeId: string;
42   noticeVersion: number;
43   itemId: string;
44   inventoryVersion: number;
45   itemVersion: number;
46   itemLabel: string;
47   owner: string;
48   dueDate: string;
49   status: "assigned" | "verified" | "completed";
50   verification: string;
51   evidence: string;
52   handledQuantity: number;
53   updatedAt: string;
54   updatedBy: string;
55 };
56 export type AuditEvent = {
57   id: string;
58   at: string;
59   actor: string;
60   type: string;
61   detail: string;
62 };
63 export type Workspace = {
64   inventory: InventoryItem[];
65   inventoryProvenance: "synthetic" | "user-supplied";
66   noticeHistory: Notice[];
67   inventoryHistory: {
68     version: number;
69     items: InventoryItem[];
70     provenance: string;
71   }[];
72   inventoryVersion: number;
73   notices: Notice[];
74   actions: ResponseAction[];
75   audit: AuditEvent[];
76   revision: number;
77   savedAt: string | null;
78 };
79 export type UserInfo = { name: string; email: string } | null;
```

package.json

```

1 {
2   "name": "lotlight",
3   "version": "1.0.0",
4   "private": true,
5   "engines": {
6     "node": ">=22.13.0"
7   },
8   "scripts": {
9     "install:ci": "node scripts/install-ci.mjs",
10    "dev": "node scripts/run-framework.mjs dev",
11    "build": "node scripts/run-framework.mjs build",
12    "start": "node --import ./scripts/sites-env.mjs ./node_modules/wrangler/bin/wrangler.js dev --config
dist/server/wrangler.json --local --persist-to .wrangler/state --ip 127.0.0.1 --inspector-port 0",
13    "lint": "eslint . --ignore-pattern dist --ignore-pattern .next",
14    "db:generate": "drizzle-kit generate",
15    "test": "tsx --test tests/engine.test.ts",
16    "typecheck": "tsc --noEmit",
17    "test:e2e": "playwright test",
18    "predev": "node scripts/build-ai.mjs",
19    "prebuild": "node scripts/build-ai.mjs"
20  },
21  "dependencies": {
22    "@base-ui/react": "^1.7.0",
23    "@hookform/resolvers": "^5.7.1",
24    "@huggingface/transformers": "3.8.1",
25    "@shadcn/react": "^0.3.0",
26    "class-variance-authority": "0.7.1",
27    "clsx": "2.1.1",
28    "cmdk": "^1.1.1",
29    "date-fns": "^4.4.0",
30    "drizzle-orm": "0.45.2",
31    "embla-carousel-react": "^8.6.0",
32    "input-otp": "^1.4.2",
33    "lucide-react": "^1.31.0",
34    "next": "16.3.4",
35    "next-themes": "^0.4.6",
36    "pdfjs-dist": "5.4.296",
37    "radix-ui": "^1.6.7",
38    "react": "19.2.6",
39    "react-day-picker": "^10.0.1",
40    "react-dom": "19.2.6",
41    "react-hook-form": "^7.85.0",
42    "react-resizable-panels": "^4.12.2",
43    "recharts": "^3.8.0",
44    "sonner": "^2.0.8",
45    "tailwind-merge": "3.6.0",
46    "vaul": "^1.1.2",
47    "zod": "^3.25.76"
48  },
49  "devDependencies": {
50    "@axe-core/playwright": "4.11.0",
51    "@cloudflare/vite-plugin": "1.37.1",
52    "@cloudflare/workers-types": "4.20260515.1",
53    "@playwright/test": "1.58.2",
54    "@tailwindcss/postcss": "4.2.1",
55    "@types/node": "22.19.19",
56    "@types/react": "19.2.14",
57    "@types/react-dom": "19.2.3",
58    "@vitejs/plugin-react": "6.0.2",
59    "@vitejs/plugin-rsc": "0.5.26",
60    "drizzle-kit": "0.31.10",
61    "esbuild": "0.28.0",
62    "eslint": "9.39.4",
63    "eslint-config-next": "16.3.4",
64    "prettier": "3.6.2",
65    "react-server-dom-webpack": "19.2.6",
66    "tailwindcss": "4.2.1",
67    "tsx": "4.21.0",
68    "tw-animate-css": "^1.4.0",
69    "typescript": "5.9.3",
70    "vinext": "1.0.0-beta.5",
71    "vite": "8.0.13",
72    "wrangler": "4.92.0"
73  },
74  "type": "module",
75  "overrides": {
76    "miniflare": {
77      "sharp": "0.35.4"
78    }
79  }

```

package.json (continued)

80 }

playwright.config.ts

```
1 import { defineConfig } from "@playwright/test";
2 export default defineConfig({
3   testDir: "tests/e2e",
4   timeout: 45000,
5   fullyParallel: false,
6   workers: 1,
7   retries: 0,
8   reporter: [
9     ["list"],
10    ["json", { outputFile: "test-results/browser-results.json" }],
11  ],
12  use: {
13    baseURL: process.env.LOTLIGHT_TEST_URL ?? "http://localhost:5174",
14    headless: true,
15    viewport: { width: 1440, height: 1000 },
16    screenshot: "only-on-failure",
17    trace: "retain-on-failure",
18  },
19 });
```

scripts/build-ai.mjs

```
1 import { build } from "esbuild";
2 import { mkdir, copyFile } from "node:fs/promises";
3 await mkdir("public/ai", { recursive: true });
4 await build({
5   entryPoints: ["lib/lotlight/semantic-worker.ts"],
6   outfile: "public/ai/semantic-worker.js",
7   bundle: true,
8   platform: "browser",
9   format: "esm",
10  target: "es2022",
11  minify: true,
12  legalComments: "eof",
13 });
14 for (const file of [
15   "ort-wasm-simd-threaded.jsep.mjs",
16   "ort-wasm-simd-threaded.jsep.wasm",
17 ])
18   await copyFile(
19     "node_modules/onnxruntime-web/dist/" + file,
20     "public/ai/" + file,
21   );
22 console.log("Built isolated browser AI worker and local WASM runtime.");
```

scripts/check-accessibility.mjs

```
1 import { chromium } from "playwright";
2 import AxeBuilder from "@axe-core/playwright";
3 const b = await chromium.launch({ headless: true });
4 const context = await b.newContext({ viewport: { width: 1440, height: 1000 } });
5 const p = await context.newPage();
6 await p.goto(process.env.LOTLIGHT_TEST_URL ?? "http://localhost:5174");
7 await p.locator('main[data-ready="true"]').waitFor();
8 const result = await new AxeBuilder({ page: p })
9   .withTags(["wcag2a", "wcag2aa"])
10  .analyze();
11 console.log(
12   JSON.stringify(
13     result.violations.map((v) => ({
14       id: v.id,
15       impact: v.impact,
16       nodes: v.nodes
17         .map((n) => ({ target: n.target, summary: n.failureSummary }))
18         .slice(0, 25),
19     })),
20     null,
21     2,
22   ),
23 );
24 await b.close();
```

scripts/check-api-isolation.mjs

```

1 import assert from 'node:assert/strict';
2 import fs from 'node:fs/promises';
3 const url=process.env.LOTLIGHT_BUILT_URL??'http://127.0.0.1:5185';
4 const suffix=crypto.randomUUID();const a='lotlight-test-a-'+suffix,b='lotlight-test-b-'+suffix;
5 const headers=user=>({'oai-authenticated-user-id':user,'oai-authenticated-user-
  email':user+'@example.invalid','Content-Type':'application/json',Origin:url});
6 const get=async u=>(await fetch(url+'/api/workspace',{headers:headers(u)})).json();
7 const post=async(u,s,command)=>fetch(url+'/api/workspace',{method:'POST',headers:headers(u),body:JSON.string
  ify({revision:s.revision,command}}));
8 assert.equal((await fetch(url+'/api/workspace')).status,401);
9 let sa=await get(a),sb=await get(b);assert.equal(sa.revision,0);assert.equal(sb.revision,0);
10 const source={title:'Isolation test',manufacturer:'Test
  Supplies',date:'2026-09-16',sourceUrl:'https://example.org/notice',text:'Test Supplies\nCatalog: TEST; Lots:
  LOT1',scope:[{catalog:'TEST',lots:['LOT1'],allLots:false,evidence:'Catalog: TEST; Lots:
  LOT1'}],action:'Synthetic source response used only for database isolation testing.',training:true};
11 assert.equal((await post(a,sa,{type:'save_notice',notice:source})).status,200);sa=await get(a);const
  created=sa.notices.find(n=>n.title==='Isolation test');assert.ok(created);sb=await
  get(b);assert.equal(sb.notices.length,1);assert.equal(sb.revision,0);
12 assert.equal((await
  post(b,sb,{type:'approve_notice',noticeId:created.id,sourceConfirmed:true})).status,400);
13 assert.equal((await post(a,{revision:0},{type:'reset_demo'})).status,409);
14 assert.equal((await fetch(url+'/api/workspace',{method:'POST',headers:{...headers(a),Origin:'https://evil.in
  valid'},body:JSON.stringify({revision:sa.revision,command:{type:'reset_demo'}}})).status,403);
15 assert.equal((await
  post(a,sa,{type:'approve_notice',noticeId:created.id,sourceConfirmed:true,approvedBy:'Forged
  actor'})).status,400);
16 await post(a,sa,{type:'clear_workspace'});await post(b,sb,{type:'clear_workspace'});
17 const result={date:new Date().toISOString(),checks:7,status:'passed',environment:'Built Worker on loopback
  with test-only trusted gateway headers',checksRun:['anonymous denied','separate account
  initialization','account A mutation invisible to B','cross-account notice mutation denied','stale revision
  denied','foreign origin denied','forged actor denied'],limitation:'Checks application tenant isolation. Does
  not validate the external Sites header-stripping boundary or full OAuth flow.'};await
  fs.writeFile('docs/api-isolation-results.json',JSON.stringify(result,null,2)+'\n');console.log(result);

```

scripts/create-submission-pdfs.py

```

1 from reportlab.pdfgen import canvas
2 from reportlab.lib.colors import HexColor, Color, white
3 from reportlab.lib.pagesizes import A4
4 from reportlab.platypus import Paragraph
5 from reportlab.lib.styles import ParagraphStyle
6 from pathlib import Path
7 from xml.sax.saxutils import escape
8 import textwrap
9 ROOT=Path(__file__).resolve().parents[1]
10 OUT=ROOT/'submission';OUT.mkdir(exist_ok=True)
11 W,H=A4
12 ink=HexColor('#263329');muted=HexColor('#65745b');lime=HexColor('#ddec8b');paper=HexColor('#f8f9f5');orange=
HexColor('#b55c36')
13 def para(c,text,x,y,w,size=10,leading=15,color=ink,bold=False):
14     s=ParagraphStyle('p',fontName='Helvetica-Bold' if bold else
'Helvetica',fontSize=size,leading=leading,textColor=color)
15     p=Paragraph(text,s);_,h=p.wrap(w,H);p.drawOn(c,x,y-h);return y-h
16 c=canvas.Canvas(str(OUT/'Lotlight-One-Page.pdf'),pagesize=A4);c.setTitle('Lotlight | One-page project
description');c.setAuthor('Shivam Gupta')
17 c.setFill(paper);c.rect(0,0,W,H,fill=1,stroke=0)
18 c.setFill(ink);c.setFont('Helvetica-Bold',9);c.drawString(42,H-42,'DSH HACKS V2 / AI x
HEALTHCARE');c.setFont('Helvetica',9);c.drawRightString(W-42,H-42,'Shivam Gupta')
19 c.setFont('Helvetica-Bold',45);c.drawString(40,H-105,'lotlight');c.setFill(lime);c.roundRect(W-117,H-
109,75,45,10,fill=1,stroke=0);c.setFill(ink);c.setFont('Helvetica-
Bold',9);c.drawCentredString(W-79.5,H-90,'NOTICE');c.drawCentredString(W-79.5,H-102,'TO RESPONSE')
20 y=para(c,'A notice is only the beginning.',42,H-127,W-84,23,28)
21 y=para(c,'Lotlight helps independent clinics reconcile a medical recall notice with spreadsheet inventory,
resolve uncertain identifiers, and document a response someone owns.',42,y-14,W-84,11,16)
22 y-=20
23 c.setStrokeColor(HexColor('#dce2d4'));c.line(42,y,W-42,y)
24 y-=21
25 left=42;right=310;col=243
26 para(c,'THE PROBLEM',left,y,col,8.5,12,muted,True);para(c,'THE WORKING
PRODUCT',right,y,col,8.5,12,muted,True)
27 y1=para(c,'A recall letter is not a completed stock check. Product names vary, lot fields are missing, and
responsibility can get lost between a notice and a shelf.',left,y-22,col,10,15)
28 y2=para(c,'Import a notice and CSV. Compare each catalog/lot pair with source evidence. Confirm the source,
assign an owner, record quantities handled, and export the audit.',right,y-22,col,10,15)
29 y=min(y1,y2)-22
30 c.setFill(HexColor('#edf2df'));c.roundRect(42,y-76,W-84,76,8,fill=1,stroke=0)
31 para(c,'A REAL NOTICE. A CLEARLY SYNTHETIC CLINIC.',56,y-13,W-112,8,11,muted,True)
32 para(c,'The demo uses two pairs from Baxter notice FA-2025-039 (29 Aug 2025). Eight fictional stock lines
contain two exact matches, two incomplete records, and four lines outside the entered scope. No patient data
or current recall status is implied.',56,y-32,W-112,9.5,14)
33 y-=101
34 para(c,'AI WITH A BOUNDARY',left,y,col,8.5,12,muted,True);para(c,'ENGINEERING THAT
MATTERS',right,y,col,8.5,12,muted,True)
35 y1=para(c,'A pinned MiniLM model compares descriptions locally in a browser worker, without paid inference
APIs. AI can surface aliases. Only deterministic identifier rules establish matches; a person confirms the
source and response.',left,y-22,col,10,15)
36 y2=para(c,'Private sign-in and database persistence. Server-validated actions. Versioned source and
inventory snapshots. Conflicting-save protection. Unknown identifiers cannot close; partial quantities stay
open.',right,y-22,col,10,15)
37 y=min(y1,y2)-25
38 para(c,'COMMERCIAL HYPOTHESIS',left,y,W-84,8.5,12,muted,True)
39 y=para(c,'Start with independent clinics using spreadsheet stock records. Test <b>$149/site/month</b>: at an
assumed $30/hour staff cost, five hours saved covers the fee. Existing enterprise competitors validate the
category; clinic demand and savings still need a supervised pilot.',left,y-22,W-84,10,15)
40 y-=20
41 para(c,'VALIDATION & LIMITS',left,y,W-84,8.5,12,muted,True)
42 y=para(c,'Automated tests cover scope pairing, missing identifiers, workflow gates, import validation and
version changes. Browser tests exercise sign-in, persistence, response completion and exports. This is a
hackathon MVP, not clinically validated or approved for unsupervised clinical use. Shared team roles, OCR
and complex recall scope remain future work.',left,y-20,W-84,9,13)
43 c.setStrokeColor(HexColor('#dce2d4'));c.line(42,88,W-42,88)
44 para(c,'CODE <link href="https://github.com/shi1720/DSH-Hacks-V2" color="#455d37">github.com/shi1720/DSH-
Hacks-V2</link>',42,75,W-84,8.5,12)
45 para(c,'SOURCES Baxter FA-2025-039 (Medline-hosted manufacturer PDF); FDA, What is a Medical Device Recall?
Full source links and limitations are in the repository.',42,57,W-84,7.5,10,muted)
46 para(c,'Project creator: Shivam Gupta. AI-assisted research, implementation, testing and presentation
preparation.',42,30,W-84,7,9,muted)
47 c.save()
48 # Authored code listing; vendor libraries and lockfiles remain in the repository.
49 files=[]
50 for prefix in ['app','lib/lotlight','db','tests']:
51     for p in (ROOT/prefix).rglob('*'):
52         if p.suffix in ['.ts','.tsx','.css','.mjs'] and p.is_file():files.append(p)
53 for name in ['scripts/build-ai.mjs','scripts/evaluate-ai.mjs','scripts/create-submission-
pdfs.py','playwright.config.ts','vite.config.ts','package.json','drizzle/0000_real_hellcat.sql']:
54     p=ROOT/name

```

scripts/create-submission-pdfs.py (continued)

```

55 if p.exists():files.append(p)
56 files += [p for p in (ROOT/'scripts').glob('*') if p.suffix in ['.mjs','.py']]
57 files += list((ROOT/'github').rglob('*.yml'))
58 files=sorted(set(files))
59 c=canvas.Canvas(str(OUT/'Lotlight-Source-Code.pdf'),pagesize=A4);c.setTitle('Lotlight | Authored source
   listing');c.setAuthor('Shivam Gupta');page=0
60 def newpage(title):
61     global page
62     if page:c.showPage()
63     page+=1;c.setFillColor(paper);c.rect(0,0,W,H,fill=1,stroke=0);c.setFillColor(ink);c.setFont('Helvetica-
   Bold',11);c.drawString(35,H-33,'LOTLIGHT / SOURCE
   RECORD');c.setFont('Helvetica',7);c.drawRightString(W-35,H-33,str(page));c.setFont('Helvetica-Bold',8);c.dra
   wString(35,H-56,title[:105]);c.setFont('Helvetica',7);c.setFillColor(muted);c.drawString(35,24,'Shivam Gupta
   \xb7 DSH Hacks V2 \xb7 AI-assisted development \xb7 Dependencies and full history in GitHub');return H-77
64 y=newpage('Contents and provenance')
65 y=para(c,'This listing includes application code, matching/workflow logic, database schema, tests and
   authored build/evaluation scripts. Vendored UI primitives, build scaffolding and package lockfiles are
   available in the repository. The MIT license applies to original code; dependencies retain their own
   licenses.',35,y,W-70,10,15)-20
66 for p in files:
67     if y<55:y=newpage('Contents (continued)')
68     c.setFont('Helvetica',8);c.setFillColor(ink);c.drawString(35,y,str(p.relative_to(ROOT)));y-=13
69 for p in files:
70     y=newpage(str(p.relative_to(ROOT)))
71     for num,line in enumerate(p.read_text().splitlines(),1):
72         line=line.expandtabs(2).encode('ascii','backslashreplace').decode('ascii')
73         chunks=textwrap.wrap(line,width=108,replace_whitespace=False,drop_whitespace=False) or ['']
74         for j,chunk in enumerate(chunks):
75             if y<45:y=newpage(str(p.relative_to(ROOT))+ ' (continued)')
76             c.setFillColor(muted if j else ink);c.setFont('Courier',7);c.drawString(35,y,(f'{num:4} ' if j==0 else '
   ')+chunk);y-=9
77 c.save()
78 print('Created one-page description and authored source listing:',len(files),'files,',page,'code pages')

```

scripts/evaluate-ai.mjs

```

1 import { chromium } from "playwright";
2 import fs from "node:fs/promises";
3 const context = await chromium.launchPersistentContext(
4   "/tmp/lotlight-ai-validation",
5   { headless: true, viewport: { width: 1440, height: 1000 } },
6 );
7 const page = context.pages()[0];
8 page.on('console', m=>console.log(m.type(),m.text().slice(0,200)));
9 page.on('requestfailed', r=>console.log('request failed',r.url(),r.failure()));
10 try {
11   await page.goto(process.env.LOTLIGHT_TEST_URL ?? "http://localhost:5174");
12   await page.locator('main[data-ready="true"]').waitFor();
13   const query = "Baxter CLEARLINK IV extension and solution sets";
14   const cases = [
15     { id: "alias-1", product: "IV line extender", relevant: true },
16     {
17       id: "alias-2",
18       product: "Intravenous fluid administration tubing",
19       relevant: true,
20     },
21     {
22       id: "alias-3",
23       product: "Extension tubing for infusion line",
24       relevant: true,
25     },
26     {
27       id: "alias-4",
28       product: "IV solution administration set",
29       relevant: true,
30     },
31     { id: "literal-1", product: "CLEARLINK extension set", relevant: true },
32     { id: "literal-2", product: "CLEARLINK solution set", relevant: true },
33     { id: "negative-1", product: "Examination gloves", relevant: false },
34     { id: "negative-2", product: "Sterile gauze dressing", relevant: false },
35     { id: "negative-3", product: "Digital thermometer", relevant: false },
36     { id: "negative-4", product: "Wheelchair footrest", relevant: false },
37     { id: "negative-5", product: "Blood glucose test strips", relevant: false },
38     { id: "negative-6", product: "Urine collection cup", relevant: false },
39   ];
40   const start = Date.now();
41   const scores = await page.evaluate(
42     ({ query, cases }) => {
43       new Promise((resolve, reject) => {
44         const w = new Worker("/ai/semantic-worker.js", { type: "module" });
45         const timeout = setTimeout(() => {
46           w.terminate();
47           reject(new Error("Model evaluation exceeded 180 seconds"));
48         }, 180000);
49         w.onmessage = (e) => {
50
51           if (e.data.type === "result") {
52             clearTimeout(timeout);
53             w.terminate();
54             resolve(e.data.scores);
55           }
56           if (e.data.type === "error") {
57             clearTimeout(timeout);
58             w.terminate();
59             reject(new Error(e.data.message));
60           }
61         };
62         w.onerror = (e) => {
63           clearTimeout(timeout);
64           reject(new Error(e.message));
65         };
66         w.postMessage({ query, items: cases });
67       });
68     }, { query, cases },
69   );
70   const ranked = cases
71     .map((c) => ({
72       ...c,
73       similarity: scores[c.id],
74       lexical: query
75         .toLowerCase()
76         .split(/\W+/)
77         .filter((w) => w.length > 3)
78         .some((w) => c.product.toLowerCase().includes(w)),
79     }))
80     .sort((a, b) => b.similarity - a.similarity);

```

scripts/evaluate-ai.mjs (continued)

```
81 const relevant = cases.filter((c) => c.relevant).length;
82 const report = {
83   date: new Date().toISOString(),
84   model: "Xenova/all-MiniLM-L6-v2",
85   revision: "751bfff37182d3f1213fa05d7196b954e230abad9",
86   dtype: "q8",
87   runtime: "actual Chromium browser / WASM",
88   query,
89   total: cases.length,
90   relevant,
91   elapsedIncludingModelLoadMs: Date.now() - start,
92   lexicalRecall:
93     ranked.filter((c) => c.relevant && c.lexical).length / relevant,
94   semanticTop6Recall:
95     ranked.slice(0, 6).filter((c) => c.relevant).length / relevant,
96   candidateThreshold: 0.3,
97   thresholdRelevant: ranked.filter((c) => c.relevant && c.similarity >= 0.3)
98     .length,
99   thresholdFalseCandidates: ranked.filter(
100     (c) => !c.relevant && c.similarity >= 0.3,
101   ).length,
102   limitations:
103     "Hand-authored synthetic smoke/ranking set, one query. Not clinical validation or an unbiased held-out
benchmark. Labels and threshold not independently adjudicated.",
104   ranked,
105 };
106 await fs.writeFile(
107   "docs/ai-evaluation.json",
108   JSON.stringify(report, null, 2) + "\n",
109 );
110 console.log(JSON.stringify(report, null, 2));
111 await page.getByRole("button", { name: "Open review", exact: true }).click();
112 await page.getByRole("button", { name: "Run local AI", exact: true }).click();
113 await page
114   .getByText(/Semantic comparison complete/)
115   .waitFor({ timeout: 120000 });
116 await page.screenshot({ path: "docs/images/ai-review.png", fullPage: true });
117 } finally {
118   await context.close();
119 }
```

scripts/execution-profile.mjs

```
1 import { readFileSync } from "node:fs";
2
3 export function readExecutionProfile() {
4   let settings;
5   try {
6     settings = JSON.parse(readFileSync(new URL("../.sites-runtime/execution-profile.json", import.meta.url),
7     "utf8"));
8   } catch (error) {
9     // Clean clones and remote builds have no checkout-local selection.
10    if (error.code === "ENOENT") return "portable";
11    throw error;
12  }
13  if (!["managed-linux", "portable"].includes(settings?.executionProfile)) {
14    throw new Error("Invalid local execution profile; rerun the Sites plugin's configure-execution-
15    profile.mjs.");
16  }
17  return settings.executionProfile;
18 }
```

scripts/install-ci.mjs

```
1 import { spawnSync } from "node:child_process";
2 import { accessSync, constants } from "node:fs";
3 import path from "node:path";
4 import { projectRoot } from "../sites-env.mjs";
5 import { readExecutionProfile } from "../execution-profile.mjs";
6
7 if (!process.env.npm_execpath) {
8   throw new Error("Run this installer with npm run install:ci.");
9 }
10
11 if (![
12   "SHARP_IGNORE_GLOBAL_LIBVIPS",
13   "SHARP_FORCE_GLOBAL_LIBVIPS",
14   "npm_config_build_from_source",
15   "NPM_CONFIG_BUILD_FROM_SOURCE",
16 ].some((key) => key in process.env)) {
17   process.env.SHARP_IGNORE_GLOBAL_LIBVIPS = "1";
18 }
19
20 if (readExecutionProfile() === "managed-linux") {
21   const result = spawnSync("bash", [path.join(projectRoot, "scripts/install-ci.sh")], {
22     stdio: "inherit",
23   });
24   if (result.error) throw result.error;
25   process.exit(result.status ?? 1);
26 }
27
28 // Invoke npm's JavaScript entrypoint, avoiding platform-specific shell shims.
29 const installed = spawnSync(
30   process.execPath,
31   [
32     process.env.npm_execpath, "ci", "--prefix", projectRoot, "--workspaces=false",
33     "--include=dev", "--include=optional", "--prefer-offline", "--no-audit", "--no-fund",
34   ],
35   { stdio: "inherit" },
36 );
37 if (installed.error) throw installed.error;
38 if (installed.status !== 0) process.exit(installed.status ?? 1);
39
40 try {
41   accessSync(
42     path.join(
43       projectRoot, "node_modules", ".bin",
44       process.platform === "win32" ? "vinext.cmd" : "vinext",
45     ),
46     process.platform === "win32" ? constants.F_OK : constants.X_OK,
47   );
48 } catch {
49   console.error("npm ci exited successfully but the local vinext executable is unavailable.");
50   process.exitCode = 69;
51 }
```

scripts/pnpm-install.mjs

```

1 import { spawn } from "node:child_process";
2 import { createHash } from "node:crypto";
3 import {
4   accessSync,
5   closeSync,
6   constants,
7   fstatSync,
8   ftruncateSync,
9   openSync,
10  readFileSync,
11  writeFileSync,
12  writeSync,
13 } from "node:fs";
14 import { constants as osConstants } from "node:os";
15 import path from "node:path";
16 import { createInterface } from "node:readline";
17 import { fileURLToPath } from "node:url";
18
19 const MAX_PACKAGES = 100_000;
20 const CACHE_SEEDS = new Set(["seed_used", "seed_unavailable", "seed_lockfile_mismatch",
21   "decision_unavailable", "not_applicable"]);
22 const STORE_STATES = new Set(["created", "seeded", "reused", "unavailable"]);
23 // The caller may use operational status only for
24 // initial-setup fallback; an established project always retains pnpm. Broad
25 // fetch failures can include TLS/auth errors and stay fatal.
26 const OPERATIONAL_FAILURE_CODES = new Set([
27   "ERR_PNPM_UNEXPECTED_STORE",
28   "ERR_PNPM_UNEXPECTED_VIRTUAL_STORE",
29   "ERR_PNPM_OUTDATED_LOCKFILE",
30   "ERR_PNPM_FROZEN_LOCKFILE_WITH_OUTDATED_LOCKFILE",
31   "ERR_PNPM_NO_LOCKFILE",
32   "ERR_PNPM_FETCH_408",
33   "ERR_PNPM_FETCH_429",
34   "ERR_PNPM_FETCH_500",
35   "ERR_PNPM_FETCH_502",
36   "ERR_PNPM_FETCH_503",
37   "ERR_PNPM_FETCH_504",
38 ]);
39 // These are pnpm-reported package IDs, not an assertion about network bytes or
40 // image-cache provenance. Do not interpret an installed-tree no-op as 100% reuse.
41 export class InstallProgress {
42   constructor(project) {
43     this.project = path.resolve(project);
44     this.reused = new Set();
45     this.downloaded = new Set();
46     this.complete = false;
47     this.added = undefined;
48     this.invalid = false;
49   }
50
51   accept(event) {
52     if (event?.name === "pnpm:stats" && event.prefix === this.project &&
53       Number.isSafeInteger(event.added) && event.added >= 0) this.added = event.added;
54     if (event?.name === "pnpm:stage" && event.stage === "importing_done" &&
55       event.prefix === this.project) this.complete = true;
56     if (event?.name !== "pnpm:progress" || event.requester !== this.project ||
57       ![ "fetched", "found_in_store" ].includes(event.status)) return;
58     if (typeof event.packageId !== "string" || !event.packageId || event.packageId.length > 4096 ||
59       this.reused.size + this.downloaded.size >= MAX_PACKAGES) {
60       this.invalid = true;
61       return;
62     }
63     if (event.status === "fetched") {
64       this.reused.delete(event.packageId);
65       this.downloaded.add(event.packageId);
66     } else if (!this.downloaded.has(event.packageId)) {
67       this.reused.add(event.packageId);
68     }
69   }
70
71   counts(success) {
72     if (!success || !this.complete || !(this.added > 0) || this.invalid ||
73       this.reused.size + this.downloaded.size === 0) return {};
74     return { packages_reused: this.reused.size, packages_downloaded: this.downloaded.size };
75   }
76 }
77
78 function openReport() {
79   const report = process.env.SITES_INSTALL_REPORT_PATH;

```

scripts/pnpm-install.mjs (continued)

```

80   if (!report || !constants.O_NOFOLLOW) return undefined;
81   let fd;
82   try {
83     fd = openSync(report, constants.O_WRONLY | constants.O_NOFOLLOW | constants.O_NONBLOCK);
84     if (fstatSync(fd).isFile()) return fd;
85   } catch {}
86   if (fd !== undefined) closeSync(fd);
87   return undefined;
88 }
89
90 function writeReport(fd, report) {
91   if (fd === undefined) return;
92   try {
93     const body = `${JSON.stringify(report)}\n`;
94     writeSync(fd, body, 0, "utf8");
95     ftruncateSync(fd, Buffer.byteLength(body));
96   } catch {}
97   // Optional telemetry must not change installation behavior or expose its path.
98 }
99 }
100
101 function showLine(line, progress, failure) {
102   let event;
103   try {
104     event = JSON.parse(line);
105   } catch {}
106   process.stdout.write(`${line}\n`);
107   return;
108 }
109 progress.accept(event);
110 if (event?.level === "error" && failure.code !== 65) {
111   failure.code = event.name === "pnpm" && OPERATIONAL_FAILURE_CODES.has(event.err?.code)
112     ? 70 : 65;
113 }
114 const message = event?.message ?? event?.err?.message;
115 if (event?.name === "pnpm:lifecycle" && typeof event.line === "string") {
116   process.stdout.write(`${event.line}\n`);
117 } else if (typeof message === "string" && ["warn", "error", "info"].includes(event.level)) {
118   const output = event.level === "error" ? process.stderr : process.stdout;
119   output.write(`${message}\n`);
120 }
121 }
122
123 async function openLock(filename, waitSeconds) {
124   let fd;
125   try {
126     if (!filename || !constants.O_NOFOLLOW || !constants.O_NONBLOCK) throw new Error("Unsupported lock");
127     fd = openSync(filename, constants.O_CREAT | constants.O_RDWR | constants.O_NOFOLLOW |
128       constants.O_NONBLOCK, 0o600);
129     if (!fstatSync(fd).isFile()) throw new Error("Expected a regular lock file");
130     const child = spawn("flock", ["-w", waitSeconds, "3"], { stdio: ["ignore", "ignore", "ignore", fd] });
131     const code = await new Promise((resolve) => {
132       child.once("error", () => resolve(undefined));
133       child.once("close", resolve);
134     });
135     if (code === 0) return { status: "locked", fd };
136     closeSync(fd);
137     return { status: code === undefined ? "unavailable" : "busy" };
138   } catch {}
139   if (fd !== undefined) closeSync(fd);
140   return { status: "unavailable" };
141 }
142
143 async function holdInstallLocks(projectLock, sharedLock, waitSeconds) {
144   // The shell holds both leases through stdin. EOF (including caller exit)
145   // releases the same open file descriptions that flock locked in its child.
146   const requests = createInterface({ input: process.stdin, crlfDelay: Infinity });
147   const project = await openLock(projectLock, "0");
148   let shared;
149   try {
150     process.stdout.write(`${project.status}\n`);
151     for await (const request of requests) {
152       if (request === "shared" && project.fd !== undefined) {
153         shared ??= await openLock(sharedLock, waitSeconds);
154         process.stdout.write(`${shared.status}\n`);
155       } else if (request === "release-shared") {
156         if (shared?.fd !== undefined) closeSync(shared.fd);
157         shared = undefined;
158         process.stdout.write("released\n");
159       }
160     }
161   }

```

scripts/pnpm-install.mjs (continued)

```

159     } else {
160       process.stdout.write("unavailable\n");
161     }
162   }
163 } finally {
164   if (shared?.fd !== undefined) closeSync(shared.fd);
165   if (project.fd !== undefined) closeSync(project.fd);
166   requests.close();
167 }
168 }
169
170 async function main() {
171   const argsIn = process.argv.slice(2);
172   if (argsIn[0] === "--hold-install-locks") {
173     await holdInstallLocks(argsIn[1], argsIn[2], argsIn[3]);
174     return;
175   }
176   if (argsIn[0] === "--report-store") {
177     const [, cacheSeed, storeScope, storeState, stdout] = argsIn;
178     if (!CACHE_SEEDS.has(cacheSeed) || !["project", "workspace", "unknown"].includes(storeScope) ||
179       !STORE_STATES.has(storeState)) { process.exitCode = 64; return; }
180     const report = { version: 1, cache_seed: cacheSeed, store_scope: storeScope, store_state: storeState };
181     const fd = openReport();
182     writeReport(fd, report);
183     if (fd !== undefined) closeSync(fd);
184     if (stdout === "1") process.stdout.write(`${JSON.stringify(report)}\n`);
185     return;
186   }
187   const [cacheSeed, storeScope, storeState, store, executable, ...prefix] = argsIn;
188   if (!CACHE_SEEDS.has(cacheSeed) || !["project", "workspace"].includes(storeScope) ||
189     !STORE_STATES.has(storeState) || storeState === "unavailable" ||
190     !store || !path.isAbsolute(store) || !executable) {
191     process.stderr.write("Invalid pnpm installation arguments.\n");
192     process.exitCode = 64;
193     return;
194   }
195   const fd = openReport();
196   const report = { version: 1, cache_seed: cacheSeed, store_scope: storeScope, store_state: storeState };
197   writeReport(fd, report);
198   const progress = new InstallProgress(process.cwd());
199   const failure = {};
200   const env = { ...process.env };
201   delete env.SITES_INSTALL_REPORT_PATH;
202   const args = [...prefix, "install", "--prod=false", "--ignore-scripts=false",
203     "--frozen-lockfile", "--prefer-offline", "--store-dir", store,
204     "--cache-dir", path.join(store, "policy-cache"), "--fetch-retries=0",
205     "--fetch-timeout=30000", "--network-concurrency=1", "--reporter=ndjson",
206     "--package-import-method=clone-or-copy"];
207   let result = { code: 1, signal: null };
208   let receivedSignal;
209   try {
210     const child = spawn(executable, args, { env, stdio: ["inherit", "pipe", "inherit"] });
211     const lines = createInterface({ input: child.stdout, crlfDelay: Infinity });
212     lines.on("line", (line) => showLine(line, progress, failure));
213     // timeout/exec owns the inherited group. Relaying would deliver signals twice.
214     const signalHandlers = ["SIGINT", "SIGHUP", "SIGTERM"].map((signal) => {
215       const handler = () => { receivedSignal ??= signal; };
216       process.on(signal, handler);
217       return [signal, handler];
218     });
219     let spawnError;
220     child.once("error", (error) => { spawnError = error; });
221     result = await new Promise((resolve) => child.once("close", (code, signal) => {
222       resolve({ code: spawnError ? (spawnError.code === "ENOENT" ? 127 : 1) : code, signal });
223     }));
224     for (const [signal, handler] of signalHandlers) process.removeListener(signal, handler);
225     lines.close();
226     result.signal ??= receivedSignal;
227     if (spawnError) {
228       process.stderr.write("Unable to start pnpm.\n");
229       result.code = spawnError.code === "ENOENT" ? 127 : 65;
230     } else if (result.code !== 0 && !result.signal) {
231       result.code = failure.code ?? 65;
232     }
233     if (result.code === 0 && !result.signal) {
234       accessSync("node_modules/.bin/vinext", constants.X_OK);
235       const lock = readFileSync("pnpm-lock.yaml");
236       writeFileSync("node_modules/.sites-install.json", `${JSON.stringify({
237         package_manager: "pnpm@11.25.0",
238         lockfile_sha256: createHash("sha256").update(lock).digest("hex"),

```

scripts/pnpm-install.mjs (continued)

```
239     node: process.version,
240     platform: `${process.platform}-${process.arch}`,
241   }, null, 2)}\n`);
242 }
243 } catch {
244   process.stderr.write("Dependency setup did not produce a usable Vinext installation.\n");
245   result = { code: 65, signal: receivedSignal ?? null };
246 } finally {
247   Object.assign(report, progress.counts(result.code === 0 && !result.signal));
248   writeReport(fd, report);
249   if (fd !== undefined) closeSync(fd);
250 }
251 if (report.packages_reused !== undefined) {
252   process.stdout.write(`[sites] pnpm reused ${report.packages_reused} packages and downloaded
253   ${report.packages_downloaded}\n`);
254 }
255 if (result.code === 0 && !result.signal) process.stdout.write("[sites] dependency setup passed\n");
256 process.exitCode = result.signal ? 128 + (osConstants.signals[result.signal] ?? 0) : result.code ?? 1;
257 if (result.signal) {
258   try { process.kill(process.pid, result.signal); } catch {}
259 }
260 }
261 if (process.argv[1] && path.resolve(process.argv[1]) === fileURLToPath(import.meta.url)) await main();
```

scripts/record-demo.mjs

```

1 import { chromium } from "playwright";
2 import fs from "node:fs/promises";
3 const base = process.env.LOTLIGHT_TEST_URL ?? "http://localhost:5174";
4 await fs.mkdir(".artifact-build/video", { recursive: true });
5 const context = await chromium.launchPersistentContext(
6   "/tmp/lotlight-ai-validation",
7   {
8     headless: true,
9     viewport: { width: 1440, height: 1000 },
10    recordVideo: {
11      dir: ".artifact-build/video",
12      size: { width: 1440, height: 1000 },
13    },
14  },
15 );
16 const page = context.pages()[0];
17 await page.goto(base);
18 await page.locator('main[data-ready="true"]').waitFor();
19 const start = Date.now();
20 const markers = [];
21 async function until(seconds) {
22   const wait = seconds * 1000 - (Date.now() - start);
23   if (wait > 0) await page.waitForTimeout(wait);
24 }
25 async function scene(seconds, name, fn) {
26   await until(seconds);
27   markers.push({ seconds, name });
28   console.log("Scene", seconds, name);
29   if (fn) await fn();
30 }
31 async function moveClick(locator) {
32   await locator.scrollIntoViewIfNeeded();
33   const box = await locator.boundingBox();
34   if (box)
35     await page.mouse.move(box.x + box.width * 0.5, box.y + box.height * 0.5, {
36       steps: 20,
37     });
38   await page.waitForTimeout(350);
39   await locator.click();
40 }
41 try {
42   await scene(0, "A notice is only the beginning");
43   await scene(19, "Bring the inventory", async () => {
44     await moveClick(
45       page.getByRole("button", { name: "Inventory", exact: true }),
46     );
47     await moveClick(page.getByRole("button", { name: "Import CSV" }));
48     await moveClick(
49       page.getByRole("button", { name: "Use synthetic example" }),
50     );
51   });
52   await scene(31, "Validate and import", async () => {
53     await moveClick(page.getByRole("button", { name: "Validate & import" }));
54   });
55   await scene(39, "Review the original source", async () => {
56     await moveClick(
57       page.getByRole("button", { name: /Recall workspace/ }).first(),
58     );
59     await moveClick(
60       page.getByRole("button", { name: "Open review", exact: true }),
61     );
62     await moveClick(page.getByRole("tab", { name: "Source & scope" }));
63   });
64   await scene(53, "Human source approval", async () => {
65     await page.getByRole("checkbox").check();
66     await moveClick(
67       page.getByRole("button", { name: "Confirm source review" }),
68     );
69   });
70   await scene(61, "AI finds an alias", async () => {
71     await moveClick(page.getByRole("tab", { name: /Inventory review/ }));
72     await moveClick(
73       page.getByRole("button", { name: "Run local AI", exact: true }),
74     );
75     await page
76       .getByText(/Semantic comparison complete/)
77       .waitFor({ timeout: 120000 });
78     await moveClick(
79       page.getByRole("button", {
80         name: "Show evidence for IV line extender in Mobile cart",

```

scripts/record-demo.mjs (continued)

```

81     }},
82   );
83 });
84 await scene(79, "Exact identifiers and rationale", async () => {
85   await moveClick(
86     page.getByRole("button", {
87       name: "Show evidence for CLEARLINK extension set in Infusion room",
88     }),
89   );
90   await page
91     .locator(".match-row")
92     .filter({
93       has: page.getByRole("button", {
94         name: "Show evidence for CLEARLINK extension set in Infusion room",
95       }),
96     })
97     .scrollIntoViewIfNeeded();
98 });
99 await scene(92, "Record a response", async () => {
100   const row = page
101     .locator(".match-row")
102     .filter({
103       has: page.getByRole("button", {
104         name: "Show evidence for CLEARLINK extension set in Infusion room",
105       }),
106     });
107   await moveClick(row.getByRole("button", { name: "Assign response" }));
108   await page
109     .getByLabel("Responsible person")
110     .fill("Maya Rao \xb7 demo coordinator");
111   await page.getByLabel("Response due date").fill("2026-09-18");
112   await page.getByRole("combobox", { name: "Response stage" }).click();
113   await page.getByRole("option", { name: /Completed/ }).click();
114   await page
115     .getByLabel("Physical-label and current-source verification")
116     .fill(
117       "Synthetic demo: checked physical catalog and lot labels against the original manufacturer letter.",
118     );
119   await page.getByLabel("Units handled").fill("24");
120   await page
121     .getByLabel("Response evidence / reference")
122     .fill(
123       "Synthetic demo evidence: isolation log Q-104 and supplier return reference RMA-210.",
124     );
125 });
126 await scene(113, "Documented completion", async () => {
127   await moveClick(
128     page.getByRole("button", { name: "Save response", exact: true }),
129   );
130 });
131 await scene(121, "Missing evidence stays open", async () => {
132   await moveClick(
133     page.getByRole("button", {
134       name: "Show evidence for CLEARLINK extension set in Procedure room",
135     }),
136   );
137   const row = page
138     .locator(".match-row")
139     .filter({
140       has: page.getByRole("button", {
141         name: "Show evidence for CLEARLINK extension set in Procedure room",
142       }),
143     });
144   await moveClick(row.getByRole("button", { name: "Assign response" }));
145   await page.getByRole("combobox", { name: "Response stage" }).click();
146 });
147 await scene(137, "Portable audit trail", async () => {
148   await page.keyboard.press("Escape");
149   await page.getByRole("button", { name: "Cancel", exact: true }).click();
150   await moveClick(page.getByRole("button", { name: "Export record" }));
151 });
152 await scene(146, "Printable record", async () => {
153   await moveClick(
154     page.getByRole("button", { name: /Readable audit report/ }),
155   );
156   await page.evaluate(() => window.scrollTo({ top: 0, behavior: "smooth" }));
157 });
158 await scene(156, "Business hypothesis and boundaries", async () => {
159   await moveClick(page.getByRole("button", { name: "Evidence & approach" }));
160   await page

```

scripts/record-demo.mjs (continued)

```

161     .getByRole("heading", { name: "A credible first customer" })
162     .scrollIntoViewIfNeeded();
163   });
164   await scene(174, "Lotlight closing", async () => {
165     await moveClick(
166       page.getByRole("button", { name: /Recall workspace/ }).first(),
167     );
168     await page.evaluate(() => window.scrollTo({ top: 0, behavior: "smooth" }));
169   });
170   await until(185);
171   const videoPath = await page.video().path();
172   await context.close();
173   await fs.copyFile(videoPath, ".artifact-build/video/lotlight-demo.webm");
174   await fs.writeFile(
175     "submission/VIDEO-TIMING.json",
176     JSON.stringify({ durationSeconds: 185, markers }, null, 2),
177   );
178   console.log("Recorded .artifact-build/video/lotlight-demo.webm");
179 } catch (e) {
180   await context.close();
181   throw e;
182 }

```

scripts/run-framework.mjs

```
1 import { spawnSync } from "node:child_process";
2 import { fileURLToPath } from "node:url";
3 import { readExecutionProfile } from "../execution-profile.mjs";
4
5 const [command, ...args] = process.argv.slice(2);
6 if (!["dev", "build"].includes(command)) throw new Error("Expected dev or build.");
7 const managedLinux = readExecutionProfile() === "managed-linux";
8
9 if (managedLinux && command === "build") {
10   const result = spawnSync("bash", [
11     fileURLToPath(new URL("../build-verified.sh", import.meta.url)), ...args,
12   ], { stdio: "inherit" });
13   if (result.error) throw result.error;
14   process.exit(result.status ?? 1);
15 }
16
17 // Import in this process so the preview owner retains its PID and signals.
18 const cli = new URL(managedLinux
19   ? "../node_modules/vite/bin/vite.js"
20   : "../node_modules/vinext/dist/cli.js", import.meta.url);
21 process.argv = [process.execPath, fileURLToPath(cli), command,
22   ...(!managedLinux && command === "dev" ? ["--port", "5173"] : []), ...args];
23 await import(cli.href);
```

scripts/sites-env.mjs

```
1 import { mkdirSync } from "node:fs";
2 import path from "node:path";
3 import { fileURLToPath } from "node:url";
4
5 export const projectRoot = fileURLToPath(new URL("../", import.meta.url));
6 const runtimeRoot = process.env.SITES_RUNTIME_ROOT || path.join(projectRoot, ".sites-runtime");
7
8 process.env.CLOUDFLARE_CF_FETCH_ENABLED ||= "false";
9 process.env.WRANGLER_SEND_METRICS ||= "false";
10 process.env.WRANGLER_WRITE_LOGS ||= "false";
11 process.env.WRANGLER_LOG_PATH ||= path.join(runtimeRoot, "wrangler/logs");
12 process.env.WRANGLER_REGISTRY_PATH ||= path.join(runtimeRoot, "wrangler/dev-registry");
13 process.env.MINIFLARE_REGISTRY_PATH ||= path.join(runtimeRoot, "wrangler/registry");
14
15 process.chdir(projectRoot);
16 for (const directory of [
17   path.dirname(process.env.WRANGLER_LOG_PATH),
18   process.env.WRANGLER_REGISTRY_PATH,
19   process.env.MINIFLARE_REGISTRY_PATH,
20 ]) {
21   mkdirSync(directory, { recursive: true });
22 }
```

tests/e2e/accessibility.spec.ts

```
1 import { test, expect } from '@playwright/test';
2 import AxeBuilder from '@axe-core/playwright';
3 test('dashboard and review have no axe WCAG A/AA violations', async({page})=>{await page.goto('/');await
  page.locator('main[data-ready="true"]').waitFor();expect((await new
  AxeBuilder({page}).withTags(['wcag2a', 'wcag2aa']).analyze()).violations).toEqual([]);await
  page.getByRole('button', {name: 'Open review', exact:true}).click();expect((await new
  AxeBuilder({page}).withTags(['wcag2a', 'wcag2aa']).analyze()).violations).toEqual([]);});
```

tests/e2e/workflow.spec.ts

```

1 import { test, expect } from "@playwright/test";
2 import { SAMPLE_CSV } from "../../lib/lotlight/sample";
3 async function ready(page: import("@playwright/test").Page) {
4   await page.goto("/");
5   await page.locator('main[data-ready="true"]').waitFor();
6 }
7 async function approve(page: import("@playwright/test").Page) {
8   await page
9     .getByRole("button", { name: "Open review", exact: true })
10    .first()
11    .click();
12   await page.getByRole("tab", { name: "Source & scope" }).click();
13   await page.getByRole("checkbox").check();
14   await page.getByRole("button", { name: "Confirm source review" }).click();
15   await expect(
16     page.getByText("Source review recorded", { exact: true }),
17   ).toBeVisible();
18 }
19 test("full demo: gated source, staff response, export and unknown cannot complete", async ({
20   page,
21 }) => {
22   await ready(page);
23   await page.getByRole("button", { name: "Open review", exact: true }).click();
24   await page
25     .getByRole("button", {
26       name: "Show evidence for CLEARLINK extension set in Infusion room",
27     })
28     .click();
29   await expect(
30     page.getByRole("button", { name: "Assign response" }),
31   ).toBeDisabled();
32   await page.getByRole("tab", { name: "Source & scope" }).click();
33   await page.getByRole("checkbox").check();
34   await page.getByRole("button", { name: "Confirm source review" }).click();
35   await page.getByRole("tab", { name: "/Inventory review/" }).click();
36   await page
37     .getByRole("button", {
38       name: "Show evidence for CLEARLINK extension set in Infusion room",
39     })
40     .click();
41   await page.getByRole("button", { name: "Assign response" }).click();
42   await page.getByLabel("Responsible person").fill("Maya Rao");
43   await page.getByRole("combobox", { name: "Response stage" }).click();
44   await page.getByRole("option", { name: "/Completed/" }).click();
45   await page
46     .getByLabel("Physical-label and current-source verification")
47     .fill(
48       "Checked 24 physical labels against the full manufacturer source in this synthetic demonstration.",
49     );
50   await page.getByLabel("Units handled").fill("24");
51   await page
52     .getByLabel("Response evidence / reference")
53     .fill(
54       "Synthetic training record: isolation log Q-104 and supplier return reference RMA-210.",
55     );
56   await page
57     .getByRole("button", { name: "Save response", exact: true })
58     .click();
59   await expect(
60     page.getByText("Response complete", { exact: true }),
61   ).toBeVisible();
62   await page
63     .getByRole("button", {
64       name: "Show evidence for CLEARLINK extension set in Procedure room",
65     })
66     .click();
67   await page
68     .getByRole("button", { name: "Assign response", exact: true })
69     .click();
70   await page.getByRole("combobox", { name: "Response stage" }).click();
71   await expect(page.getByRole("option", { name: "/Completed/" })).toHaveAttribute(
72     "data-disabled",
73     "",
74   );
75   await page.keyboard.press("Escape");
76   await page.getByRole("button", { name: "Cancel", exact: true }).click();
77   await page.getByRole("button", { name: "Export record" }).click();
78   const downloaded = page.waitForEvent("download");
79   await page.getByRole("button", { name: "/Complete audit record/" }).click();
80   const file = await downloaded;

```

tests/e2e/workflow.spec.ts (continued)

```

81 expect(file.suggestedFilename()).toBe("lotlight-audit-record.json");
82 const stream = await file.createReadStream();
83 let output = "";
84 for await (const chunk of stream!) output += chunk;
85 const exported = JSON.parse(output);
86 expect(exported.payload.workspace.actions[0].handledQuantity).toBe(24);
87 expect(exported.payload.inventoryProvenance).toBe("synthetic");
88 expect(exported.integrity.digest).toHaveLength(64);
89 });
90 test("CSV errors preserve draft, import works, corrections preserve provenance", async ({
91   page,
92 }) => {
93   await ready(page);
94   await page.getByRole("button", { name: "Inventory", exact: true }).click();
95   await page.getByRole("button", { name: "Import CSV" }).click();
96   await page
97     .getByLabel("Inventory CSV", { exact: true })
98     .fill(SAMPLE_CSV.replace("24", "-2"));
99   await page.getByRole("button", { name: "Validate & import" }).click();
100  await expect(page.getByRole("alert")).toContainText("whole number");
101  await page.getByRole("button", { name: "Use synthetic example" }).click();
102  await page.getByRole("button", { name: "Validate & import" }).click();
103  await expect(page.getByText(/synthetic \xb7 8 lines/)).toBeVisible();
104  await page
105    .getByRole("button", {
106      name: "Correct CLEARLINK extension set in Procedure room",
107    })
108    .click();
109  await page.getByLabel("Lot", { exact: true }).fill("R25C31031");
110  await page.getByRole("button", { name: "Save correction" }).click();
111  await expect(page.getByText("R25C31031", { exact: true })).toHaveCount(2);
112 });
113 test("new source form resets after success; unsupported source pairing rejected", async ({
114   page,
115 }) => {
116   await ready(page);
117   await page.getByRole("button", { name: "New recall review" }).click();
118   await page
119     .getByRole("button", { name: "Use historical training extract" })
120     .click();
121   await page.getByRole("button", { name: "Save for source review" }).click();
122   await expect(
123     page.getByRole("button", { name: "Open review", exact: true }),
124   ).toHaveCount(2);
125   await page.getByRole("button", { name: "New recall review" }).click();
126   await expect(page.getByLabel("Product / review title")).toHaveValue("");
127 });
128 test("local sign-in, account persistence, stale revision and origin rejection", async ({
129   page,
130   request,
131 }) => {
132   await ready(page);
133   expect((await request.get("/api/workspace")).status()).toBe(401);
134   await page.getByRole("link", { name: "Sign in to save" }).click();
135   await expect(
136     page.getByText("Account storage", { exact: true }),
137   ).toBeVisible();
138   const reset = await page.evaluate(async () => {
139     const s = await (await fetch("/api/workspace")).json() as {revision:number};
140     const r = await fetch("/api/workspace", {
141       method: "POST",
142       headers: { "Content-Type": "application/json" },
143       body: JSON.stringify({
144         revision: s.revision,
145         command: { type: "reset_demo" },
146       }),
147     });
148     return r.status;
149   });
150   expect(reset).toBe(200);
151   await page.reload();
152   await page.locator('main[data-ready="true"]').waitFor();
153   await expect(
154     page.getByText("Account storage", { exact: true }),
155   ).toBeVisible();
156   await approve(page);
157   await page.reload();
158   await page.locator('main[data-ready="true"]').waitFor();
159   await expect(
160     page.getByText("SOURCE REVIEWED", { exact: true }),

```

tests/e2e/workflow.spec.ts (continued)

```

161 ).toBeVisible();
162 const status = await page.evaluate(async () => {
163   const s = await (await fetch("/api/workspace")).json() as {revision:number};
164   const r = await fetch("/api/workspace", {
165     method: "POST",
166     headers: { "Content-Type": "application/json" },
167     body: JSON.stringify({
168       revision: s.revision - 1,
169       command: { type: "reset_demo" },
170     }),
171   });
172   return r.status;
173 });
174 expect(status).toBe(409);
175 const bad = await page.request.post("/api/workspace", {
176   headers: {
177     Origin: "https://evil.example",
178     "Content-Type": "application/json",
179   },
180   data: { revision: 0, command: { type: "reset_demo" } },
181 });
182 expect(bad.status()).toBe(403);
183 await page.getByRole("link", { name: "Sign out" }).click();
184 await expect(
185   page.getByRole("link", { name: "Sign in to save" }),
186 ).toBeVisible();
187 expect((await page.request.get("/api/workspace")).status()).toBe(401);
188 });
189 test("mobile workflow fits viewport and retains navigation", async ({
190   page,
191 }) => {
192   await page.setViewportSize({ width: 390, height: 844 });
193   await ready(page);
194   expect(
195     await page.evaluate(
196       () => document.documentElement.scrollWidth <= window.innerWidth,
197     ),
198   ).toBe(true);
199   await page.getByRole("button", { name: "Open review", exact: true }).click();
200   await expect(
201     page.getByRole("button", { name: "Run local AI" }),
202   ).toBeVisible();
203   await page.getByRole("button", { name: "Toggle Sidebar" }).click();
204   await page.getByRole("button", { name: "Inventory", exact: true }).click();
205   await page.keyboard.press("Escape");
206   await expect(
207     page.getByRole("heading", { name: "Your inventory." }),
208   ).toBeVisible();
209 });
210 test("WebMCP read tool handles valid and invalid input without mutation", async ({
211   page,
212 }) => {
213   await page.addInitScript(() => {
214     (window as any).capturedTools = [];
215     Object.defineProperty(document, "modelContext", {
216       value: {
217         registerTool(tool: any) {
218           (window as any).capturedTools.push(tool);
219         },
220       },
221     });
222   });
223   await ready(page);
224   const result = await page.evaluate(() => {
225     const tool = (window as any).capturedTools.at(-1);
226     const good = tool.execute({});
227     let rejected = false;
228     try {
229       tool.execute({ extra: 1 });
230     } catch {
231       rejected = true;
232     }
233     return { name: tool.name, good, rejected };
234   });
235   expect(result.name).toBe("lotlight_read_review");
236   expect(result.good.inventoryLines).toBe(8);
237   expect(result.rejected).toBe(true);
238 });
239
240 test('text-based PDF notice import preserves text and extracts explicit pairs', async ({page})=>{

```

tests/e2e/workflow.spec.ts (continued)

```
241 await ready(page);await page.getByRole('button',{name:'New recall review'}).click();await
    page.getByLabel('Upload recall notice').setInputFiles('public/samples/baxter-training-extract.pdf');await
    expect(page.getByLabel('Original notice text / clearly labeled training
    extract')).toContainText('R25C31031');await expect(page.getByLabel('Catalog',{exact:true})).toHaveCount(2);
242 });
```

tests/engine.test.ts

```

1 import { test } from "node:test";
2 import assert from "node:assert/strict";
3 import {
4   matchItem,
5   identifier,
6   extractScope,
7   validateScope,
8   parseCsv,
9   exportCsv,
10 } from "../lib/lotlight/engine";
11 import {
12   initialWorkspace,
13   SAMPLE_CSV,
14   sampleNotice,
15 } from "../lib/lotlight/sample";
16 import { reduceWorkspace, activeAction } from "../lib/lotlight/commands";
17 import type { InventoryItem, Notice } from "../lib/lotlight/types";
18 const fixture = () => initialWorkspace();
19 const item = (patch: Partial<InventoryItem> = {}) => ({
20   ...fixture().inventory[0],
21   ...patch,
22 });
23 const approve = () =>
24   reduceWorkspace(
25     fixture(),
26     { type: "approve_notice", noticeId: "baxter-demo", sourceConfirmed: true },
27     "Test reviewer",
28   );
29 const action = (patch: Record<string, unknown> = {}) => ({
30   type: "save_action",
31   noticeId: "baxter-demo",
32   itemId: "sample-1",
33   owner: "Maya Rao",
34   dueDate: "2026-10-01",
35   status: "assigned",
36   verification: "",
37   evidence: "",
38   handledQuantity: 0,
39   ...patch,
40 });
41 test("exact match requires manufacturer, paired catalog and lot", () =>
42   assert.equal(matchItem(item(), sampleNotice).kind, "exact"));
43 test("cross-product lot must not match", () =>
44   assert.equal(
45     matchItem(item({ lot: "R25A13024" }), sampleNotice).kind,
46     "unlisted",
47   ));
48 test("missing lot never becomes exact at maximum semantic similarity", () =>
49   assert.equal(matchItem(item({ lot: "" }), sampleNotice, 1).kind, "missing"));
50 test("semantic candidate cannot override explicit catalog mismatch", () =>
51   assert.equal(
52     matchItem(item({ catalog: "WRONG" }), sampleNotice, 1).kind,
53     "unlisted",
54   ));
55 test("missing catalog can only be an AI candidate", () =>
56   assert.equal(
57     matchItem(item({ catalog: "" }), sampleNotice, 0.9).kind,
58     "possible",
59   ));
60 test("different manufacturer cannot become exact", () =>
61   assert.equal(
62     matchItem(item({ manufacturer: "Other" }), sampleNotice, 1).kind,
63     "missing",
64   ));
65 test("missing manufacturer remains unresolved", () =>
66   assert.equal(
67     matchItem(item({ manufacturer: "" }), sampleNotice).kind,
68     "missing",
69   ));
70 test("normalization preserves zeros, punctuation and ambiguous letters", () => {
71   for (const [a, b] of [
72     ["00123", "123"],
73     ["AB-12", "AB12"],
74     ["R2501", "R2501"],
75     ["R25I1", "R2511"],
76   ])
77     assert.notEqual(identifier(a), identifier(b));
78   assert.equal(identifier(" ab-12 "), "AB-12");
79 });
80 test("same catalog on two lines supports either exact paired lot", () => {

```

tests/engine.test.ts (continued)

```

81   const n = {
82     ...sampleNotice,
83     scope: [
84       ...sampleNotice.scope,
85       {
86         catalog: "2C8632",
87         lots: ["OTHER"],
88         allLots: false,
89         evidence: "2C8632 OTHER",
90       },
91     ],
92   };
93   assert.equal(matchItem(item({ lot: "OTHER" }), n).kind, "exact");
94 });
95 test("extractor keeps product-to-lot pairs intact", () => {
96   const s = extractScope("Catalog: A; Lots: 111\nCatalog: B; Lots: 222");
97   assert.deepEqual(
98     s.map((r) => [r.catalog, r.lots]),
99     [
100       ["A", ["111"]],
101       ["B", ["222"]],
102     ],
103   );
104 });
105 test("extractor abstains on ranges and exclusions", () => {
106   for (const s of ["1 through 99", "1 to 99", "all lots except 3", "1..99"])
107     assert.equal(extractScope("Catalog: X; Lots: " + s).length, 0);
108 });
109 test("source validation refuses fabricated or substring identifiers", () => {
110   const n = {
111     manufacturer: "Baxter",
112     text: "Baxter Catalog: A; Lots: 1111",
113     scope: [
114       {
115         catalog: "A",
116         lots: ["111"],
117         allLots: false,
118         evidence: "Catalog: A; Lots: 1111",
119       },
120     ],
121   };
122   assert.match(validateScope(n) ?? "", /exactly/);
123   n.scope[0].evidence = "made up";
124   assert.match(validateScope(n) ?? "", /verbatim/);
125 });
126 test("all-lot scope needs clear source wording and no exclusions", () => {
127   const n = {
128     manufacturer: "Baxter",
129     text: "Baxter A all lots except X",
130     scope: [
131       {
132         catalog: "A",
133         lots: [],
134         allLots: true,
135         evidence: "Baxter A all lots except X",
136       },
137     ],
138   };
139   assert.match(validateScope(n) ?? "", /unambiguous|match|pair/);
140 });
141 test("CSV preserves missing identifiers and quantities", () => {
142   const data = parseCsv(SAMPLE_CSV);
143   assert.equal(data.length, 8);
144   assert.equal(
145     data.reduce((a, i) => a + i.quantity, 0),
146     218,
147   );
148   assert.equal(data[2].lot, "");
149 });
150 test("CSV supports BOM, CRLF, quoted commas and embedded newlines", () => {
151   const data = parseCsv(
152     '\uFEFFproduct,manufacturer,catalog,lot,quantity,location\r\n"Device, deluxe",Maker,C,L,2,"Room\nA"',
153   );
154   assert.equal(data[0].product, "Device, deluxe");
155   assert.equal(data[0].location, "Room\nA");
156 });
157 test("CSV rejects malformed, duplicate, oversized and invalid inputs", () => {
158   assert.throws(() => parseCsv("product,product\nx,y"), /duplicate/);
159   assert.throws(
160     () => parseCsv(SAMPLE_CSV + "\n" + SAMPLE_CSV.split("\n")[1]),

```

tests/engine.test.ts (continued)

```

161     /duplicate/,
162   );
163   assert.throws(
164     () => parseCsv(SAMPLE_CSV.replace(",24," , "0,")),
165     /whole number/,
166   );
167   assert.throws(
168     () => parseCsv(SAMPLE_CSV.replace(",24," , "1.2,")),
169     /whole number/,
170   );
171   assert.throws(() => parseCsv(SAMPLE_CSV + '\n"broken'), /unclosed/);
172   assert.throws(() => parseCsv("x".repeat(500001)), /limit/);
173 });
174 test("CSV exports neutralize formulas and quote unsafe values", () => {
175   const s = exportCsv([
176     ['=HYPERLINK("evil")', " +1", "@SUM(A1)", "-2", "normal, value"],
177   ]);
178   assert.ok(s.includes('=HYPERLINK'));
179   assert.ok(s.includes(' +1'));
180   assert.ok(s.includes('@SUM'));
181   assert.ok(s.includes('normal, value'));
182 });
183 test("unapproved source blocks direct action mutation", () =>
184   assert.throws(
185     () => reduceWorkspace(fixture(), action(), "Attacker"),
186     /source/,
187   );
188 test("approval actor and timestamp are server inputs, extra client fields rejected", () => {
189   assert.throws(() =>
190     reduceWorkspace(
191       fixture(),
192       {
193         type: "approve_notice",
194         noticeId: "baxter-demo",
195         sourceConfirmed: true,
196         approvedBy: "Doctor",
197       },
198       "Real user",
199     ),
200   );
201   const s = approve();
202   assert.equal(s.notices[0].approvedBy, "Test reviewer");
203 });
204 test("assigned response is version-bound and attributed", () => {
205   const s = reduceWorkspace(approve(), action(), "Operator");
206   assert.equal(s.actions[0].updatedBy, "Operator");
207   assert.equal(activeAction(s, "baxter-demo", "sample-1").status, "assigned");
208 });
209 test("cannot claim handled units before verification", () =>
210   assert.throws(
211     () =>
212       reduceWorkspace(approve(), action({ handledQuantity: 1 }), "Operator"),
213     /Verify/,
214   );
215 test("uncertain identifiers cannot be verified or completed", () =>
216   assert.throws(
217     () =>
218       reduceWorkspace(
219         approve(),
220         action({
221           itemId: "sample-3",
222           status: "completed",
223           verification: "Checked label and source",
224           evidence: "Return record Q100",
225           handledQuantity: 8,
226         }),
227         "Operator",
228       ),
229     /missing identifiers/,
230   );
231 test("completion rejects absent evidence, partial counts and excessive counts", () => {
232   const a = {
233     status: "completed",
234     verification: "Checked physical label and current letter",
235     handledQuantity: 24,
236   };
237   assert.throws(
238     () => reduceWorkspace(approve(), action(a), "Operator"),
239     /evidence/,
240   );

```

tests/engine.test.ts (continued)

```

241   assert.throws(
242     () =>
243       reduceWorkspace(
244         approve(),
245         action({
246           ...a,
247           evidence: "Return receipt RMA-100",
248           handledQuantity: 20,
249         }),
250         "Operator",
251       ),
252     /all recorded/,
253   );
254   assert.throws(
255     () =>
256       reduceWorkspace(
257         approve(),
258         action({
259           ...a,
260           evidence: "Return receipt RMA-100",
261           handledQuantity: 25,
262         }),
263         "Operator",
264       ),
265     /exceed/,
266   );
267 });
268 test("partial handling stays verified and full handling can complete", () => {
269   const partial = reduceWorkspace(
270     approve(),
271     action({
272       status: "verified",
273       verification: "Checked label and current notice",
274       evidence: "Isolated under Q-100",
275       handledQuantity: 12,
276     }),
277     "Operator",
278   );
279   assert.equal(partial.actions[0].status, "verified");
280   const done = reduceWorkspace(
281     partial,
282     action({
283       status: "completed",
284       verification: "Checked label and current notice",
285       evidence: "Return receipt RMA-100",
286       handledQuantity: 24,
287     }),
288     "Operator",
289   );
290   assert.equal(done.actions.length, 1);
291   assert.equal(done.actions[0].status, "completed");
292   assert.equal(done.audit.length, 4);
293 });
294 test("source edit invalidates approval and earlier responses", () => {
295   const s = reduceWorkspace(approve(), action(), "Operator");
296   const { id, title, manufacturer, date, sourceUrl, text, scope, training } =
297     s.notices[0];
298   const next = reduceWorkspace(
299     s,
300     {
301       type: "save_notice",
302       notice: {
303         id,
304         title,
305         manufacturer,
306         date,
307         sourceUrl,
308         text,
309         scope,
310         training,
311         action:
312           "Amended response summary from current manufacturer instructions.",
313       },
314     },
315     "Editor",
316   );
317   assert.equal(next.notices[0].approvedAt, null);
318   assert.equal(next.notices[0].version, 2);
319   assert.equal(activeAction(next, id, "sample-1"), undefined);
320   assert.equal(next.actions.length, 1);

```

tests/engine.test.ts (continued)

```

321 });
322 test("inventory replacement cannot inherit completed responses", () => {
323   const s = reduceWorkspace(approve(), action(), "Operator");
324   const next = reduceWorkspace(
325     s,
326     { type: "import_inventory", csv: SAMPLE_CSV },
327     "Editor",
328   );
329   assert.equal(activeAction(next, "baxter-demo", "sample-1"), undefined);
330   assert.equal(next.inventoryVersion, 2);
331   assert.equal(next.actions.length, 1);
332   assert.equal(
333     activeAction(next, "baxter-demo", next.inventory[0].id),
334     undefined,
335   );
336 });
337 test("reject unsafe source URLs and impossible dates", () => {
338   const { id, approvedAt, approvedBy, version, ...n } = sampleNotice;
339   assert.throws(() =>
340     reduceWorkspace(
341       fixture(),
342       {
343         type: "save_notice",
344         notice: { ...n, sourceUrl: "javascript:alert(1)" },
345       },
346       "Editor",
347     ),
348   );
349   assert.throws(() =>
350     reduceWorkspace(approve(), action({ dueDate: "2026-02-30" }), "Editor"),
351   );
352 });
353 test("unknown commands and actions on unrelated inventory are rejected", () => {
354   assert.throws(() =>
355     reduceWorkspace(fixture(), { type: "admin_override" }, "Attacker"),
356   );
357   assert.throws(
358     () =>
359     reduceWorkspace(
360       approve(),
361       action({ itemId: "other-user-item" }),
362       "Attacker",
363     ),
364     /not found/,
365   );
366   assert.throws(
367     () =>
368     reduceWorkspace(approve(), action({ itemId: "sample-8" }), "Operator"),
369     /no listed/,
370   );
371 });
372
373 test("reject cross-pair support spanning two products", () => {
374   const n = {
375     manufacturer: "Baxter",
376     text: "Baxter\nCatalog: A; Lots: 111\nCatalog: B; Lots: 222",
377     scope: [
378       {
379         catalog: "A",
380         lots: ["222"],
381         allLots: false,
382         evidence: "Catalog: A; Lots: 111\nCatalog: B; Lots: 222",
383       },
384     ],
385   };
386   assert.notEqual(validateScope(n), null);
387 });
388 test("qualified all-lot statements are unsupported", () => {
389   for (const qualifier of [
390     "manufactured before January 2024",
391     "within serial range 10-20",
392     "shipped after 2025",
393   ]) {
394     const evidence = "Catalog: A; Lots: all lots " + qualifier;
395     assert.notEqual(
396       validateScope({
397         manufacturer: "Baxter",
398         text: "Baxter\n" + evidence,
399         scope: [{ catalog: "A", lots: [], allLots: true, evidence }],
400       }),

```

tests/engine.test.ts (continued)

```

401     null,
402   );
403 }
404 });
405 test("manufacturer aliases remain in attention queue", () =>
406   assert.equal(
407     matchItem(
408       item({ manufacturer: "Baxter Healthcare Corporation" }),
409       sampleNotice,
410       0.99,
411     ).kind,
412     "missing",
413   );
414 test("single-line correction preserves other actions and prior inventory snapshot", () => {
415   let s = reduceWorkspace(approve(), action(), "Operator");
416   const other = s.inventory[2];
417   s = reduceWorkspace(
418     s,
419     {
420       type: "correct_item",
421       id: other.id,
422       product: other.product,
423       manufacturer: other.manufacturer,
424       catalog: other.catalog,
425       lot: "R25C31031",
426       quantity: other.quantity,
427       location: other.location,
428     },
429     "Editor",
430   );
431   assert.ok(activeAction(s, "baxter-demo", "sample-1"));
432   assert.equal(s.inventoryHistory[0].items[2].lot, "");
433   assert.equal(s.inventory[2].lot, "R25C31031");
434 });
435 test("source history preserves original evidence after edits", () => {
436   const current = approve();
437   const { approvedAt, approvedBy, version, ...n } = current.notices[0];
438   const next = reduceWorkspace(
439     current,
440     {
441       type: "save_notice",
442       notice: { ...n, action: "A revised staff-entered response summary." },
443     },
444     "Editor",
445   );
446   assert.equal(next.noticeHistory[0].text, current.notices[0].text);
447   assert.equal(next.noticeHistory[0].approvedBy, "Test reviewer");
448 });

```

vite.config.ts

```

1 import vinext from "vinext";
2 import { defineConfig } from "vite";
3 import hostingConfig from "../openai/hosting.json";
4 import { readExecutionProfile } from "../scripts/execution-profile.mjs";
5 import { sites } from "../build/sites-vite-plugin";
6
7 const SITE_CREATOR_PLACEHOLDER_DATABASE_ID =
8   "00000000-0000-4000-8000-000000000000";
9
10 const { d1, r2 } = hostingConfig;
11
12 // macOS Seatbelt blocks FSEvents, so Codex previews need polling for HMR.
13 const isCodexSeatbeltSandbox = process.env.CODEX_SANDBOX === "seatbelt";
14 const managedLinux = readExecutionProfile() === "managed-linux";
15
16 const localBindingConfig = {
17   main: "vinext/server/fetch-handler",
18   compatibility_flags: ["nodejs_compat"],
19   d1_databases: d1
20   ? [
21     {
22       binding: d1,
23       database_name: "site-creator-d1",
24       database_id: SITE_CREATOR_PLACEHOLDER_DATABASE_ID,
25     },
26   ]
27   : [],
28   r2_buckets: r2
29   ? [
30     {
31       binding: r2,
32       bucket_name: "site-creator-r2",
33     },
34   ]
35   : [],
36 };
37
38 export default defineConfig(async () => {
39   // Use Miniflare's local Request.cf placeholder unless fetching is requested.
40   process.env.CLOUDFLARE_CF_FETCH_ENABLED ??= "false";
41   process.env.WRANGLER_SEND_METRICS ??= "false";
42
43   // Keep Wrangler and Miniflare state project-local. These are non-secret tool
44   // settings; application environment belongs in ignored `.env*` files.
45   process.env.WRANGLER_WRITE_LOGS ??= "false";
46   process.env.WRANGLER_LOG_PATH ??= ".wrangler/logs";
47   process.env.WRANGLER_REGISTRY_PATH ??= ".wrangler/dev-registry";
48   process.env.MINIFLARE_REGISTRY_PATH ??= ".wrangler/registry";
49
50   // Wrangler snapshots its log path while the Cloudflare plugin is imported.
51   const { cloudflare } = await import("@cloudflare/vite-plugin");
52
53   return {
54     server: {
55       ...(managedLinux ? { host: "0.0.0.0", allowedHosts: ["terminal.local"] } : {}),
56       ...(isCodexSeatbeltSandbox ? { watch: { useFsEvents: false, usePolling: true } } : {}),
57     },
58     plugins: [
59       vinext(),
60       sites({ mockAuth: !managedLinux }),
61       cloudflare({
62         viteEnvironment: { name: "rsc", childEnvironments: ["ssr"] },
63         inspectorPort: false,
64         config: localBindingConfig,
65       }),
66     ],
67   };
68 });

```